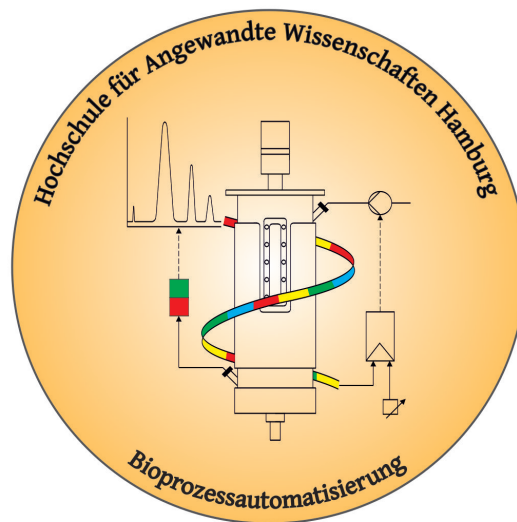

Conceptualization of a Simulation System for Testing Fermentation Processes and the Implementation of a *Pichia pastoris* Model



Hamburg University of Applied Sciences
Faculty of Life Sciences
Department of Biotechnology
Master's Program in Pharmaceutical Biotechnology
Master's Thesis

Philipp Yuma Iff
2385958

Supervisors:
Prof. Dr. Gesine Cornelissen (HAW Hamburg)
Prof. Dr. Christian Kaiser (HAW Hamburg)
Hamburg, 29.11.2024

Abstract

Simulation-based methods are becoming increasingly important in biotechnology research and production to optimize fermentation processes and scale them up to industrial standards. The aim of this work was to develop a software platform allowing the simulation of various organisms. In the process of that, the methylotrophic organism *Pichia pastoris* was implemented. The mechanistic model, based on mathematical equations, accurately describes the biological and chemical processes in the bioreactor. A relational database system has been integrated as a central data repository, allowing seamless data management and supporting eventual linking to real process data. Emphasis is placed on the software implementation of the simulation in a MATLAB® environment, complemented by an interactive user interface that simplifies process control and visualization of results. The work builds on earlier research projects, in particular the *Escherichia coli* simulation by Lena Sophia Kaletsch and the initial concept by Prof. Dr.-ing. Luttmann. During the development of the revised software architecture, care was always taken to create a basis for future migration to a Digital Twin. To test the newly implemented *P. pastoris* model and process control, a model fermentation was performed according to the standard cultivation strategy for this organism. Results show that the developed model can realistically simulate complex fermentation dynamics of *P. pastoris*. The implementation of the database structure supports flexible and robust management of the simulation data and allows for a prospective extension for applications in the context of Industry 4.0.

Danksagung

An dieser Stelle möchte ich mich ganz herzlich bei Prof. Dr. Gesine Cornelissen für die Möglichkeit bedanken, meine Masterarbeit im Labor für Bioprozessautomatisierung durchführen zu dürfen. Zudem danke ich ihr für die jahrelange Unterstützung während meines Studiums.

Mein besonderer Dank gilt auch Dipl.-Ing. Ulrich Scheffler für die Betreuung meiner Arbeit sowie für die wertvollen Tipps und Ideen, die maßgeblich zum Gelingen dieser Arbeit beigetragen haben.

Ebenso bedanke ich mich bei Prof. Dr. Christian Kaiser für die freundliche Übernahme der Rolle als Zweitgutachter.

Ich danke meinen Eltern und meiner Schwester für ihre unermüdliche Unterstützung in all den Jahren meines Studiums. Im Kontext dieser Masterarbeit möchte ich vor allem meiner Freundin Anna danken, die mir in den vergangenen sechs Monaten bedingungslos zur Seite stand. Sie hat mich nicht nur in allen Belangen unterstützt, sondern auch stets Verständnis gezeigt, wenn ich mal wieder bis spät in die Nacht am Programmcode gearbeitet habe.

Contents

1	Introduction	1
2	Technical Background	5
2.1	Software Architecture	5
2.2	The Model Organism <i>Pichia pastoris</i>	7
2.2.1	AOX Induction	8
2.2.2	Target Protein Expression	15
2.2.3	Protein Secretion	19
2.2.4	Methanol Toxicity	20
2.3	Process Automation Principles	22
2.3.1	MFCs/win	22
2.3.2	Feeding Strategies	23
3	Methods	27
3.1	MATLAB® 2024a	27
3.1.1	MATLAB® App Designer	27
3.1.2	Dynamic Properties	28
3.1.3	Timer Function	29
3.1.4	ODE Solver	30
3.2	Database	30
3.2.1	Structured Query Language (SQL)	31
3.2.2	SQLite Database	31
3.2.3	Normalization	32
3.2.4	Trigger	32
3.3	MATLAB® Database Communication	33
3.4	Parameter and Variable Pool	34
3.5	Model Process Strategy	36
4	Results and Discussion	38
4.1	Front-End	39
4.1.1	Starting Screen	39
4.1.2	Control App	41

4.1.3	Figure App	43
4.1.4	Data Table	44
4.1.5	Project Export	45
4.1.6	Process End	46
4.1.7	Process Automation	46
4.2	Back-End	48
4.2.1	SQLite Database	48
4.2.2	Calculation App	51
4.3	Model Implementation	56
4.4	Exemplary Process	57
4.4.1	Fully Automated Model Process	57
4.4.2	Evaluation AOX Induction and Target Protein Production	59
4.4.3	Methanol Toxicity	62
4.4.4	Limitations of the Platform Concept	63
5	Summary and Outlook	64
6	Appendix	74
6.1	MATLAB® Code	74
6.1.1	MATLAB Model Implementation	74
6.2	Dropbox	76
6.3	Database Schemes	76
6.4	Simulation Process Information	80
6.5	<i>P. pastoris</i> Model Script	81
6.6	<i>P. pastoris</i> Parameter Table	103

List of Figures

2.1	General concept of the simulation platform	7
2.2	Schematic of the regulation of cellular AOX formation in the <i>P. pastoris</i> model	9
2.3	Schematic of the regulation of cellular target protein formation in the <i>P. pastoris</i> model	16
2.4	Exemplary MFCS/win recipe of a batch phase	23
2.5	Variable editing in a Parameter Phase in MFCS/win	24
2.6	Variable condition for a phase transition in MFCS/win	24
3.1	Data structure in the BIOSIM software	35
3.2	Standard fermentation strategy conducted in BPA laboratory	37
4.1	Software architecture overview	38
4.2	Starting Screen and Project Creator GUIs	40
4.3	Load Project GUI	41
4.4	Model Configurator GUI	42
4.5	Illustration of implemented parameter organization	43
4.6	Control App GUI	44
4.7	Figure App GUI	45
4.8	Project Export GUI	46
4.9	Closing Screen GUI	47
4.10	Flow diagram of the Calculation App and its interfaces	52
4.11	Flow diagram of the calculationFcn()	54
4.12	Plot of the simulated process	58
4.13	AOX induction in a simulated <i>P. pastoris</i> process	59
4.14	Target protein formation in a simulated <i>P. pastoris</i> process	61
4.15	Methanol toxicity in a simulated <i>P. pastoris</i> process	62
6.1	Dropbox QR code	76
6.2	ER diagram of simulation related tables	77
6.3	ER diagram of process control related tables	78
6.4	ER diagram of plotting related tables	79
6.5	Process panels of discussed simulation	80

List of Tables

6.1 Model Parameters 104

List of Abbreviations

AOX	:=	Alcohol Oxidase
AOX1	:=	Alcohol Oxidase Gene 1
AOX2	:=	Alcohol Oxidase Gene 2
BPA	:=	Laboratory for Bioprocess Automation
DBMS	:=	Database Management System
DCU	:=	Digital Control Unit
DDL	:=	Data Definition Language
DML	:=	Data Manipulation Language
DCL	:=	Data Control Language
GFP	:=	Green Fluorescent Protein
GUI	:=	Graphical User Interface
MFCS/win	:=	Multi Fermentor Control System for Windows
Mut ⁺	:=	Methanol Utilization Plus (<i>P. pastoris</i> phenotype)
Mut ⁻	:=	Methanol Utilization Minus (<i>P. pastoris</i> phenotype)
Mut ^S	:=	Methanol Utilization Slow (<i>P. pastoris</i> phenotype)
ODE	:=	Ordinary Differential Equation
OTS	:=	Operational Training Simulation
<i>P. pastoris</i>	:=	<i>Pichia pastoris</i> (yeast organism)
PID	:=	Proportional-Integral-Derivative (controller)
SQL	:=	Structured Query Language
UI	:=	User Interface

Nomenclature

$a_{P1script}$	:=	Transcription factor for target protein	$[h^{-1}]$
$a_{S2script}$	:=	Transcription factor for AOX induction	$[h^{-1}]$
$b_{P1script}$	:=	Transcription amplification factor for target protein	$[h^{-1}]$
$b_{S2script}$	:=	Transcription amplification factor for AOX induction	$[h^{-1}]$
c_{P1L}	:=	Target protein concentration in liquid phase	$[g\ l^{-1}]$
c_{P1X}	:=	Intracellular target protein concentration	$[g\ l^{-1}]$
c_{S1L}	:=	Glycerol concentration in liquid phase	$[g\ l^{-1}]$
c_{S1R1}	:=	substrate concentration in reservoir 1	$[g\ l^{-1}]$
c_{S2L}	:=	Methanol concentration in liquid phase	$[g\ l^{-1}]$
c_{XL}	:=	cell concentration in liquid phase	$[g\ l^{-1}]$
c_{XLj}	:=	cell concentration in the beginning of the fed-batch phase	$[g\ l^{-1}]$
$F_{I_{max}}$	:=	maximum capacity of the pump of reservoir $I = R1, R2, R3$	$[g\ l^{-1}]$
F_{nG}	:=	total aeration rate	$[l\ min^{-1}]$
F_{R1}	:=	feeding rate from reservoir 1	$[l\ h^{-1}]$
F_{R1j}	:=	initial feeding rate at t_j	$[l\ h^{-1}]$
k_I	:=	utilization factor of the pump of reservoir $I = R1, R2, R3$	$[-]$
k_{I21}	:=	Methanol-glycerol inhibition constant	$[g\ l^{-1}]$
k_{I22}	:=	Methanol-methanol inhibition constant	$[g\ l^{-1}]$
$k_{P1\alpha}$	:=	Rate constant for target protein secretion	$[h^{-1}]$
k_{S2ind}	:=	Methanol induction turn-on concentration	$[g\ l^{-1}]$
k_{S2tox}	:=	Methanol toxicity turn-on concentration	$[g\ l^{-1}]$
N_{St}	:=	stirrer speed	$[min^{-1}]$
μ	:=	Cell-specific growth	$[h^{-1}]$
μ_{1w}	:=	set growth rate for substrate 1	$[h^{-1}]$
μ_{2max}	:=	Maximum specific growth rate on methanol	$[h^{-1}]$
κ_{ind}	:=	Order of the methanol induction mechanism	$[-]$
κ_{tox}	:=	Order of the methanol toxicity mechanism	$[-]$
$q_{P1/X}$	:=	Cell-specific target protein formation rate	$[h^{-1}]$
Q_{S1in}	:=	volumetric glycerol intake rate	$[g(l\ h)^{-1}]$
Q_{S2in}	:=	volumetric methanol intake rate	$[g(l\ h)^{-1}]$
$q_{S1/Xm}$	:=	cell-specific maintenance substrate uptake rate	$[h^{-1}]$

$q_{S2/X}$	:=	Cell-specific methanol uptake rate	$[h^{-1}]$
$q_{X/X}$	:=	Cell-specific growth rate	$[h^{-1}]$
$q_{X/Xm}$	:=	Maintenance part of the cell-specific growth rate	$[h^{-1}]$
$q_{X/Xtox}$	:=	Methanol toxicity part of the cell-specific death rate	$[h^{-1}]$
t	:=	Process time	$[h]$
t_j	:=	fed-batch start time	$[h]$
V_{Lj}	:=	volume in the beginning of the fed-batch phase	$[l]$
$y_{X/S1gr}$	:=	Growth yield on glycerol	$[-]$
$y_{X/S2gr}$	:=	Growth yield on methanol	$[-]$

1 Introduction

Biopharmaceuticals are playing an increasingly important role in the pharmaceutical industry. Although biopharmaceuticals account for only 10% of the drug market, they dominated 2023's top-selling drugs, generating \$141.3 billion in sales [5, 7]. The production of these relatively large, complex molecules is both complicated and costly. Most drugs are small molecules typically produced synthetically, whereas biopharmaceuticals are mainly derived from biological sources and require a costly purification and isolation process from the host [22]. It is essential to ensure the production process of this valuable product functions efficiently.

In 2014, it was estimated that 50% of deviations in biopharmaceutical production were due to human error [50]. Improving the training of operators is one method of reducing deviations in a production process. Training on an actual bioreaction process is often time consuming and expensive. Operational training simulation (OTS) serves as an alternative to training on a real process [14]. Virtual simulators have become essential for training in industries where safety is critical. These include the aviation industry, the performance of medical operations, or the use of weapons in combat [3, 27, 25]. The application of OTS in bioprocessing is useful for simulating critical and complex scenarios within fermentation processes. These conditions can be replicated many times without the need to set up a real fermentation process, which takes several days.

In 2013, Gerlach et al. examined the training effects of OTS on the simulation of *Saccharomyces cerevisiae* for bioethanol production and *Escherichia coli* for green fluorescence protein (GFP) production. The multi-shell model platform comprised various sub-models, including a plant-wide model, a bioreactor model, and both physicochemical and biological sub-models. The training simulator utilized was the Bio-ProcessTrainer application, which integrated the submodels as dynamic link libraries (DLLs) within WinErs [14]. WinErs (Ingenieurbüro Dr.-Ing. Schoop GmbH, Hamburg, Germany) represents a modular process control and simulation system employed in various OTS methodologies [4, 15, 16]. A multiple choice test was conducted before and after the training, to assess the effectiveness of the training. The results of the study show that OTS significantly improves process operation skills [14].

OTS solutions are already being offered in industry that allow training for the product offered. Sartorius Stedim Biotech GmbH, located in Göttingen, Germany, produces bioreactor systems and provides an interactive software designed for training in bioprocess control. The BIOSTAT® T software enables users to navigate the menu of a Sartorius digital control unit effectively [18]. Users have the option to select either mammalian or yeast cell cultivation. The software retrieves process data from an actual fermentation and employs algorithms to generate cell growth curves from this data. The user can subsequently regulate the process conditions and identify optimal growth parameters. A case study by Cotoras utilized the BIOSTAT® T yeast software to examine the efficacy of bioreactor computer simulations within a hybrid classroom and online learning context. Initially, an instructor input parameter data into the software, and the students analyzed the software's response as part of a frontal teaching session. In a subsequent simulation run, the students had the opportunity to operate the software independently under the instructor's supervision. The survey results indicated that students considered the independent operation of the simulation software positively, resulting in a notable increase in motivation [9]. This suggests that interactive bioprocess simulation software can lead to beneficial learning outcomes.

In the early 2000s, Prof. Dr.-Ing. Luttmann developed an online simulation system utilizing mathematical models, named BIOSIM, at the Hamburg University of Applied Sciences (HAW) [29]. The simulation system functions as independent simulation software on a process computer and can also interface with the process control system UBICON (esd - electronic system design GmbH, Hannover, Germany) through a digital control unit (DCU). This enables the user to select either a fully emulated simulation or one linked to actual hardware. The variant associated with the actual hardware served as a training tool for the Bioprocess Automation Special Course within the master's program in Pharmaceutical Biotechnology at HAW until 2023. The mathematical model of the BIOSIM software was updated to reflect recent technological advancements and translated into the programming language MATLAB® by Lena Sophia Kaletsch as part of her master's thesis [23].

Simulation models created for application in OTS aim to closely replicate real-world conditions. Such models serve as digital twins of their real, physical counterparts. The concept of Digital Twins plays an increasingly significant role in *in silico* process development and process optimization [46]. A virtual representation of a real bioprocess, initially created for OTS purposes, can be considered an early-stage Digital Twin [21].

Digital Twins can use different types of models. Typically, a distinction is made between mechanistic, non-mechanistic, and hybrid models. Mechanistic models provide mathematical descriptions of the mechanical, physical, chemical, and biochemical structures of a process, while non-mechanistic models rely solely on data-driven approaches. Hybrid models combine the advantages of both mechanistic and non-mechanistic approaches. Mechanistic models have the advantage of providing mathematical explanations for all developments in the simulation. This allows for predictions about the future course of the process. Non-mechanistic models can be viewed as black boxes since the progress of the simulation is solely dependent on mathematical correlations. Data from established processes are reproduced by experiments [21]. The advantage is the reduced time needed to develop a non-mechanistic model [49]. However, predictions about the future course of the process are limited because they can only account for situations that have already occurred and are represented in the data [17]. Implementation of a hybrid model requires careful integration of both approaches to ensure consistency and accuracy. Practical challenges include calibration and validation of the model, handling of uncertainties, and ensuring robustness and generalizability in different biotechnological applications [30].

Digital Twins are playing a promising role in the digitalization of bioprocessing and are referred to as the heart of the digital factory [20]. In the era of Industry 4.0, where the Internet of Things (IoT) connects all systems and machine learning is widely implemented, Digital Twins serve as a crucial interface between data and models. They have the potential to evolve into a smart digital ecosystem, integrating real-time data and predictive models to enhance bioprocessing.

The aim of this thesis was to develop a software platform for fermentation simulations of different organisms. These simulations should allow the user to interactively learn about fermentation technology, but also to develop a process strategy and test its possible outcomes. In addition to the use of the platform as simulation software, it should also be possible to use it as a tool for process optimization. Perspectively, it should be able to read and process data from a real process in real time, and not just serve as a database for standardized data, but also be refined into a Digital Twin. The basis of the simulation software is the bioprocess simulation developed by Lena Sophia Kaletsch [23], in which the *E. coli* model developed by Prof. Dr. Luttmann was implemented in MATLAB®. In addition, *P. pastoris* is introduced as a new organism into this simulation platform, which is often cited as a eukaryotic model system in the Laboratory of

Bioprocess Automation (BPA).

2 Technical Background

2.1 Software Architecture

The architecture of a software plays a fundamental role as it defines the core structures and principles. Therefore, decisions made at the beginning of software development have a significant impact on the scalability and maintainability of the software in the future [48].

For the further development of the BIOSIM software, the existing software architectures of BIOSIM were initially reviewed and analyzed for development potential and limitations.

In the manual of the simulation software BIOSIM by Luttmann, two different ways of organizing the software have been distinguished. In the so-called stand-alone architecture, the automation hardware and the process control system of a real digital control unit (DCU) are used and optionally connected to a real process (bioreactor) or its simulation via electrical interfaces. The simulation runs independently on a computer, allowing the user to switch between the real process and the simulation at will. This flexibility allows verification of the functionality or validation of the automation system before it is applied to real processes [29]. This approach allows trainees to interact directly with the hardware used in real processes, practice specific operations, and promises optimal training results. However, scalability is a major drawback due to limited and costly automation hardware, which makes it challenging to keep up with the latest technology, resulting in training with outdated equipment.

The second variant takes a different approach, where the BIOSIM simulation is integrated into the UBICON process control system and runs parallel on a process computer. Communication occurs through a shared memory area, known as the UBICON data pool. The advantage of this approach for training purposes is that acquiring new computers is significantly cheaper than purchasing additional real-process hardware. Moreover, the BIOSIM software can be continuously updated without the need for complete replacement. However, the training outcome may not be entirely comparable to

training with real hardware systems as used in the industry.

The BPA laboratory at HAW has been pursuing the stand-alone architecture approach for BIOSIM for a long time, as the hardware was available. Since both the software and hardware of this setup became outdated, a new simulation software was developed by Lena Sophia Kaletsch in 2023 as part of her master's thesis, based on recent technological advancements [23]. In this new software, the simulation and control unit were integrated and could be operated via a graphical user interface (GUI). The software used for this work was MATLAB®2022b, with the BIOSIM model translated into MATLAB® and compiled into a functional integrated simulation application. This software was successfully used for two years in the BPA Special Course for the "Reaction Kinetics" experiment.

The objective of this thesis is to add a new organism as a second model to be cultured by simulation alongside the *E. coli* based model. The first MATLAB®-based version of BIOSIM was tailored to the *E. coli* model, which did not allow the implementation of a second model. All parameters were defined in the code as properties, with certain initial values adjustable by the user. The model was tightly embedded in the software, which limited scalability. In addition, all data generated in the process was cached locally in MATLAB® and exported at the end of the process.

The most apparent approach for establishing a new model would be to develop a second application similar to the first, where parameter values would be simply adjusted to fit into the new organism. The mathematical functions of the model would only need to be slightly modified and only a few new model aspects added. However, since this approach would require a new application for each new model and has scalability limitations, a different strategy was pursued in this work.

To allow multiple organism models to be accessible with one software multiple changes within the software architecture have to be conducted. For Version 2.0 of the bioprocess simulation software, a central database is to be implemented, where all changeable information is organized and stored. The software architecture chosen here is intended to lay the foundation for the later use of the software as a tool for planning and optimizing fermentation processes, i.e. a kind of Digital Twin.

The plan is to restructure the software previously used as an application for the simulation of *E. coli* into a platform that allows several organisms to be simulated. In Figure

2.1, the intended concept for the software platform is illustrated. The focus here is on a modular software architecture designed to facilitate the addition of further components. To ensure that all interfaces have access to up-to-date information, structured storage and organization should be the top priority in this data-oriented approach. The database is therefore at the center between the user interface (UI) and the script, which is responsible for generating simulation data.



Figure 2.1: General concept of the simulation platform

In this version, all structures specific to individual organisms need to be removed from the main UI to allow the software to support a broader range of organisms. The structures have now been organized into separated, callable functions files. Parameters and variables are defined in the database and linked to an organism through relational keys. These are then read out depending on the organism and handled as local properties in the simulation software.

2.2 The Model Organism *Pichia pastoris*

In the context of this work, a new model organism is being integrated into the simulation platform: the yeast *Pichia pastoris*, also known as *Komagataella phaffii*. This eukaryotic microorganism is a prominent host for the production of heterologous proteins due to its ability to carry out post-translational modifications and to secrete the target protein [12, 24]. *P. pastoris* distinguishes itself as a versatile host for recombinant protein production due to its robust genetic toolbox, which is largely enabled by its similarity to the well-studied yeast *Saccharomyces cerevisiae* [11].

P. pastoris is a methylotrophic yeast, which means that it can use methanol as its sole

carbon source. This property is particularly advantageous for recombinant protein production, as methanol metabolism in *P. pastoris* driven by a tightly regulated expression system. The genes encoding the alcohol oxidases *AOX1* and *AOX2*, which catalyse the oxidation of methanol to formaldehyde, are under the control of strong promoters. While AOX is undetectable in the presence of preferred substrates such as glucose or glycerol, it accounts for up to 35% of total cellular protein when *P. pastoris* is grown on methanol as the sole substrate [10].

There is a wide range of *P. pastoris* expressing host strains, all derived from the wild-type strain NRRL-Y 11430 (Northern Regional Research Laboratories, Peoria, IL) [11]. One feature that distinguishes the different phenotypes is the extent to which the strains are capable of utilizing methanol. While the Mut⁺ (methanol utilization plus) phenotype possesses both the stronger *AOX1* gene and the *AOX2* gene, the Mut⁻ (methanol utilization minus) phenotype lacks both genes. Thus, lines with this phenotype are no longer able to grow on methanol. The third phenotype, Mut^S (methanol utilization slow), lacks the *AOX1* gene and therefore relies on the weaker *AOX2* gene for growth on methanol [11].

For several decades, *P. pastoris* has been utilized as a host organism in the BPA laboratory at HAW. This is largely due to its ability to grow at high cell densities and secrete proteins, which significantly simplifies product purification [8, 40, 47, 6]. In addition, this organism is the subject of the interdisciplinary research project "PharmCycle" as a potential host for the production of an antimicrobial peptide as an alternative to antibiotics to reduce contamination of the aquatic environment [1].

A successful integration of *P. pastoris* into the simulation software requires consideration of the unique characteristics of this organism. Therefore, a model of AOX induction and subsequent expression of target proteins will be presented. In addition, a model will be developed to account for the potential toxicity of methanol and, in particular, its metabolite formaldehyde.

2.2.1 AOX Induction

The basis for the development of the AOX induction model is the production phase of a high cell density cultivation of *P. pastoris*, involving partial bisubstrate feeding, which

was conducted as part of the dissertation by Prof. Dr. Cornelissen in 2004 [8]. The reaction kinetic model was then developed with consideration of genetic mechanisms, based on simple control engineering approaches.

The strain used in the production phase had the Mut^S phenotype, meaning that the foreign gene was located behind the *AOX1* promoter. In this case, the strain expressed the chemokine 1-8del MCP-1. As a result, methanol metabolism was only controlled by the weaker *AOX2* promoter, P_{AOX2} . For model development, this meant that AOX induction and protein expression had to be considered separately. It is described that both models are divided into three consecutive phases. This division is based on the different phases of protein biosynthesis, taking into account their temporal delays.

The first phase represents transcription in the nucleus, while the second phase represents the transport of the formed mRNA from the nucleus to the cytosol and ribosomes. In the third phase, translation occurs, where the mRNA is read by the ribosomes to produce the corresponding protein. This protein is either AOX or the target protein. Figure 2.2 shows the regulatory scheme of the model's intracellular AOX formation.

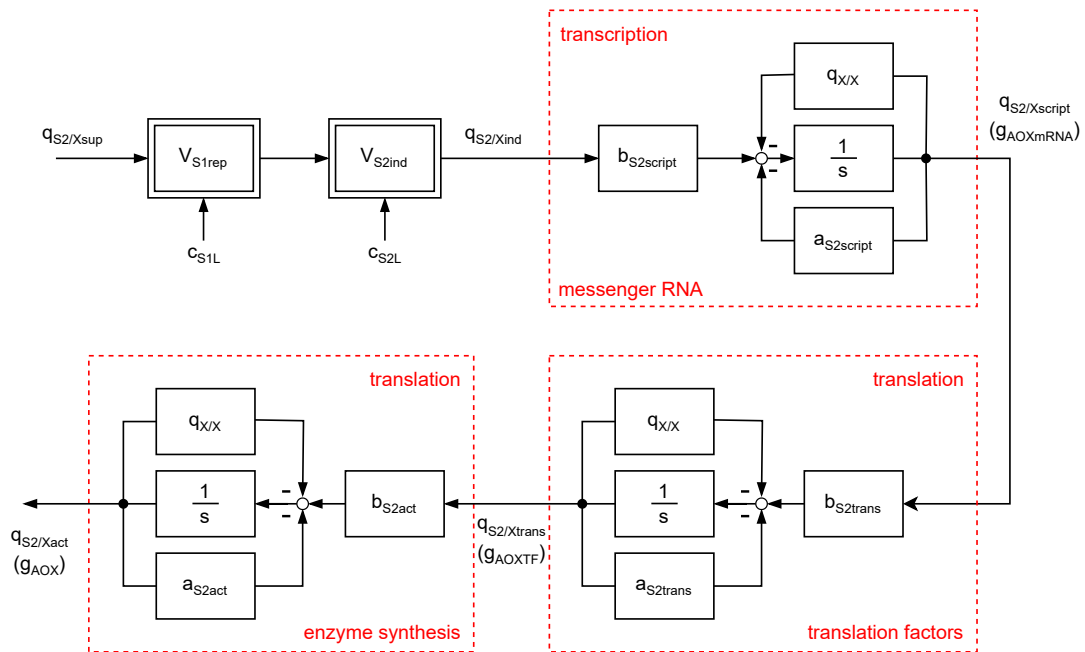


Figure 2.2: Schematic of the regulation of cellular AOX formation in the *P. pastoris* model

AOX formation is controlled by substrate concentration of glycerol and methanol via three first-order metabolic delay systems. Adapted from Cornelissen (2004).

The following section presents the mathematical functions involved in the regulatory scheme. This model considers two substrates: glycerol, which is assigned the index 1, and methanol, which is assigned the index 2. The genetic AOX potential, denoted as $q_{S2/Xsup}$, represents the upper limit of the cell-specific mRNA induction rate, which is represented by $q_{S2/Xind}$,

$$q_{S2/Xind}(t) = V_{S1rep}(c_{S1L}, t) \cdot V_{S2ind}(c_{S2L}, t) \cdot q_{S2/Xsup} \quad (2.1)$$

with:

$q_{S2/Xind}$	:=	cell-specific mRNA induction rate	$[h^{-1}]$
V_{S1rep}	:=	glycerol repression function	$[-]$
c_{S1L}	:=	concentration of glycerol in liquid phase	$[g\ l^{-1}]$
V_{S2ind}	:=	methanol induction function	$[-]$
c_{S2L}	:=	concentration of methanol in liquid phase	$[g\ l^{-1}]$
$q_{S2/Xsup}$	:=	upper limit of methanol induction reaction potential	$[h^{-1}]$
t	:=	process time	$[h]$.

The genetic AOX potential is regulated by the switch-on and switch-off functions V_{S2ind} and V_{S1rep} . The switch-on function V_{S2ind} of the AOX expression,

$$V_{S2ind}(c_{S2L}, t) = \frac{\left(\frac{c_{S2L}(t)}{k_{S2ind}}\right)^{\kappa_{ind}}}{1 + \left(\frac{c_{S2L}(t)}{k_{S2ind}}\right)^{\kappa_{ind}}} \quad (2.2)$$

with:

k_{S2ind}	:=	methanol induction turn-on concentration	$[g\ l^{-1}]$
κ_{ind}	:=	order of the induction mechanism	$[-]$,

is controlled by the methanol concentration c_{S2L} . The function outputs a value between 0 and 1. The limitation has the order κ_{ind} , which influences the steepness of

the function around the switch-on concentration k_{S2ind} , while k_{S2ind} itself represents the methanol concentration at which the switch-on function outputs a value of 0.5. The repression of AOX formation is achieved through a comparable mechanism. The switch-off function, designated V_{S1rep} ,

$$V_{S1rep}(c_{S1L}, t) = \frac{1}{1 + \left(\frac{c_{S1L}(t)}{k_{S1rep}} \right)^{\kappa_{rep}}} \quad (2.3)$$

with:

$$\begin{aligned} k_{S1rep} &:= \text{glycerol repression turn-off concentration} & [g\ l^{-1}] \\ \kappa_{rep} &:= \text{order of the repression mechanism} & [-], \end{aligned}$$

also returns a value between 0 and 1. However, this function depends on the glycerol concentration and inhibits AOX expression when $c_{S1L} \gg k_{S1rep}$, giving an output value close to zero.

The cell-specific mRNA induction rate, $q_{S2/Xind}$, depends on substrate concentration and influences the transcription rate, $q_{S2/Xscript}$. The transcription delay is modeled by the first order differential,

$$\dot{q}_{S2/Xscript}(t) = - [a_{S2script} + q_{X/X}(t)] \cdot q_{S2/Xscript}(t) + b_{S2script} \cdot q_{S2/Xind}(t) \quad (2.4)$$

with:

$$\begin{aligned} a_{S2script} &:= \text{AOX transcription factor} & [h^{-1}] \\ q_{X/X} &:= \text{cell-specific growth rate} & [h^{-1}] \\ q_{S2/Xscript} &:= \text{cell-specific transcription rate} & [h^{-1}] \\ b_{S2script} &:= \text{AOX transcription amplification factor} & [h^{-1}]. \end{aligned}$$

The unknown cell-specific proportion of mRNA for the formation of AOX is represented by the parameter $g_{AOXmRNA}$.

In the next step, the transcription rate influences the translation dynamics,

$$\dot{q}_{S2/Xtrans}(t) = - [a_{S2trans} + q_{X/X}(t)] \cdot q_{S2/Xtrans}(t) + b_{S2trans} \cdot q_{S2/Xind}(t) \quad (2.5)$$

with:

$a_{S2trans}$	:=	AOX transcription factor	$[h^{-1}]$
$q_{S2/Xtrans}$	:=	cell-specific translation rate	$[h^{-1}]$
$b_{S2trans}$	:=	AOX translation amplification factor	$[h^{-1}]$

which reflects the temporal change of the cell-specific translation rate $q_{S2/Xtrans}$. This in turn is a measure of the initiation of AOX formation via translation factors (g_{AOXTF}).

The AOX formation is described in the third phase. The cell-specific AOX activity $q_{S2/Xact}$ includes the AOX content (g_{AOX}) on the one hand and controls the methanol conversion of the cell on the other. The change in the AOX activity is described by,

$$\dot{q}_{S2/Xact}(t) = - [a_{S2act} + q_{X/X}(t)] \cdot q_{S2/Xact}(t) + b_{S2act} \cdot q_{S2/Xind}(t) \quad (2.6)$$

with:

$q_{S2/Xact}$	:=	cell-specific activity rate	$[h^{-1}]$
a_{S2act}	:=	AOX activity factor	$[h^{-1}]$
b_{S2act}	:=	AOX activity amplification factor	$[h^{-1}]$.

The AOX activity enables the calculation of the cell-specific methanol uptake rate $q_{S2/X}$,

$$q_{S2/X}(t) = q_{S2/Xact}(t) \cdot \frac{c_{S2L}(t)}{k_{S2} + c_{S2L}(t)} \cdot \frac{k_{I22}}{k_{I22} + c_{S2L}(t)} \cdot \frac{k_{I21}}{k_{I21} + c_{S1L}(t)} \quad (2.7)$$

with:

$q_{S2/X}$	:=	methanol uptake rate	$[h^{-1}]$
k_{S2}	:=	methanol saturation constant	$[g\ l^{-1}]$
k_{I22}	:=	methanol-methanol inhibition constant	$[g\ l^{-1}]$
k_{I21}	:=	methanol-glycerol inhibition constant	$[g\ l^{-1}]$,

which describes the processing of AOX. The uptake rate $q_{S2/X}$ includes classical approaches of enzyme kinetics, in which a limitation is caused by the saturation constant k_{S2} , an auto-inhibition via the inhibition constant k_{I22} and a foreign inhibition by glycerol via the constant k_{I21} .

While eight of the 14 newly introduced parameters have to be identified experimentally, the remaining six can be determined by considering the stationary behavior. In the BIOSIM model of *E. coli*, μ_{2max} is determined from the temperature and pH dependent optimal cell-specific growth rate μ_{2opt} . Temperature and pH are included in the determination of μ_{2max} via a factor, while μ_{2opt} is given as a parameter [23]:

$$\mu_{2max}(t) = \mu_{2opt} \cdot \kappa_{\vartheta_L}(t) \cdot \kappa_{pH}(t) \quad (2.8)$$

with:

μ_{2opt}	:=	optimal maximum cell-specific growth rate on methanol	$[h^{-1}]$
κ_{ϑ_L}	:=	environmental growth control function of temperature	$[-]$
κ_{pH}	:=	environmental growth control function of pH	$[-]$

μ_{2max} is then used to calculate the maximum cell-specific uptake rate $q_{S2/Xmax}$,

$$q_{S2/Xmax} = \frac{\mu_{2max} + q_{X/Xm}}{Y_{X/S2gr}} \quad (2.9)$$

with:

$q_{S2/Xmax}$	:=	maximum cell-specific methanol uptake rate	$[h^{-1}]$
---------------	----	--	------------

q_{X/X_m}	$:=$	maintenance part of the cell-specific growth rate	$[h^{-1}]$
$y_{X/S_{2gr}}$	$:=$	growth part of the methanol yield coefficient	$[-]$.

In contrast, in the model of Cornelissen, maximum substrate-specific growth occurs when an optimal concentration of methanol c_{S2Lopt} is reached in the absence of glycerol,

$$c_{S2Lopt} = \sqrt{k_{S2} \cdot k_{I22}} \quad (2.10)$$

with:

c_{S2Lopt}	$:=$	optimal methanol concentration	$[h^{-1}]$.
--------------	------	--------------------------------	--------------

The optimal methanol concentration c_{S2Lopt} together with the genetic AOX potential $q_{S2/Xsup}$ can then be used to determine the maximum cell-specific methanol uptake rate $q_{S2/Xmax}$,

$$q_{S2/Xmax} = q_{S2/Xsup} \cdot \frac{c_{S2Lopt}}{k_{S2} + c_{S2Lopt}} \cdot \frac{k_{I22}}{k_{I22} + c_{S2Lopt}} \quad (2.11)$$

which in turn used to determine the maximum cell-specific growth rate μ_{2max} ,

$$\mu_{2max} = y_{X/S_{2gr}} \cdot q_{S2/Xmax} - q_{X/X_m} \quad (2.12)$$

Thus, in the BIOSIM model, μ_{2max} is determined exclusively by the temperature in the liquid phase and the pH value, while in the Cornelissen model it is determined by the optimal methanol concentration. To combine both models, μ_{2opt} is no longer given as a parameter, but determined by the optimal methanol concentration c_{S2Lopt} via $q_{S2/Xmax}$,

$$\mu_{2opt} = y_{X/S_{2gr}} \cdot q_{S2/Xmax} - q_{X/X_m} \quad (2.13)$$

μ_{2max} is determined via the temperature and pH factors according to Equation 2.13.

The total induction amplification K_{S2ind} ,

$$K_{S2ind} = \frac{b_{S2script}}{a_{S2script} + \mu_{2max}} \cdot \frac{b_{S2trans}}{a_{S2trans} + \mu_{2max}} \cdot \frac{b_{S2act}}{a_{S2act} + \mu_{2max}}, \quad (2.14)$$

is then set equal to 1 to determine the three parameters $b_{S2script}$, $b_{S2trans}$ and b_{S2act} ,

$$b_{S2K} = a_{S2K} + \mu_{2max}, \quad K = \text{script, trans, act.} \quad (2.15)$$

All the parameter values assigned for this model are summarized in Table X.

In Listing 6.1, the implementation in MATLAB® is shown. The ODE function was outsourced to an external file (Listing 6.2).

2.2.2 Target Protein Expression

Compared to AOX induction, the expression of the target protein is more complex due to the strong *AOX1* promoter [8]. As can be seen in Figure 4.14, a transcription regulator has been integrated into the model, which produces an overexpression via feedback.

The induction rate $q_{P1/Xind}$,

$$q_{P1/Xind}(t) = V_{S1rep}(c_{S1L}, t) \cdot V_{S2ind}(c_{S2L}, t) \cdot q_{P1/Xsup} \quad (2.16)$$

with:

$q_{P1/Xind}$	$:=$	cell-specific target protein induction rate	$[h^{-1}]$
$q_{P1/Xsup}$	$:=$	upper limit of product formation potential	$[h^{-1}]$,

of transcription is regulated, as in AOX induction, by switch-on and -off functions (Equation 2.2 and 2.3). The differential of the transcription rate $q_{P1/Xscript}$,

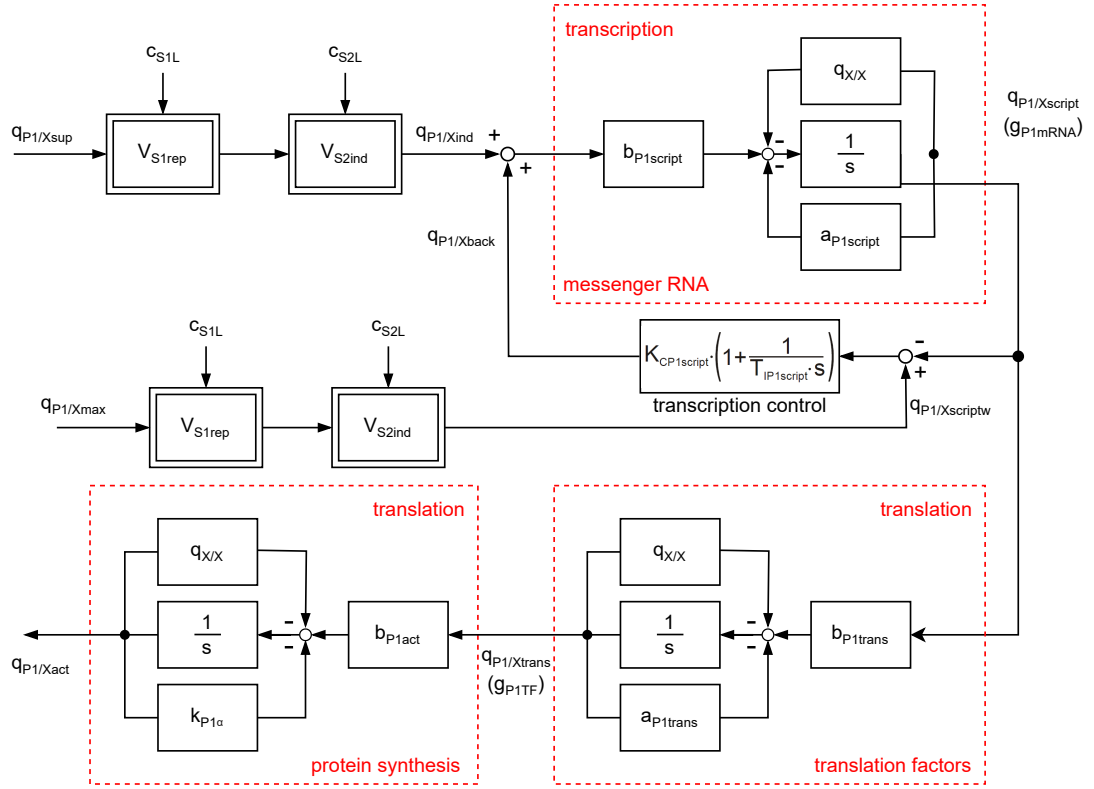


Figure 2.3: Schematic of the regulation of cellular target protein formation in the *P. pastoris* model

Target protein formation is regulated by substrate concentration of glycerol and methanol via three first-order metabolic delay systems. An P-T1 control system is used to mimic overexpression. Adapted from Cornelissen (2004).

$$\dot{q}_{P1/Xscript}(t) = - [a_{P1script} + q_{X/X}(t)] \cdot q_{P1/Xscript}(t) + b_{P1script} \cdot [q_{P1/Xind}(t) + q_{P1/Xback}(t)] \quad (2.17)$$

with:

$a_{P1script}$	$:=$	target protein transcription factor	$[h^{-1}]$
$q_{P1/Xscript}$	$:=$	target protein transcription rate	$[h^{-1}]$
$b_{P1script}$	$:=$	target protein transcription amplification factor	$[h^{-1}]$
$q_{P1/Xback}$	$:=$	target protein feedback signal	$[h^{-1}]$,

includes the feedback signal $q_{P1/Xback}$,

$$q_{P1/Xback}(t) = K_{CP1script} \cdot \left[\Delta q_{P1/Xscript}(t) + \frac{1}{T_{IP1script}} \int_0^t \Delta q_{P1/Xscript}(t^*) dt^* \right] \quad (2.18)$$

with:

$K_{CP1script}$	$:=$	control amplification of target protein transcription	$[-]$
$\Delta q_{P1/Xscript}$	$:=$	target protein transcription rate	$[h^{-1}]$
$T_{P1script}$	$:=$	target protein transcription delay	$[h]$.

The feedback signal is influenced by the PI behavior of the transcription regulator and the control difference $\Delta q_{P1/Xscript}$,

$$\Delta q_{P1/Xscript}(t) = q_{P1/Xscriptw}(t) - q_{P1/Xscript}(t) \quad (2.19)$$

with:

$q_{P1/Xscriptw}$	$:=$	target protein transcription rate set point	$[h^{-1}]$.
-------------------	------	---	--------------

The set point of the transcription rate $q_{P1/Xscriptw}$,

$$q_{P1/Xscriptw}(t) = V_{S1rep}(c_{S1L}, t) \cdot V_{S2ind}(c_{S2L}, t) \cdot q_{P1/Xmax} \quad (2.20)$$

with:

$q_{P1/Xmax}$	$:=$	maximum target protein formation rate	$[h^{-1}]$,
---------------	------	---------------------------------------	--------------

is switched on and off like the induction rate via the induction or repression function and is derived from the maximum cell-specific production rate $q_{P1/Xmax}$.

The transcription regulator can be regarded as a PI-controlled P-T1 control system, where the reference variable $q_{P1/X_{\max}}$ and the disturbance variable $q_{P1/X_{\sup}}$ are controlled via the glycerol and methanol concentrations. The transcription rate derived from the transcriptional regulator activates the translation factors for protein production, similar to AOX induction,

$$\dot{q}_{P1/X_{\text{trans}}}(t) = - [a_{P1_{\text{trans}}} + q_{X/X}(t)] \cdot q_{P1/X_{\text{trans}}}(t) + b_{P1_{\text{trans}}} \cdot q_{P1/X_{\text{script}}}(t) \quad (2.21)$$

with:

$a_{P1_{\text{trans}}}$	:=	target protein translation factor	$[h^{-1}]$
$q_{P1/X_{\text{trans}}}$	:=	target protein translation rate	$[h^{-1}]$
$b_{P1_{\text{trans}}}$	:=	target protein translation amplification factor	$[h^{-1}]$,

which in turn increases the translation rate of the target protein in $q_{P1/X_{\text{act}}}(t)$,

$$\dot{q}_{P1/X_{\text{act}}}(t) = - [k_{P1\alpha} + q_{X/X}(t)] \cdot q_{P1/X_{\text{act}}}(t) + b_{P1_{\text{act}}} \cdot q_{P1/X_{\text{trans}}}(t) \quad (2.22)$$

with:

$k_{P1\alpha}$	:=	target protein secretion coefficient	$[h^{-1}]$
$q_{P1/X_{\text{act}}}$	:=	target protein activity rate	$[h^{-1}]$
$b_{P1_{\text{act}}}$	:=	target protein activity amplification factor	$[h^{-1}]$,

which is positively influenced by the cell-specific translation activity $q_{P1/X_{\text{trans}}}$ and negatively influenced by the growth rate $q_{X/X}$ and the secretion coefficient $k_{P1\alpha}$ in a first-order rate.

Ultimately, the previously determined translation activity can be equated to the desired cell-specific product formation rate $q_{P1/X}$,

$$q_{P1/X}(t) = q_{P1/X_{\text{act}}}(t) \quad (2.23)$$

with:

$$q_{P1/X} \quad := \quad \text{cell-specific product formation rate} \quad [h^{-1}].$$

In Listing 6.3, the implementation in MATLAB® is shown. The ODE function was outsourced to an external file (Listing 6.4).

2.2.3 Protein Secretion

P. pastoris is capable of secreting proteins as a eukaryotic expression system. To integrate this dynamic into the simulation model, a variable for the intracellular product concentration, c_{P1X} , was introduced along with the product concentration in the liquid phase, c_{P1L} . In the model developed by Cornelissen for target protein expression, a coefficient $k_{P1\alpha}$ was introduced to account for protein secretion (Equation 2.22). This adjustment indicates that product formation is affected by the pre-existing product content within the cell, due to the limitations of intracellular space. The mass concentration balance for c_{P1X} ,

$$\dot{c}_{P1X}(t) = q_{P1/X}(t) \cdot c_{XL}(t) - \left[\frac{F_{in}(t)}{V_L(t)} + k_{P1\alpha} \right] \cdot c_{P1X}(t) \quad (2.24)$$

with:

$$\begin{array}{llll} c_{P1X} & := & \text{cell internal product concentration in liquid phase} & [g \ l^{-1}] \\ c_{XL} & := & \text{cell concentration in liquid phase} & [g \ l^{-1}] \\ F_{in} & := & \text{total feed rate} & [l \ h^{-1}] \\ V_L & := & \text{volume of liquid phase} & [l] \end{array}$$

is derived from the secretion rate.

The concentration of the product in the total intracellular space increases due to product formation, whereas it decreases as a result of liquid phase dilution and secretion.

Although the product is not resolved in the liquid phase, the dilution of the liquid phase is also taken into account, as the cell and liquid spaces are regarded as a single entity.

After formation within the cell, the product is secreted into the extracellular space. In the modified concentration balance for product in the liquid phase c_{P1L} ,

$$\dot{c}_{P1L}(t) = \frac{F_{R1}(t)}{V_L(t)} \cdot c_{P1R1} + \frac{F_{R2}(t)}{V_L(t)} \cdot c_{P1R2} + k_{P1\alpha} \cdot c_{P1X}(t) - \frac{F_{in}(t)}{V_L(t)} \cdot c_{P1L}(t) \quad (2.25)$$

with:

c_{P1L}	$:=$	product concentration in liquid phase	$[g\ l^{-1}]$
F_{R1}	$:=$	feed rate from reservoir 1	$[l\ h^{-1}]$
c_{P1R1}	$:=$	target protein concentration in reservoir 1	$[g\ l^{-1}]$
F_{R2}	$:=$	feed rate from reservoir 2	$[l\ h^{-1}]$
c_{P1R2}	$:=$	target protein concentration in reservoir 2	$[l\ h^{-1}]$

is increased by product pumped from both reservoirs and secretion, while dilution results in a decrease.

2.2.4 Methanol Toxicity

The organism's vulnerability to methanol toxicity presents a challenge in *P. pastoris* cultivation. The toxicity of methanol is mainly due to the metabolite formaldehyde, which is a product of the AOX pathway. Methanol concentrations above 5% (v/v) negatively influence the viability of *P. pastoris*, leading to a halt of product formation. Nevertheless, a particular methanol concentration is required as an inducer to attain an effective production rate [24].

The development of a mathematical model of methanol-induced toxicity was inspired by the previously described models of AOX induction and target protein expression. It uses a methanol concentration controlled function V_{S2ind} , which acts similar to a switch. Since the model to be developed also includes a methanol-controlled effect, this dy-

namic provides a suitable basis for an approximate representation. The underlying equation 2.2 has a sigmodal curve with the inflection point at the substrate concentration k_{S2ind} . This curve is useful for modeling toxicity because the maximum possible mortality is limited. Analogous to the turn-on concentration for AOX induction, the toxicity turn-on concentration k_{S2tox} is introduced for the toxicity model. With κ_{S2tox} , defining the slope of the sigmodal curve, a function for toxicity switch-on V_{S2tox} ,

$$V_{S2tox}(c_{S2L}, t) = \frac{\left(\frac{c_{S2L}(t)}{k_{S2tox}}\right)^{\kappa_{tox}}}{1 + \left(\frac{c_{S2L}(t)}{k_{S2tox}}\right)^{\kappa_{tox}}} \quad (2.26)$$

with:

$$\begin{aligned} k_{S2tox} &:= \text{methanol toxicity turn-on concentration} & [g\ l^{-1}] \\ \kappa_{tox} &:= \text{order of the toxicity mechanism} & [-] \end{aligned}$$

can be developed. Together with the maximum potential methanol induced death rate $q_{X/Xtox1}$, $q_{X/Xtox}$ is calculated by

$$q_{X/Xtox}(t) = q_{X/Xtox1} \cdot V_{S2tox}(c_{S2L}, t) \quad (2.27)$$

with:

$$\begin{aligned} q_{X/Xtox} &:= \text{methanol toxicity part of the death rate} & [h^{-1}] \\ q_{X/Xtox1} &:= \text{maximum potential methanol induced toxicity} & [h^{-1}] \end{aligned}$$

Finally, the cell-specific growth rate $q_{X/X}$,

$$q_{X/X}(t) = q_{X/Xgr}(t) - q_{X/Xm} - q_{X/Xtox}(t) \quad (2.28)$$

with:

$$q_{X/Xgr} := \text{growth part of the cell-specific growth rate} \quad [l\ h^{-1}]$$

$q_{X/X_m} :=$ maintenance part of the cell-specific growth rate [l h⁻¹]

is composed of a growth, maintenance and death part, the latter induced by methanol.

2.3 Process Automation Principles

The control and automation of bioprocesses is implemented using process control systems (PCS) such as MFCS/win from Sartorius (Göttingen, Germany). MFCS/win stands for “Multi Fermentor Control System for Windows” and is used for data acquisition and process control of bioprocesses [8]. PCS can be used to develop recipes that can specify a process sequence and thus allow the process to be automated. Based on process automation, different process phases can be set up.

2.3.1 MFCS/win

Figure 2.4 shows the recipe for a cultivation process running in MFCS/win.

The view is organized like a flowchart, with process steps displayed as blocks. The blocks can have different functions, such as displaying a defined message box or adjusting parameter values. Figure 2.5 shows the parameter setting window for the *renew_setpoints* phase. It can be observed that the variables can be selected within the window, and that their properties, such as set points, can be adjusted.

After each block, one or more specific conditions, such as responses from message boxes (yes/no) or process variables like the oxygen partial pressure p_{O_2} , are defined to determine the further course of the process. Branches are possible, which can lead the process in different directions. Figure 2.6 shows a condition attached to the course of the process.

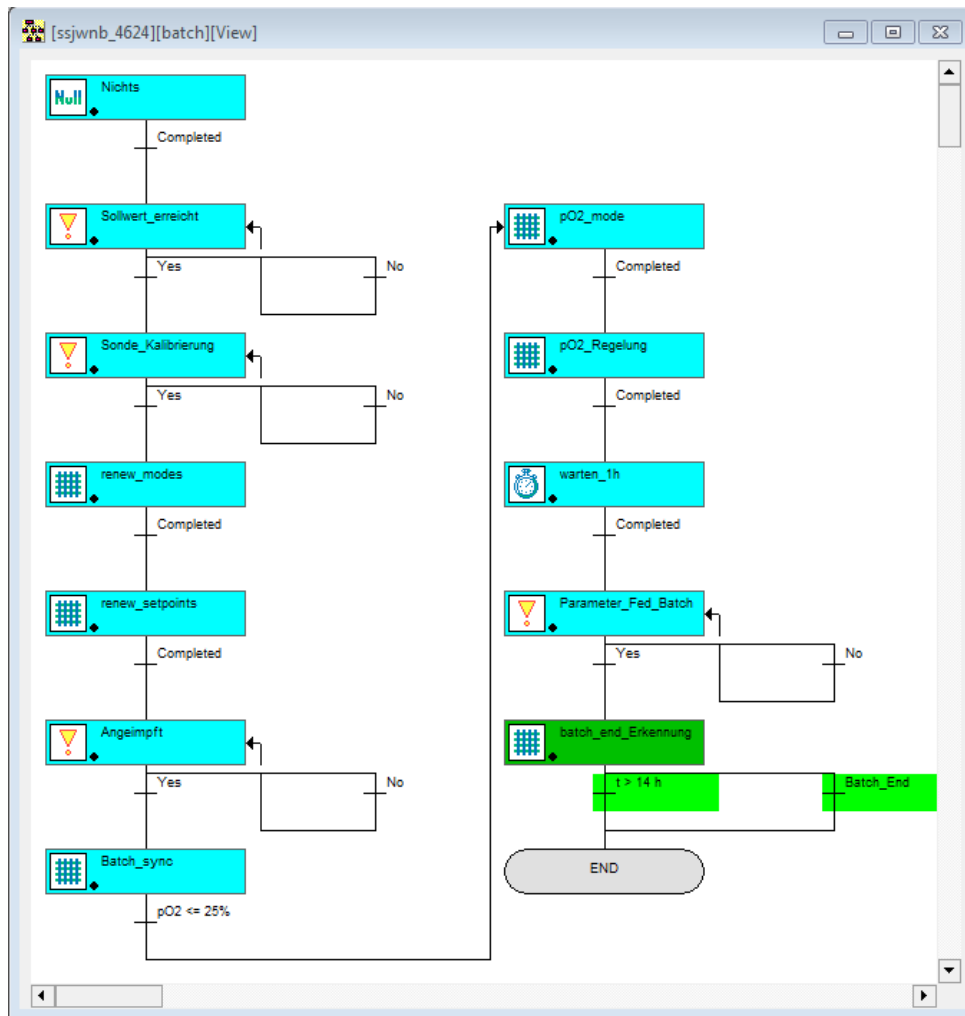


Figure 2.4: Exemplary MFCS/win recipe of a batch phase

2.3.2 Feeding Strategies

While the inspiration for the automation interface, MFCS/win, does not itself take over control in real processes, this simulation platform represents a fully integrated system and thus carries out simulation, control and process management. As part of the implementation of automation mechanisms, the simulation platform should be given the option of using different feeding strategies. While constant feeding is straightforward, defined by a parameter specifying the desired feed rate as a factor relative to the maximum pump capacity, open-loop and closed-loop feeding require the pumping rates to be dynamically determined at each point in time. In the actual processes conducted at the BPA laboratories, this task is typically handled by external controllers or toolboxes.

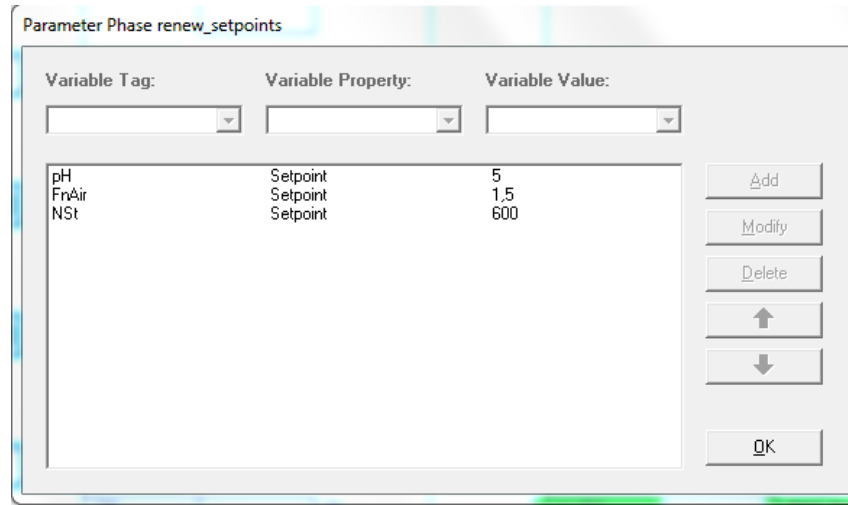


Figure 2.5: Variable editing in a Parameter Phase in MFCS/win

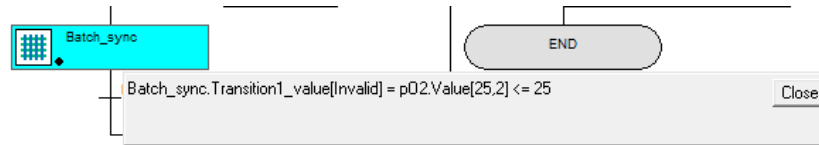


Figure 2.6: Variable condition for a phase transition in MFCS/win

However, in this integrated approach, these calculations are performed within the same software that simulates the biological system.

Open-Loop Feed Control

Open-loop control is a control strategy in which the control of the system is based on predefined inputs without feedback to adjust the control parameters. In contrast to closed-loop (feedback) control, in which process variables are continuously measured and the control is adjusted based on these feedback signals, open-loop control is performed independently of the current system response [19].

In the following, the open-loop feed control used for fed-batch phases in cultivations of *P. pastoris* is presented. The feeding rate from reservoir 1 of substrate 1 at process time t can be calculated using,

$$F_{R1}(t) = F_{R1j} \cdot e^{\mu_{1w}(t-t_j)} \quad (2.29)$$

with:

F_{R1}	:=	feeding rate from reservoir 1	$[l\ h^{-1}]$
F_{R1j}	:=	initial feeding rate at t_j	$[l\ h^{-1}]$
t_j	:=	fed-batch start time	$[h]$
μ_{1w}	:=	set growth rate for substrate 1	$[h^{-1}]$.

The initial feeding rate F_{R1j} is determined by,

$$F_{R1j} = \frac{\mu_{1w} + q_{S1/Xm} \cdot Y_{X/S1gr}}{Y_{X/S1gr} \cdot c_{S1R1}} \cdot V_{Lj} \cdot c_{XLj} \quad (2.30)$$

with:

$q_{S1/Xm}$	:=	cell-specific maintenance substrate uptake rate	$[h^{-1}]$
$Y_{X/S1gr}$	:=	growth part of the cell-specific yield coefficient	$[g\ g^{-1}]$
c_{S1R1}	:=	substrate concentration in reservoir 1	$[g\ l^{-1}]$
V_{Lj}	:=	volume in the beginning of the fed-batch phase	$[l]$
c_{XLj}	:=	cell concentration in the beginning of the fed-batch phase	$[g\ l^{-1}]$.

This results in a predefined, exponentially increasing feeding profile that allows cells to grow in a controlled, substrate-limited manner at a defined growth rate μ_{1w} . Controlled growth allows high-cell-density cultivations with cell densities $>100\ g\ l^{-1}$ cell dry weight [11].

Online measurement of substrate concentrations such as glucose or glycerol is difficult to implement in real processes [28]. Ideally, a fed-batch phase should start when all by-products of overflow metabolism from the batch phase, such as ethanol in *E. coli* or ethanol as an alternative carbon source in *S. cerevisiae*, have been removed, since the presence of these metabolites during the long fed-batch process can negatively affect cell viability [51].

A tool for the automated detection of the actual batch end via variable progressions is integrated into the software.

Pulse Feed

A tool for pulse feeding, i.e. a constant feeding rate, is introduced into the automation. Adding feed for a defined time t , a defined amount of e.g. substrate can be added to the vessel. In practice, this method is used, for example, for induction via isopropyl- β -D-1-thiogalactopyranoside (IPTG) or to initiate the production phase in *P. pastoris* with methanol.

The pump rate F_I ,

$$F_I = k_I \cdot F_{I_{\max}} \quad (2.31)$$

with:

$$\begin{aligned} k_I &:= \text{utilization factor of the pump of reservoir } I = R1, R2, R3 & [-] \\ F_{I_{\max}} &:= \text{maximum capacity of the pump of reservoir } I = R1, R2, R3 & [\text{g l}^{-1}] \end{aligned}$$

consists of the maximum capacity of a pump and a utilization factor.

Closed-Loop Feed Control

In order to keep the methanol concentration in the production phase during the cultivation of *P. pastoris* in a small desired range, feedback control of the substrate concentration is often used [8]. In this process model, a proportional-integral-derivative (PID) controller is implemented to provide the cells with a sufficient supply of methanol while preventing saturation, which could otherwise result in cell death. This controller has already been implemented in the *E. coli* model and is used for the pO₂-agitation, pO₂-aeration, pO₂-gasmix, pO₂-feed and liquid weight harvest control [23]. The structure was adopted for the substrate control and new parameters introduced. The implementation in the source code is shown in Listing 6.6 in the Appendix from line 70.

3 Methods

3.1 MATLAB® 2024a

MATLAB® is a software platform that was developed in the 1970s and is distributed by The MathWorks, Inc. MATLAB® stands for MATrix LABoratory and is one of the most well-known tools for calculating and simulating complex mathematical and technical problems. On the one hand, MATLAB® can be used as an interactive calculation and simulation environment, but also as an environment for self-written MATLAB® programs [43]. In addition, MATLAB® offers a wide range of add-ons. These include apps, products, extension packages and toolboxes [34]. For this work, the Database Toolbox (see Section 3.3) and the App Compiler are of particular interest.

The integrated MATLAB® App Designer allows users to create their own applications. In this work, all front-end GUIs are developed on the basis of the MATLAB® App Designer and then compiled into a stand-alone application. The back-end logic consists of App Designer functionality, separate MATLAB® functions and a database. The version used in this work is 2024a.

3.1.1 MATLAB® App Designer

As in the previous version of the biofermentation simulation software, the MATLAB® App Designer is used for the UI of the simulation platform. It is an easy-to-use development environment offered by MathWorks that allows the creation of interactive applications [32]. The MATLAB® App Designer provides a collection of interactive elements, such as buttons, graphs, drop-down menus, etc., that can be dragged and dropped onto a UI window [31]. Due to the close interaction of the App Designer with MATLAB®, functions already developed in MATLAB® can be seamlessly integrated into the UI. These functions can then be called by the user via "callbacks". They play a crucial role in an interactive application, as they allow event-driven execution of functions by enabling responding to user input [35].

The application is programmed object-oriented in the integrated MATLAB® editor. A code analyzer automatically helps to identify programming problems and issues warning and error messages. In the App Designer application, it is possible to switch between a design view and a code view. The design view is used to place objects and assign callbacks. The callback function itself can then be defined in the code view [31].

The App Designer provides a user-friendly development environment, particularly suited for those with limited UI programming experience. It provides a rigid framework, that clears out which sections can be edited, and which must remain unedited. When objects are added via the design view, a constructor for that object is automatically inserted into the code, which cannot be edited. Properties of this object must then be changed in the Component Browser. However, individual properties and functions can be added in the method block. Properties can be assigned a data type in the constructor or later in a function. They can be assigned a series of values (also known as a vector or matrix in a mathematical context), but also objects, functions or classes. These properties can then be accessed using the syntax `app.[PropertyName]`, where `app` is the name of the class [33].

The Application Compiler add-on allows to turn a user interface developed in MATLAB® App Designer, into a standalone application [36]. The compiled application can then be shared with other users, that can install the application on their own system. This automatically downloads the MATLAB® Runtime, which contains all the necessary libraries. A licensed copy of MATLAB® is not required.

In this software platform, both MATLAB® and MATLAB® App Designer were used for programming. MATLAB®, in particular its editor, was utilized to create organism-specific functions, while the App Designer was used only for front-end related purposes.

3.1.2 Dynamic Properties

Adding further organisms to the program required the introduction of new parameters and variables. In the previous version, hence only one organism had to be implemented, all parameters and variables were stored as properties. These properties are embedded in the code, which means they must be redefined and managed for each

new organism. This results in a lack of clarity and can lead to problems with cross-class access to these properties.

One approach to solve this issue, is to utilize the struct data type and dynamically fill it with new fields and values. Data from models and projects are read from the database via queries and assigned to the parameters/values via loops. An example for filling mentioned structs is shown in Listing 3.1.

Listing 3.1: MATLAB® example code for the construction of dynamic properties

```

1 properties (Access = public)
2     p = struct() % parameter struct is defined as a property
3 end
4 methods (Access = public)
5     function constructParameterProperties(app) % function definition
6         conn = sqlite(app.dbpath,"readonly"); % read-only database connection
7         T = fetch(conn, 'SELECT name, value FROM exampleTable'); % exemplary data fetch
8         for i = 1:size(T,1) % loop for the ammount of rows
9             name = T.name(i); % parameter name
10            value = T.value(i); % parameter value
11            app.p.(name) = value; % Add struct field and insert value
12        end
13        close(conn) % close connection
14    end
15 end

```

Using this dynamic approach with the struct *app.p*, in which all parameter values are saved via a loop, an unlimited number of new parameters can now be defined in the database. These parameters can then be accessed in the calculation file via e.g. *app.p.cXL0*. The variable data is stored in the struct *app.v* and auxiliary parameters in *app.a* according to the same principle, whereby each field is cyclically extended by the current process value.

3.1.3 Timer Function

In the previous version, the timer function was used to cyclically call the underlying mechanical model of the organism. As a result, Δt was added in each cycle, which resulted in the dynamic simulation. In this modular approach, the calculation script acts independently from the UI. In order to keep the UI, which acts as a client, up to date throughout the simulation, additional timer functions must be included. This applies to

the Calculation App and the Figure App, which will be introduced in Section 4.1.2 and 4.1.3.

3.1.4 ODE Solver

MATLAB® offers a collection of solvers for ordinary differential equation (ODE) problems. The selection of a suitable solver is based on the stiffness of the system, whereas MathWorks suggests trying ode45 first [37]. While the ode45 solver performs well with most ODE problems, other solver may be more efficient for certain problems. A stiff differential equation system contains variables with drastically different scalings, whereby variables in nonstiff systems change relatively uniformly.

Complex biofermentation models are often considered stiff systems because some variables, such as pO_2 , may fluctuate in certain situation while other variables, such as c_{XL} , do not. For this reason, the previously used non-stiff ode45 solver was replaced by the stiff ode15s solver as an optimization step. This is a variable-step, variable-order (VSVO) solver and allows for low to medium accuracy. Due to the stiff nature of the solver, fewer calculation steps are performed in stiff areas than with the ode45 solver, leading to an increase in performance [45].

3.2 Database

Databases play a key role in modern software systems. For this software concept, a relational database was used for storing, managing, and retrieving large amounts of data. Databases can be divided into different types. The best-known ones include hierarchical, object-oriented and relational databases [44].

Relational databases store data in tabular form, and allow relationships between data through the use of keys. Data organized in logically linked entities allows for data space reduction, since repetitive data is outsourced into external datasets. Relational databases store data in tabular form and establish relationships between data using keys. Organizing data into logically linked entities reduces data redundancy, allowing a clear separation of the data structures, such as models, parameters and simulation

results. Large quantities of data can be managed and fetched in an integral and consistent way, which is crucial for the underlying simulation software.

The database acts as a central storage medium in the interactive application. This architecture enables a clear separation between data storage and simulation logic. In addition, the relational structure allows for easy expansion and potential connection to real processes, enabling the OTS to be transformed into a Digital Twin at a later stage.

3.2.1 Structured Query Language (SQL)

Structured Query Language (SQL) is the standard language for communication with relational databases. SQL allows not only the creation and management of databases but also enables reading, modifying, or deleting data from a database (examples shown in Listing 3.2). Unlike so-called procedural languages such as C++, which define how to solve a problem, SQL tells a database management system (DBMS) what result is desired when interacting with relational databases. For this reason, SQL is also referred to as a sub-language since a database application can only be created in conjunction with a complete programming language such as C# or C++. SQL is thus a specialized language for accessing database systems [13].

SQL consists of three languages: The Data Definition Language (DDL) allows for the creation, modification, or deletion of elements within a database. These elements include tables, views, schemes, etc. The Data Manipulation Language (DML) allows reading, inserting, updating, or deleting data within a database. The last language group, the Data Control Language (DCL), is used to minimize the impact of unwanted and destructive influences on the database. However, it is an advanced SQL function that is not supported by all database systems, including SQLite, which is used in this project.

3.2.2 SQLite Database

This software platform uses SQLite 3, a single-user, serverless database. The embedded database does not require any special installation or configuration, moreover,

it is license-free [41]. For the underlying use case, it is an advantage that SQLite can be stored locally as a file and can be directly accessed via a database management system (DBMS) or via MATLAB® (see Section 3.3). Hence SQL is used for communication with SQLite, the scheme developed in this thesis can serve as a basis for a possible future migration to a server-based database (e.g., MySQL®, PostgreSQL®, Oracle®). Thus, the basic structure of tables, relationships, and queries remains the same, and any necessary adjustments are limited to optimizations and specific functional extensions supported by the new database technology. This compatibility allows for scalability and expansion without making fundamental changes to data modeling.

A characteristic that distinguishes SQLite from other database engines is the way it handles data types. Here, values are assigned to a storage class, rather than a static, rigid data type. This means that more data space is provided depending on the size of the value [42].

3.2.3 Normalization

Normalization is a concept for structuring the database in a way that redundancies are minimized and anomalies after data modification are avoided. Following the principles of normalization allows for a consistent, efficient, and scalable database structure. This can be reached by following the principles of so-called normal forms (NF). They build on each other, with the first three being the most important. NF1 makes sure that every cell of the table only contains one atomic value, while NF2 defines that every non-key attribute has to depend on the primary key. NF3 mandates the avoidance of transitive dependencies, ensuring that no non-key attribute relies on another non-key attribute, only directly on the primary key.

3.2.4 Trigger

As in other SQL-based databases, triggers can also be created in SQLite. A trigger is automatically activated when data is modified and a specified condition is met. The trigger then defines how the attempted data modification should be processed.

In this work, triggers are used to document parameter changes during the process. After each change, an entry is made in the logger along with a timestamp. In addition, a rule is introduced that checks whether a time stamp has already been stored in the *timeTab* of a project. If this is the case, the transaction for the variable data upload is interrupted via a "ROLLBACK" and data integrity is ensured. The trigger defined in DDL shown in Appendix X (follows).

3.3 MATLAB® Database Communication

The native library of MATLAB® does not include functions for connecting to relational databases. However, MathWorks offers an add-on called Database Toolbox, which provides drivers for connecting to databases and functions for database interaction [39]. The Java Database Connectivity (JDBC) driver required for connecting to SQLite is included and remains in a compiled app alongside the functions [38]. The functions used for interaction with the SQLite database are shown in Listing 3.2. First, a connection to the database is established via the path of the db-file and the SQL query is inserted into the `exec()`- (for INSERT, UPDATE or DELETE) or `fetch()`-function (for SELECT queries). The connection can also be made read-only. This is useful if several interfaces access the database simultaneously, and only want to read data via SELECT queries.

Listing 3.2: MATLAB® example code for database communication

```

1 % Example for an INSERT query
2 conn = sqlite(app.dbpath, "connect"); % Connect to database
3 exec(conn, "INSERT INTO projectTab (name) VALUES ('MyProject')");
4 close(conn); % Disconnect from database
5
6 % Example for a SELECT query
7 conn = sqlite(app.dbpath, "readonly"); % Read-only connection
8 T = fetch(conn, "SELECT * projectTab");
9 close(conn);
10
11 % Example for a transaction
12 conn = sqlite(app.dbpath, "connect");
13 conn.AutoCommit = 'off'; % Turn off auto commit
14 try
15     exec(conn, "BEGIN TRANSACTION");
16     % Define queries to be executed at once
17     sqlquery1 = 'UPDATE...';
18     sqlquery2 = 'INSERT...';

```

```

19     sqlquery3 = 'DELETE...';
20     exec(conn, sqlquery1);
21     exec(conn, sqlquery2);
22     exec(conn, sqlquery3);
23     % Commit changes at once
24     exec(conn, "COMMIT");
25 catch
26     exec(conn, "ROLLBACK"); % Undo changes if error occurs
27 end
28 close(conn); % Close connection

```

3.4 Parameter and Variable Pool

Data organization and categorization play an important role in the development of normalized databases. All data are individually identified through key attributes. One objective in the concept of the simulation software is to keep all data in the database continuously up-to-date, ensuring that other interfaces accessing the database are always provided with the latest values. Only this way, data consistency can be ensured. For this reason, all parameters relevant to the simulation must first be characterized and then categorized.

In the BIOSIM software, a distinction was made in the data structure between different types of quantities. They were classified into control variables, initial values, variables, parameters and output variables. Control variables are input variables, that influence the process from the outside. Parameters are time-invariant process variables, which are thus unchangeable. These include natural constants, such as the molar mass of oxygen M_{O_2} . Time-variant, on the other hand, are variables, that are differentiated as dynamic and static. Dynamic variables are also called state variables and are described by a first-order differential equation. The solution of an differential equation requires an initial value. The static variables, on the other hand, are time-dependent, but are determined without a differential equation and therefore do not require an initial value [29]. The data structure with example parameters and variables is shown in Figure 3.1.

For this simulation platform, data is categorized into variables, parameters and auxiliary parameters. The latter are special parameters that need to be calculated once

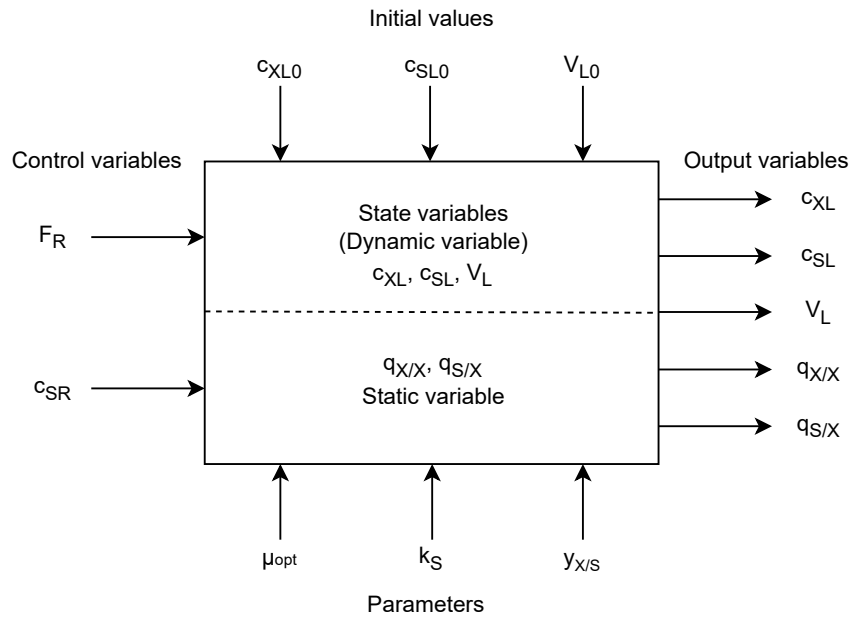


Figure 3.1: Data structure in the BIOSIM software

The different quantities that affect the BIOSIM simulation system are shown. The figure is taken from the BIOSIM Manual (2000), where μ_{max} is used instead of μ_{opt} . This parameter has been replaced by the cell-specific growth rate μ_{opt} under optimal conditions, as μ_{max} is temperature- and pH-dependent and therefore becomes time-variant.

prior simulation start, such as the Henry Coefficient for oxygen H_{O_2} . The quantities control variables, initial values and parameters, shown in Figure 3.1, are categorized as parameters, while output variables are classified as variables.

Parameters can be further classified as into static and cyclic parameters. Examples of static parameters are instrumentation characteristics like the maximum feeding rate of a pump. Cyclic parameters, the other hand, can be manipulated at any time by the operator, such as set points or control parameters. Since multiple interfaces can access these parameters, this must be taken into account.

Since some variables are more important for the evaluation of the process than others, a distinction is made between the variables that are uploaded cyclically to the database and those that are only uploaded at the end of the process. For dynamic variables, it is important that they are uploaded at least once at the end in order to have an initial value in case the process is continued. It is advisable not to upload all variable data

cyclically, as otherwise a large amount of data will quickly accumulate.

The parameters and variables are each listed in a table. These tables do not contain any numerical values, only attributes of these quantities such as unit, Tex notation or description, whereby model-specific attributes must be stored in other tables. If new organisms are to be introduced, either the existing quantities can be included in the model or new quantities can be added. Numerical values are stored in other tables and refer to the parameters and variables via foreign keys. This ensures data integrity and avoids redundancy. In Appendix 6.3, the respective tables are shown in entity-relationship (ER) diagrams. The categorized parameters of the newly introduced organism *P. pastoris* are listed in Table 6.1.

3.5 Model Process Strategy

To examine the dynamics of the model of *P. pastoris* introduced in this thesis and to verify the mechanisms for process automation, a process is simulated in the context of this thesis. The process is based on the process strategy of *P. pastoris*, which is usually conducted in the BPA laboratory. Figure 3.2 illustrates the course of the planned process. The fermentation of *P. pastoris* is divided into three phases. In the batch phase, the cells first consume the provided substrate. Once the substrate is consumed, the fed-batch phase begins, using an open-loop feed control system (see Section 2.3.2) to add substrate at a controlled rate. This ensures that cells grow in a substrate-limited manner at a set growth rate, denoted as μ_{1w} . The final phase is the production phase, which starts with a pulse addition of methanol (see Section 2.3.2). This allows the cells to adjust to the new conditions and start producing alcohol oxidase (AOX) to metabolize methanol. Following the initial methanol pulse, the concentration is maintained by a closed-loop feed control (see Section 2.3.2) to ensure optimal conditions for product formation without harming cell vitality.

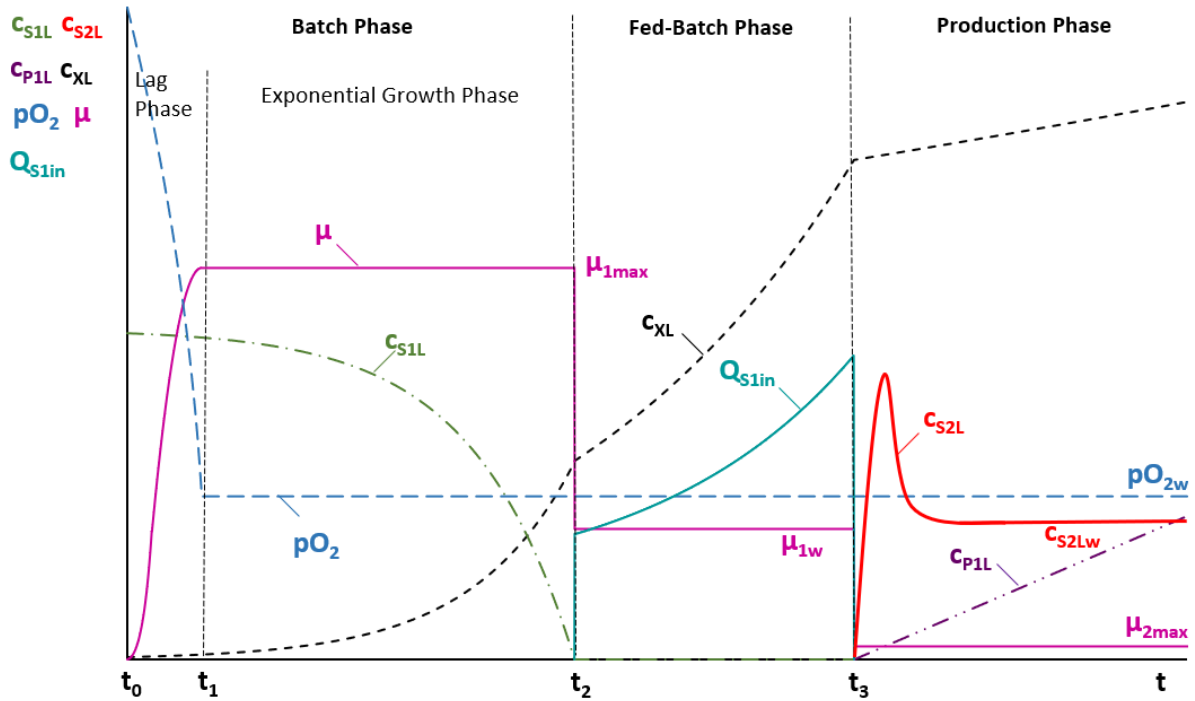


Figure 3.2: Standard fermentation strategy conducted in BPA laboratory

The three-stage fermentation consists of a batch, fed-batch and production phase. After a short lag phase, cells get used to the medium and grow at maximum rate μ_{1max} . Oxygen partial pressure is controlled to pO_{2w} by pO_2 -agitation. After the primary substrate c_{S1L} is completely consumed, the fed-batch begins, where primary substrate is fed as Q_{S1in} . This happens at a certain rate to maintain the growth rate μ_{1w} . At t_3 , feeding with substrate 1 (glycerol) ends and substrate 2, in this case methanol, is pumped into the reactor. The substrate concentration of methanol is then kept at c_{S2Lw} via closed-loop feed control. During this phase, the product c_{P1L} is expressed. The process course is inspired from Cornelissen (2004) and was created with Microsoft Excel (Microsoft Corporation, Redmond, WA, USA)

4 Results and Discussion

In the results and discussion section, the different levels of the software architecture, divided into front- and back-end, are presented. Subsequently, the implementation of the new organism *P. pastoris* is explained and the dynamics of the model is demonstrated on the basis of an exemplary process.

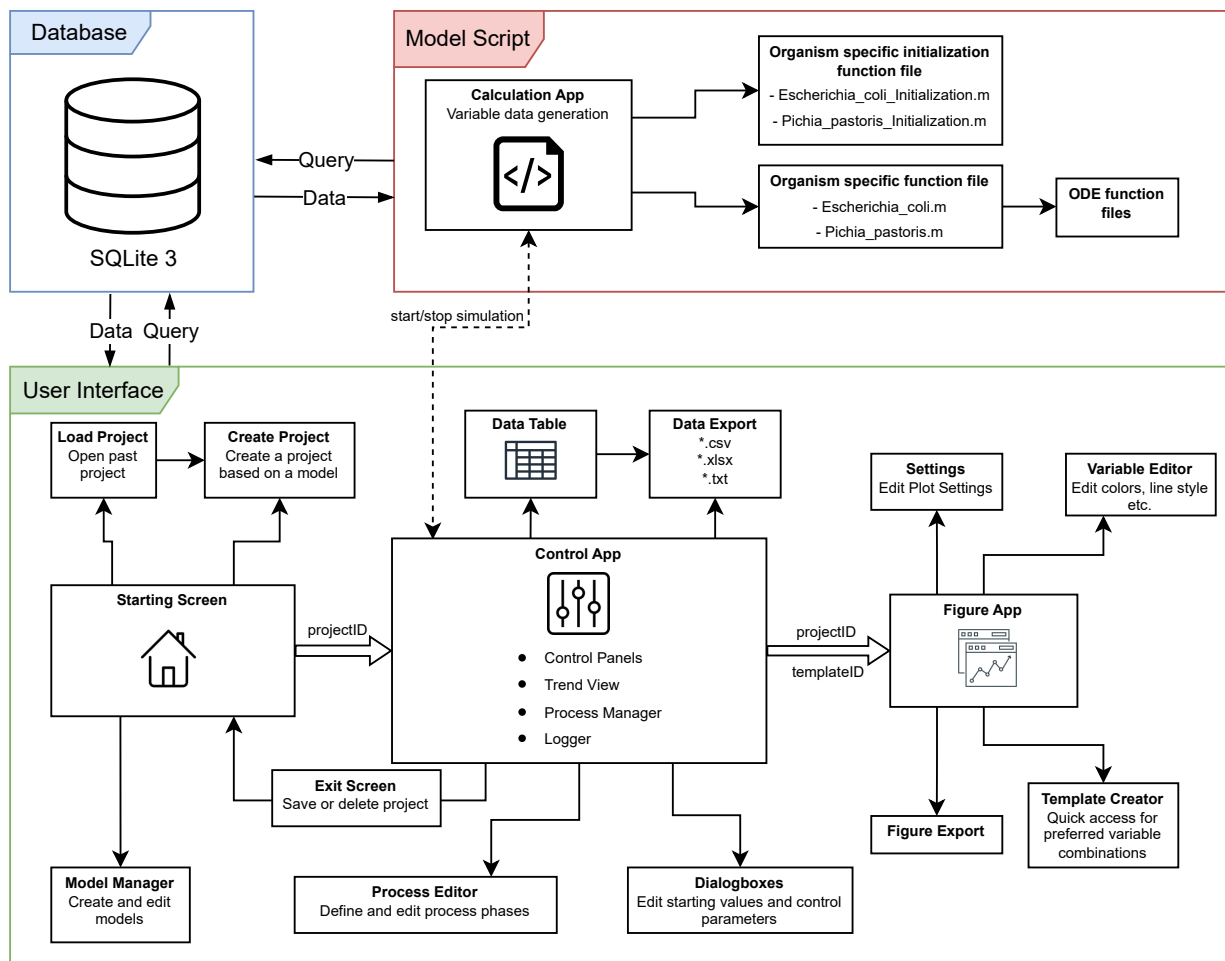


Figure 4.1: Software architecture overview

The software structure consists of three subsequent levels. Model Script and User Interface fetch data and manipulate the database through queries written in DML. The figure was created with draw.io.

Figure 4.1 shows all the components involved in this software platform and their relationships between each other. As already shown in Figure 2.1, the components can

be divided into three levels. While the GUI built with the MATLAB® App Designer represents the front-end, the database and the Model Script together form the back-end. The concept developed in this work includes that all these modules are supposed to run independently and that the only cross-module storage medium is the database. In this context, a module is a unit that accesses the database at regular intervals. Thus, the Model Script is a module, as are the Control App and the Figure App. This division was chosen to allow the perspective of adding more modules that interact with the database in parallel.

Front-end applications function as a type of client that periodically retrieves information, similar to the behavior typically observed in websites. Each of the modules receives a MATLAB® timer function, which was introduced in Section 3.1.3. In Figure 6.3, a connection between the Calculation App and the Control App can be observed. This is because the lightweight SQLite 3 database used in this work lacks advanced features to transmit signals for starting and stopping the simulation. It was considered to store the simulation state as a boolean in the database. However, it was not implemented to prevent scenarios where abnormally terminated clients might cause the Model Script to continue running indefinitely without receiving a stop signal from the client. For this reason, this interface between the Control App and the Calculation App was retained.

4.1 Front-End

4.1.1 Starting Screen

Upon launching the application, the Starting Screen is displayed (see Figure 4.2). Here, users can select models to initiate new projects or load previously started projects to continue from the last saved state.

By pressing the **Quick Start** button, a project based on the first listed model is started. To initiate a new project, users can select **Start New Project**. Here, the name, author, and a description of the project can be entered, and the corresponding model can be selected. Once confirmed, the new project is created, and the "Control App" is opened to start the simulation.

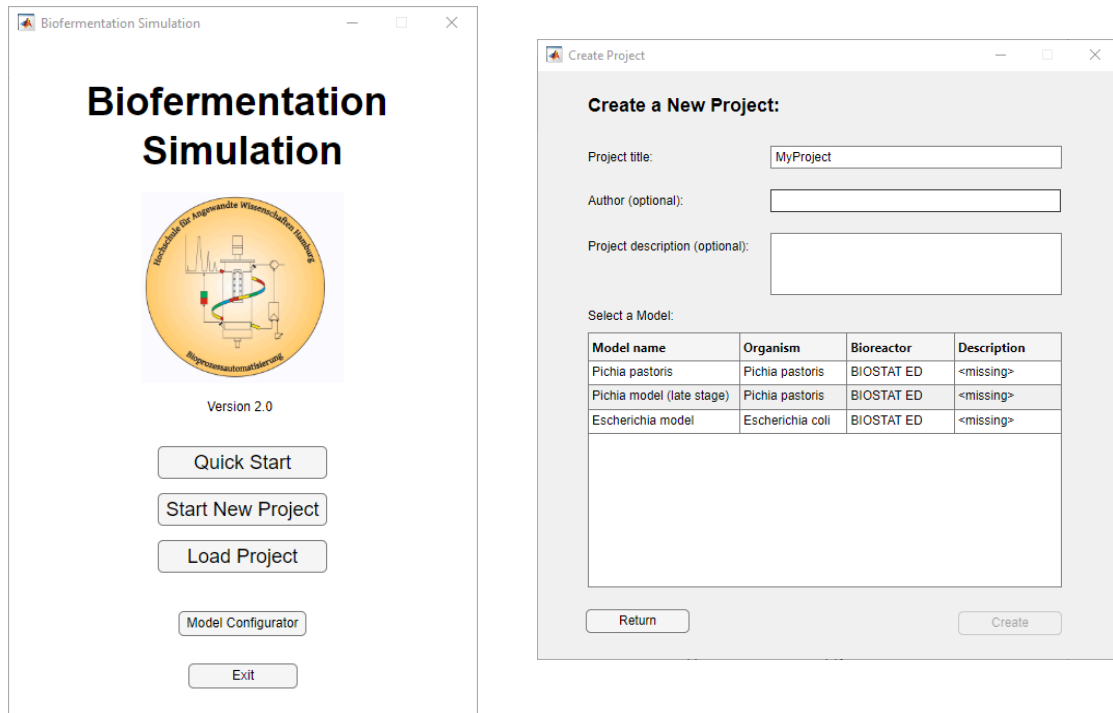


Figure 4.2: Starting Screen (left) and Project Creator (right) GUIs

The Starting Screen is displayed once the platform is launched. It allows for a quick project start, creating a desired project or relaunching a past project. Furthermore, the Model Configurator enables the user to combine organism and bioreactor for an individual simulation process. The button "Start New Project" initiates the Project Creator UI. Here, a new project can be initiated based on a listed model. The GUIs are based on MATLAB®.

If an existing project needs to be restarted, users can select **Load Project**. This will open a window listing all projects stored in the database. From here, projects can be opened or deleted (see Figure 4.3).

In the **Model Configurator**, users can create new models based on various combinations of organisms and bioreactors, and adjust their parameters (see Figure 4.4). This feature enables users to generate models with customized parameter sets as needed, or modify parameters from existing models. However, only parameters can be modified; organism-specific functions within the simulation cannot be customized.

In this context, a model can be seen as a template for a process, containing a collection of parameters, while projects are executable and derived from models. A model is derived from a collection of default parameters of an organism and a bioreactor, while a project is a copy of the mentioned model (see Figure 4.5).

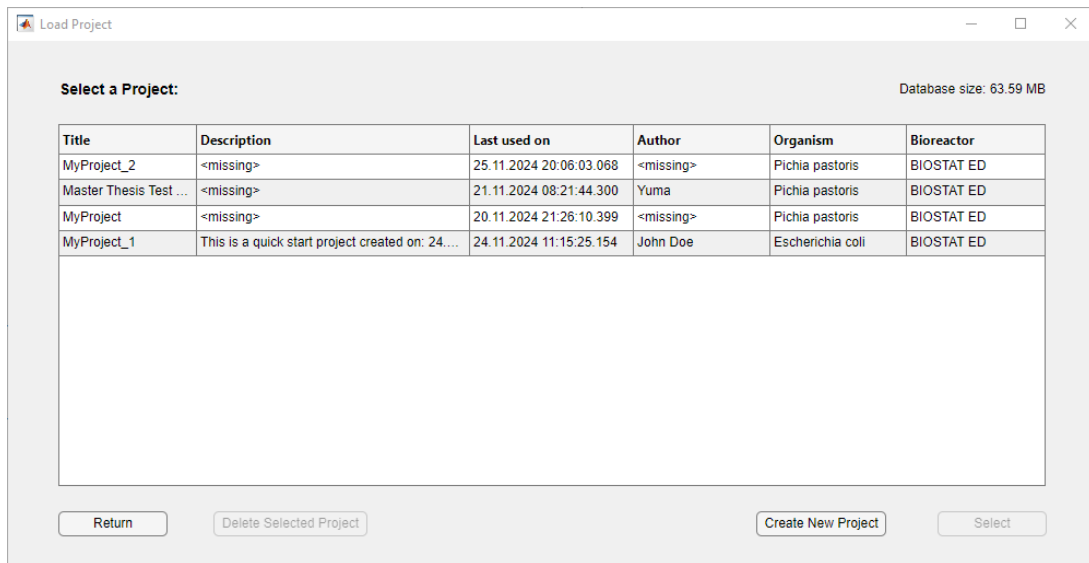


Figure 4.3: Load Project GUI

The Load Project UI lists all past projects and enables to relaunch a desired project. The GUI is based on MATLAB®.

Since every simulation is fully monitored within the database, previously conducted projects can be restored and resumed. Deleting a project leads to cascading removal of all parameters, logger entries, time stamps, and variable data connected to the project. This occurs due to the relationships through keys between all involved tables.

4.1.2 Control App

After starting or loading a project, the "Control App" is opened. The "Control App" is the main user interface of the simulation. The window is divided into a process control bar, where the process run can be started, and a tab group. The functions of each tab are briefly described below.

The control settings in the simulation is managed via the **Control Options** tab. The controllers are categorized into pH, temperature, pO₂, level, and substrate, and are displayed on individual panels. These panels allow users to turn the controllers on and off, set the set points, and adjust the control parameters.

The newly implemented **Process Manager** is a feature, that provides an option to create and manage process phases. This process automation tool enables users to define

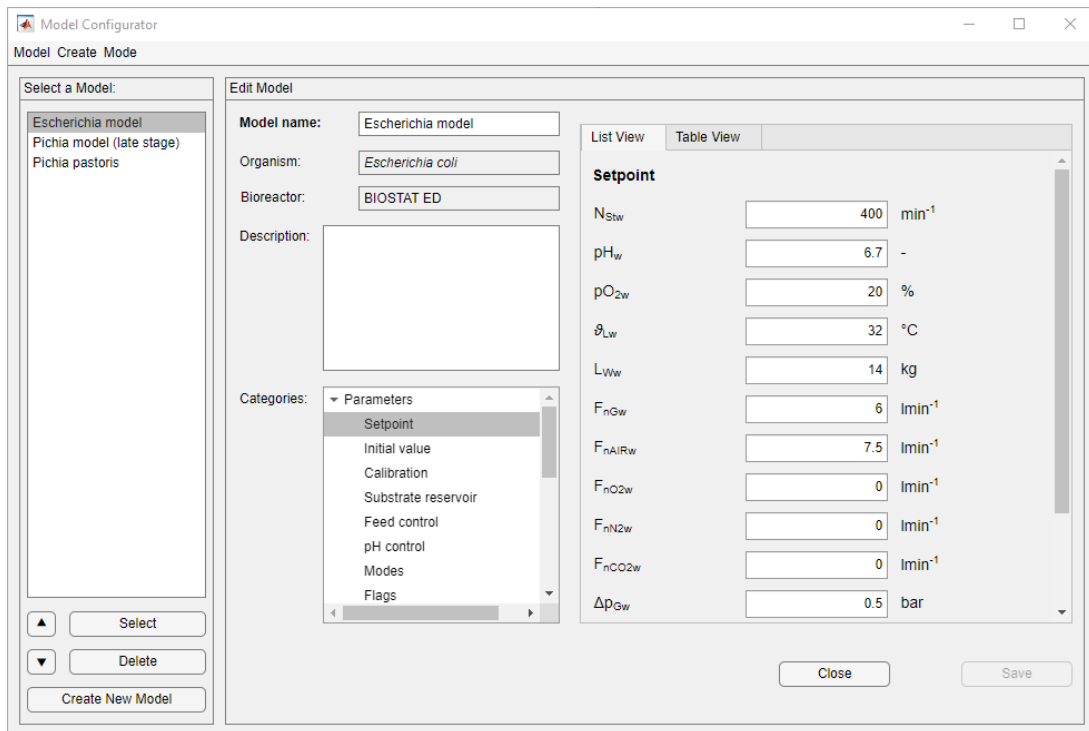


Figure 4.4: Model Configurator GUI

The Model Configurator enables the creation of individual parameter collections. These models allow customised parameter sets and are based on a combination of organism and bioreactor. The GUI is based on MATLAB®.

framework conditions, replicating real process scenarios even before the process begins. It was inspired by MFCS/win, a PCS that is utilized in the BPA laboratory (see Section 2.3.1). The functionalities of the Process Manager are described in Section 4.1.7.

The **logger** records all significant events that occur during a process. This includes parameter adjustments, as well as events such as the process start, inoculation time, or phase transitions. Event entries are generated in the script using a dedicated function, while parameter changes are automatically saved in the database via an update trigger.

The **Trend View** displays current values of selected variables. Additionally, a trend is calculated from the last ten values of each variable, indicating whether the trend is negative, neutral, or positive with an arrow. Current flow rates through pumps and volume balances in reservoirs are continuously displayed on the right-hand side of the tab.

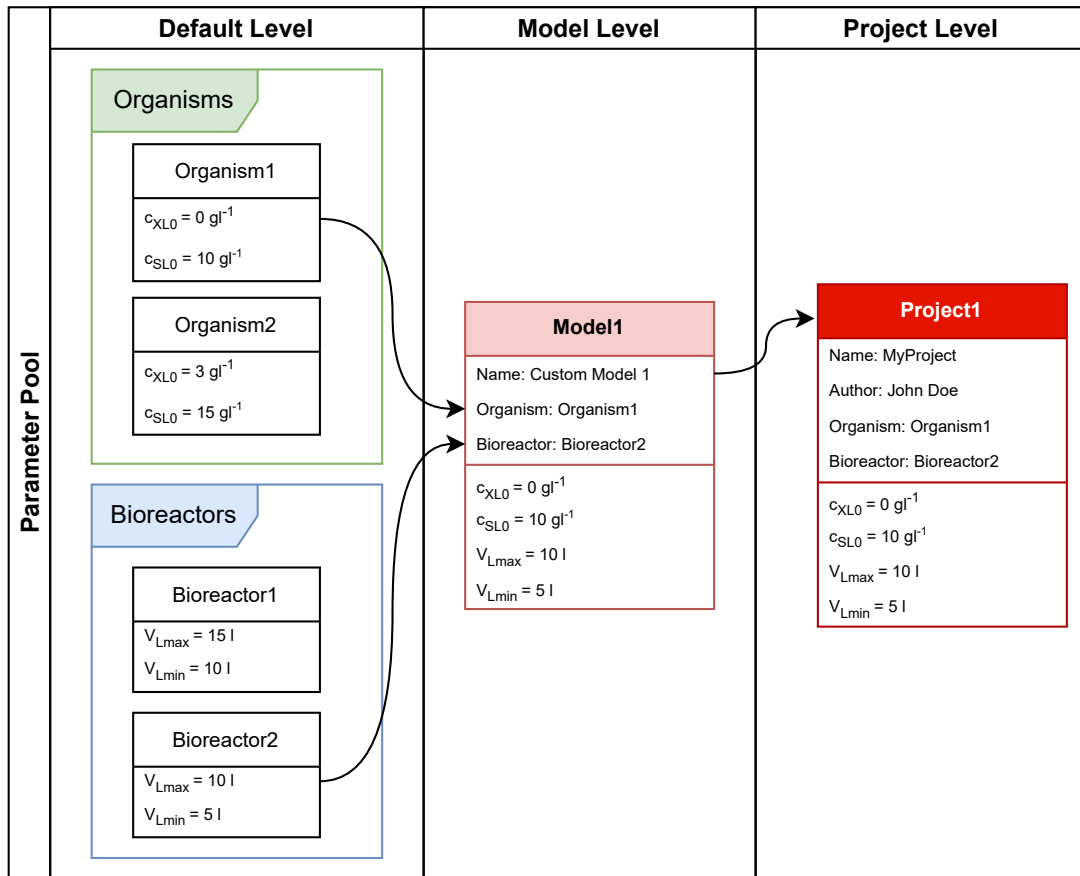


Figure 4.5: Illustration of implemented parameter organization

A default set of parameters is stored for each organism and bioreactor. When a model is created based on an organism and bioreactor, the corresponding parameters are merged and copied into the model table. These parameters can then be customised and used to initiate a project and thus a simulation.

4.1.3 Figure App

The "Figure App" has been completely reworked compared to the previous version (see Figure 4.7). Before, variables to be plotted were spread across two axes; now, each variable is assigned its own y-axis. These y-axes are positioned parallel to each other on the left side of the plot. The order of the axes is automatically sorted by their maximum values, with the highest y-axis value displayed at the outermost edge. The number of y-axes displayed simultaneously is limited to six. The displayed x- and y-axis ranges can be set manually in the axis editing area on the left, or assigned automatically by selecting a checkbox. Each variable has a default y-axis range stored

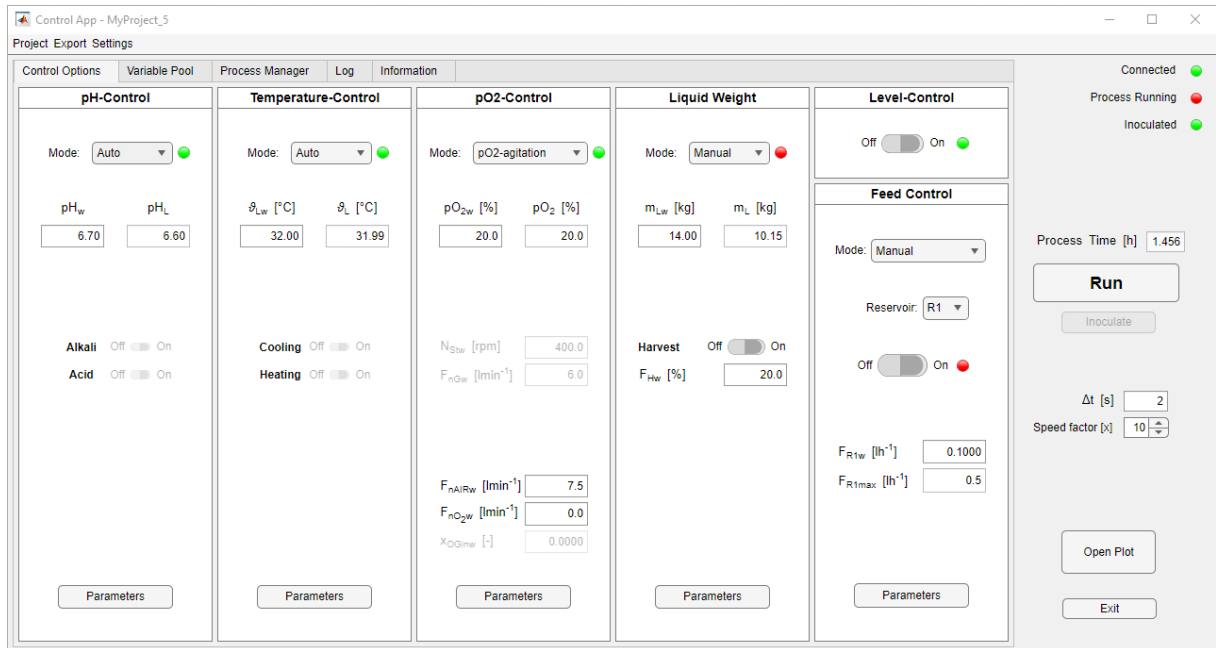


Figure 4.6: Control App GUI

The Control App is the main simulation UI. It consists of multiple tabs, each dedicated a specific task. In the Control Options tab, all control instances can be manipulated. The Variable Pool shows the trend a selection of variables. The Process Manager is a simple PCS framework, allowing process automation. In the Log, all events and parameter adjustments are displayed. The GUI is based on MATLAB®.

in the database, along with an option for automatic range determination. These and other variable properties, such as color or line style, can be customized through the "Edit Variables" menu. Additional settings, such as line thickness of the axes or font size in the plot, can be adjusted via the "Settings" menu. Changes are saved to the currently selected template, which contains an individual combination of variables for quick access to frequently used views. It is also possible to open multiple plot windows in parallel, if necessary.

4.1.4 Data Table

The raw data of a simulation can be viewed through the "Data Table" menu. The window opens with a list of all selectable variables, where multiple variables can be chosen using checkboxes. These selected variables are then displayed in a table with the corresponding time values. The table does not update automatically and must be refreshed during the ongoing process via the update button. The displayed table, con-

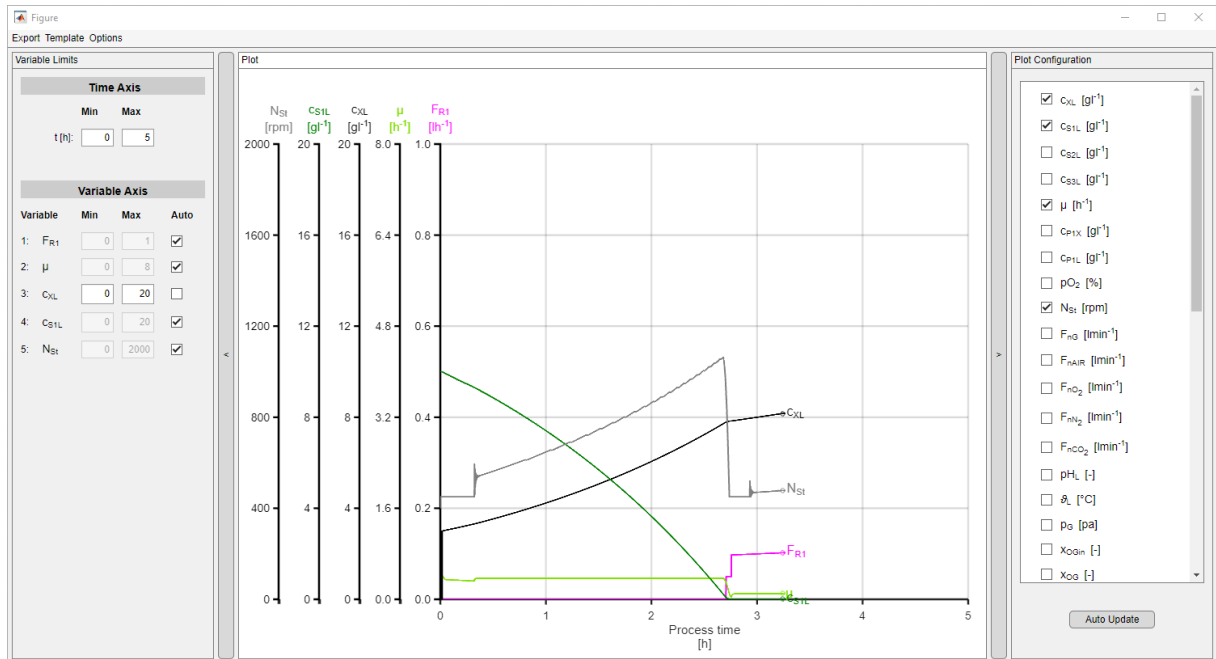


Figure 4.7: Figure App GUI

The Figure App visualizes the current simulation data. Up to six variables can be displayed in parallel. The y-axis limits can be determined manually or automatically. Multiple Figure App instances can be opened at a time. The GUI is based on MATLAB®.

taining all data, can be exported via the export menu, with options to choose between ".xlsx", ".csv", and ".txt" file formats.

4.1.5 Project Export

Evaluating a completed simulation requires the export of all relevant data from the database. This is done using the “Export” window, which can be opened via the “Export” menu. The variables to be exported and the desired format can be selected here (see Figure 4.8). In addition, a file with the logger data and a table with all parameter values can be generated. These are saved by default in a folder named after the project on the desktop.

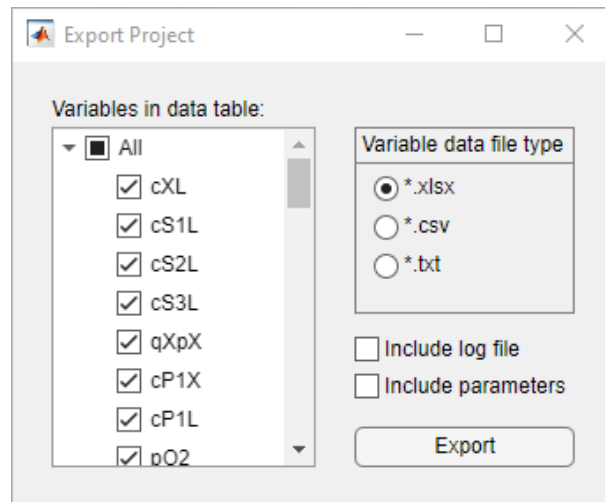


Figure 4.8: Project Export GUI

The Project Export GUI allows the user to export an individual collection of variable data. Additionally, a logger file and all parameter values can be exported. The GUI is based on MATLAB®.

4.1.6 Process End

If the program is closed via the close-menu or the x-button in the corner of the "Control App", a window will open to determine how to proceed with the project (see Figure 4.9). The project can be saved, saved and exported, or deleted. If the user wishes to resume the simulation, the Process End window can be closed by selecting "Return".

4.1.7 Process Automation

In the context of this work, a conception was introduced in the simulation software that allows for the automation of the process. This was done on the basis of the MFCS/win PCS. Fermentation processes are usually divided into different phases. The transition from one phase to the next requires the adjustment of several parameters. The process phases can be managed in the Control App via the Process Manager tab (see Section 4.1.2). An unlimited number of phases can be added and each can be deleted.

A phase consists of a name, type, starting and ending condition, and a status. The different components of the phases are explained below.

Figure 4.9: Closing Screen GUI

The Closing Screen allows the user to update project information. Before closing the project, the user decides whether the project should be saved or deleted.

Phase Types

Five phase types have been introduced with the Process Manager. The empty phase is intended to represent an empty phase and can be used as a bridge. The main reason for the introduction of this phase type is the batch phase, which represents a phase but usually does not require any further automation steps after inoculation. The second phase type is the stop phase. This phase can be selected if the process is to be stopped after a certain condition has been reached. For example in case when a critical reactor volume is exceeded during the fed-batch phase. The third phase is called “Parameter Update” and allows the controlled adjustment of any parameter that is read cyclically from the database. The fourth phase type is called “Pulse Feed” which allows constant substrate feeding from a selected reservoir during the entire time this phase is active. The feed rate can be set in a separate window depending on the maximum feed rate of the pump. The fifth phase type introduces open-loop feed control. This phase type allows the execution of a process phase that is generally known as a fed-batch. All relevant parameters can be set individually in advance in a separate window. When this phase starts, the initial feed rate F_{Rj} is determined according to Equation 2.30 and then the new feed rate is calculated cyclically according to the open-loop principle

shown in Equation 2.31, resulting in an exponential feed profile.

Starting and Ending Condition

Each phase is assigned a starting and ending condition, which is used to control the status of the phase. Different types of conditions have been defined for this purpose. Several start conditions have been implemented. With “End of previous phase”, the phase begins immediately after it has been given the status “pending”, which occurs automatically after the end of the previous phase. With the "Variable Condition" feature, a phase can be initiated as soon as a specified condition for a selected variable is met. Users can choose either the process time or a variable from a predefined list and apply relational operators to define the condition. The last start condition is the batch end detection, in which a transition from batch to fed-batch phase takes place, based on the course of oxygen partial pressure and stirrer speed.

The end of the phase can be initiated in three different ways. Firstly, the end of the current phase can be signaled via “Next phase condition”. This happens as soon as the start condition of the next phase applies. Secondly, the end of the phase can occur via a variable condition. The principle behind this is identical to that for the start condition described above. Thirdly, the end of the phase can be defined via a timer. To do this, a phase duration in hours is specified and added to the process time at the start of the phase. As soon as the process time is greater than or equal to this calculated time, the phase is terminated and assigned the status “completed”.

4.2 Back-End

4.2.1 SQLite Database

The core of the simulation platform is a central database, which stores all uploaded data according to a predefined scheme. In the development of a concept for a database system, necessary elements, such as tables and triggers, were created in a DBMS. In this section, the database is illustrated in the form of ER diagrams.

The database schema can be divided into three sections. The first section contains all tables that are directly related to the simulation. The second section, the process tables, contains all the information that is relevant in the process manager. The last section (plotting tables) contains all the data and settings relevant for generating plots. The types of relationships between the tables are not elaborated in detail here, as optimizing these relationships was beyond the scope of this work and represents a potential area for future projects.

Simulation Tables

Figure 6.2 shows an ER diagram with the tables relevant for the simulation. These can be further subdivided into two table groups.

The information for the different simulation models is stored in the model group. The models are all based on a specific organism that is cultivated in a assigned bioreactor. When creating a new model, an organism is selected from *organismTab* and a bioreactor from *bioreactorTab*. For each of these entities, a collection of default parameters is stored in the tables *default_modelTab* and *default_bioreactorTab*. The table with default values of organisms is called *default_modelTab* because it contains not only parameters that are unique to the organism, but also natural constants that are required for the simulation of the model.

When a new model is created, information such as the name of the model is stored in *modelTab*. In addition, foreign keys refer to the organism and bioreactor used. Furthermore, the respective parameter data from the default tables are copied into *model_parameterTab*, where they can be altered in the Model Configurator without changing the default parameters.

The second group includes all tables that are related to projects. A project is always created prior to the start of a new simulation process and takes over the information from the selected model. As a result, a project also refers to the underlying model. However, no reaction has been defined in the relationship between project and model if the model is deleted in the project table. This ensures that all projects (child) continue to exist even after a model (parent) has been removed, leaving an empty reference. After a project has been created, all parameters are copied from the respective table

model_parameterTab to *project_parameterTab*. When the simulation is started, time stamps are then uploaded cyclically to *timeTab* with a reference to the project. In the course of this, data of the variables are uploaded to *dataTab* with a reference to the respective timestamp. Events and parameter adjustments by the user are stored in *logTab*. Entries are automatically created in *logTab* via a trigger.

Outside of the two groups are the tables with information on parameters and variables, which are to be considered separately from the other tables in the course of normalization. The table *parameterTab* contains the name of the parameter as well as further information such as the unit or the parameters category. All parameters are categorized into specific groups such as set point, control parameters or initial values via *categoryTab*. The retrieval rate of the parameters from the database is defined in mentioned table.

In addition to the parameters, information about the variables is also considered separately. Thus, only a foreign key of a variable is stored in *dataTab*, which stores all data of the static and dynamic variables calculated in the simulation. The foreign key refers to the primary key of *variableTab*, in which attributes of the respective variable, such as its unit and, if applicable, a description, are stored. This design ensures consistent nomenclature for process variables, for example the cell concentration c_{XL} across different models. If new variables are to be introduced in the course of a new organism being added to the simulation, *variableTab* can simply be extended to include the new variables and the project can then refer to these variables. New parameters can be introduced according to the same principle. These variables are then assigned a new primary key, to which the actual values can then refer. In this way, normalization takes place and data redundancy is minimized.

With the introduction of the process manager to the simulation platform, a way had to be created to store process phase information in the database. In general, all phases that are created are stored in *processTab*, including a foreign key of the respective project and all relevant properties of the phases. The name of the phase, the current status and the phase type are defined in the table. In addition, all necessary conditions for the phase starting and ending are stored there. Taking into account the principles of normalization, all attributes for which predefined options are available have been outsourced to child tables. Therefore, for example, the four statuses “upcoming”, “pending”, “active” and “complete” are defined in *process_statusTab*. In *pro-*

cess_operatorTab the different comparison operators for the variable check are stored and in *process_conditiontypeTab* the different conditions to be checked for the beginning or the end of the phase. The table *process_parameterTab* additionally refers to *parameterTab*. For this reason, all parameter values to be changed with the phase start are stored in there. *process_variableTab* lists all the variables that are available for selection as variable conditions.

Plotting Tables

The plotting group combines all tables that provide information for generating plots. It is possible to create templates, containing both a selection of variables and the settings for the desired plot. Thus, all settings are stored as attributes in *plot_templateTab*. In addition, all variables that can be plotted are stored in *plot_variableTab*, along with the properties of the individual variables, such as color or the y-axis limits. Creating a template adopts the current settings, allowing quick access to regularly used variable combinations. To enable the templates to be used across organisms, there are no direct or indirect references to *organismTab* in the plotting tables. The organism-specific variable collection is defined in elsewhere, and are retrieved via a query in the Figure App. The possible properties for the variables to be plotted are stored in *plot_linestyleTab*, *plot_colorTab* and *plot_decimalTab* to avoid redundancies.

4.2.2 Calculation App

In this simulation platform concept, a script was introduced that is responsible for calculating the simulation and generating variable data. It was ensured that the script and UI ran independently of each other as much as possible, in order to enable the addition of new interfaces in the future. In the present framework, it was not yet possible to decouple them completely from each other, because some functions that are available in server-based database systems are not supported by SQLite. Figure 4.10 shows the schematic course of a simulation. The user calls up a project in the starting screen, which causes the script for the simulation and the Control App to be called up as the corresponding UI. Both instances are MATLAB® classes and receive the corresponding key of the project (*projectID*) as input values, which allows all necessary information

from the database to be called up in the project initialization. All variables relevant to the project will be initialized and will take either the value stored in the model for $t = 0$ h or all values from the previous run. In addition, all relevant parameters are downloaded from the database and initialized in a struct according to Section 3.1.2.

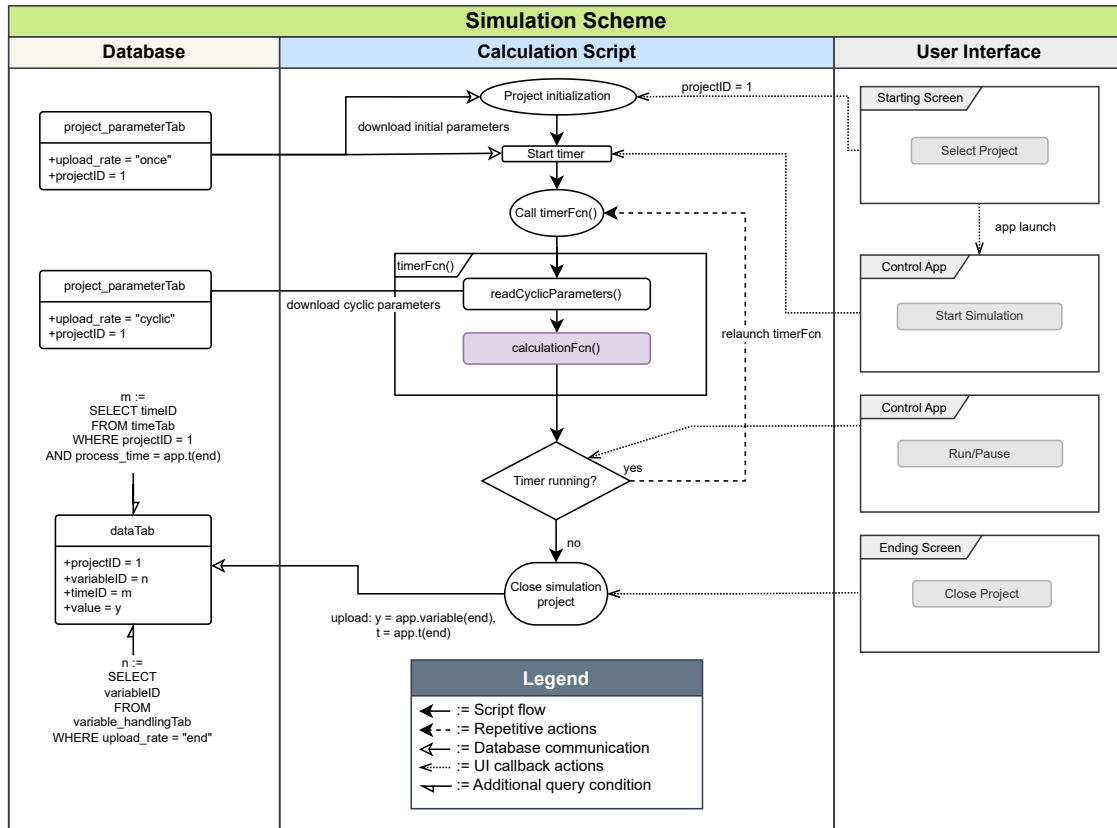


Figure 4.10: Flow diagram of the Calculation App and its interfaces

Following the project initialization in Starting screen, relevant information is downloaded from the database. The simulation start signal from the Control App launches the timer function `timerFcn()` in the Calculation App, where parameters with the attribute "cyclic" are downloaded and the `calculationFcn()` is called up. The flow of the `calculationFcn()` is illustrated in Figure 4.11. The timer periodically calls up the `timerFcn()`, until the it retrieves a stop signal from Control App. Once the Control App is closed by the user, additional variable data of the last cycle is uploaded to the database. The variable data was not previously uploaded cyclically and is required as initial values if the project is called up again. This data The flow diagram was created with draw.io.

Once the user starts the simulation by clicking the “Start Simulation” button, the timer-Fcn() starts and, as a result, the functions readCyclicParameters() and calculationFcn() are called at regular intervals of Δt .

The function readCyclicParameters() reads all parameters from the database that have the property “cyclic” for the attribute “reading_rate”. These include parameters such as set points or control parameters. The function calculationFcn() is responsible for calculating variable values and can be divided into three sections, as illustrated in Figure 4.11. The first and third sections check the status of process phases in the Process Manager. In the first section, it is checked whether a phase is active. If no phase has the status “active”, the start condition of the phase with the status “pending” is checked. After the initial start of the project, the first phase is automatically assigned the status “pending”. If the start condition of the phase is met, its event is executed according to Section 4.1.7 and the phase is given the status “active”. If no phases have been set up, this step is skipped.

In the second section of calculationFcn(), the simulation calculation is conducted and variable data generated. The organisms that can be simulated within the simulation platform are based on different models and therefore different formulas for calculating the output variables. Hence, a modular approach had to be chosen here, in which the model script of the organisms was outsourced to m-files. The exact name of the corresponding file is stored in the *organismTab* table under the “calculation_file” attribute. This name was read in the initialization of the project and is converted into a function as a string using the MATLAB function str2fcn(). The organism-specific function receives the entire class as input and returns an adapted version of exactly this class as output, shown in Listing 6.6 in the appendix.

The m-function has access to all instances of the class because it receives the entire class as an input. In the organism-specific function file, all control-related operations are then carried out and the numerical solution of differential equations is determined, analogous to the previous version of the software. The resulting data is then appended to the end of the respective variable array. After the last line of the function file has been reached, the adapted class is returned and replaces the previous class of the application.

The next step is to upload the data generated in this cycle to the database. First,

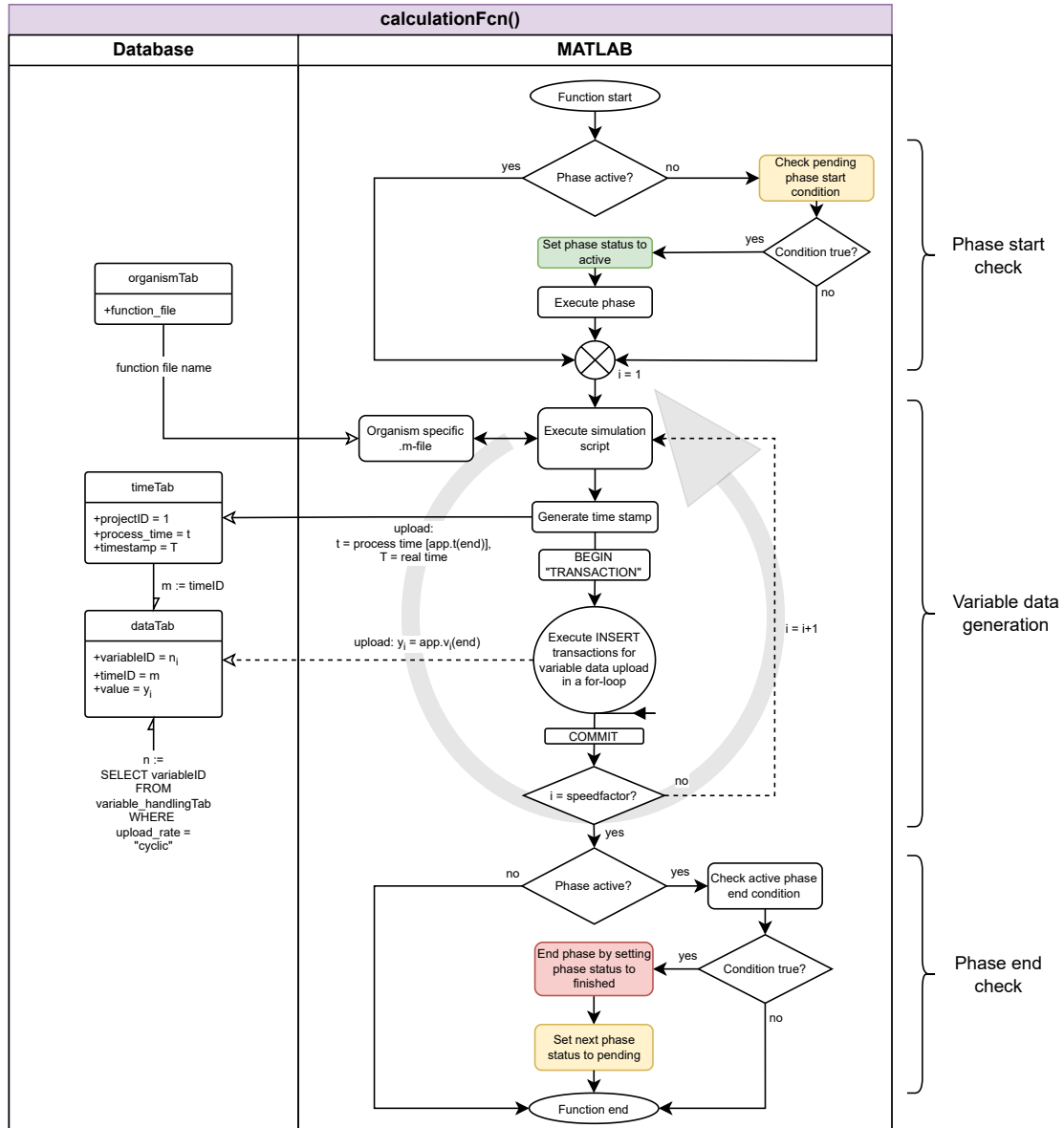


Figure 4.11: Flow diagram of the calculationFcn()

The `calculationFcn()` is separated into three sections. In the first section, the current phase status is checked, while in the second section, the simulation calculation is executed. In the third section, the ending condition of the phase is checked, once a phase is active.

a timestamp is created, with the process time and the real time being uploaded to *timeTab* with reference to the project. Then, the last added values of the variables are uploaded to the *dataTab* table in a loop using INSERT queries, where a reference to the associated timestamp is made using a nesting.

The simulation allows the process to be accelerated using the speed factor parameter. In the previous version, this was solved by increasing the frequency of the timerFcn() according to the speed multiplier. This version, a different approach was taken, since increasing the frequency of the timer function, also increases database interactions and thus possibility for issues with data integrity. Also, delays in computation would only make the queue grow longer, with new instructions always placed at the end of the queue and thus executed with a long delay. To avoid this, the execution of the function file was packed into a for-loop, which is executed according to the speed factor.

After the calculation of the last cycle, the phase status is checked again in the third section. If a phase is declared as “active”, the end condition of this phase is examined. If this condition is met, this phase is declared as completed and the next phase is set to “pending” when it exists. If the condition is not met, the active phase remains active.

After calculationFcn() and timerFcn() have finished, timerFcn() is called as soon as Δt has elapsed and the next cycle starts, as long as the timer is active.

If the user decides to end and save the simulation, data of the less relevant variables from the last cycle are uploaded to the database. This is necessary because many calculations, such as the solution of ordinary differential equation systems, require a starting value. Before uploading the data of variables with the entry *end* for the attribute *upload_rate*, previously uploaded end data is deleted. The data of these variables is then transferred to the database in the same way as cyclic upload and can be accessed after the project has been reloaded.

4.3 Model Implementation

In order to develop a mechanistic mathematical model of such a complex process as the fermentation of an organism, many assumptions and simplifications have to be made. As in the simulation software BioProcessTrainer [14], the underlying *E. coli* model can be divided into different sub-models. However, these sub-models interact with each other, which complicates the complete separation of the sub-models.

While the physicochemical sub-model, which represents the temperature and pH dynamics, remained almost unchanged, some adjustments had to be made in the bioreactor sub-model. This sub-model describes the inflow and outflow rates to and from the bioreactor. Two reservoirs are introduced for the *P. pastoris* model, while there was only one in the *E. coli* model. Therefore, the volume ODE had to be adjusted (see Listing 6.6). The control sub-model has also been expanded with the introduction of the PCS. It now includes both open-loop and closed-loop feed control, as well as constant feed.

However, the most significant adaptation has been made to the biological sub-model. Similar to the BIOSIM model, it is also based on the bottleneck principle [23]. In this model, the cell-specific growth rate is either substrate-limited or oxygen-limited. In the *P. pastoris* model, however, the substrates glucose as substrate 1 (S1), glycerol as substrate 2 (S2), and acetate as substrate 3 (S3) were exchanged for glycerol as S1 and methanol as S2. This leads to a new dynamic, since acetate as simultaneous product P1 is no longer present.

Here, a new dynamic has been introduced, namely the induction of AOX and the expression of the target protein of the *P. pastoris* phenotype Mut^S (see Section 2.2). In addition, target protein secretion has been introduced, in which a distinction is made between the concentration of the target protein inside the cell c_{P1X} and the dissolved protein concentration c_{P1L} . In this context the target protein is referred to as P1. Although the protein chemokine 1-8del MCP-1 was produced by the *P. pastoris* strain in Cornelissen's original model, no specific protein is defined in this model. Thus, a desired protein can be selected for each scenario to be represented in the simulation, since a mass concentration is always given and not, as in the BIOSIM model, a molar concentration. The implementation of the sub-models can be seen in the MATLAB®

script in the Appendix 6.6.

4.4 Exemplary Process

4.4.1 Fully Automated Model Process

For the investigation of the growth dynamics of the newly implemented organism *P. pastoris*, a cultivation inspired from a real process was carried out. The cultivation strategy is explained in Section 3.5. Since the model is not yet fully parameterized, this fermentation run cannot claim to have an identical course as in reality. Instead, the aim is to discuss the dynamics of the system and to test the newly introduced PCS. While normal *P. pastoris* cultivations can take up to several days [6, 8], this process is conducted as a short version of it. The process strategy displayed on panels within the Process Manager, is shown in Appendix 6.5.

Figure 4.12 shows the course of the simulated cultivation. The batch phase was conducted with an initial glycerol concentration of 10 gl^{-1} . After inoculating, the oxygen partial pressure $p\text{O}_2$ went down to 0%. This happened, even though $p\text{O}_2$ -agitation control was active. With a short delay, stirrer speed rose to control $p\text{O}_2$ to 20%. The phenomenon behind the late reaction of the controller is called integral windup, where a large error accumulates within the integral part of the PID controller leads to a delayed reaction [2]. After $t = 0.6$ h, the maximum cell-specific growth rate of $\sim 0.335 \text{ h}^{-1}$ was reached. Due to increased oxygen demand with growing cell density, the aeration with air had to be manually increased to not reach the maximum capacity of the stirrer. After $t = 3.5$ h, the batch phase has ended and the 2 hour fed-batch phase began. This was initiated by the PCS, feeding glycerol at a defined rate to maintain a set growth rate of μ_{1w} of 0.1 h^{-1} . It can be observed that substrate-limited growth performed well, keeping the substrate concentration close to $c_{\text{S1L}} = 0 \text{ gl}^{-1}$, while maintain the desired growth rate.

With the end of the fed-batch phase, methanol began to be pumped into the bioreactor. The pump rate was set a too high, resulting in a methanol concentration of approximately $c_{\text{S2L}} = 7 \text{ gl}^{-1}$ ($\sim 0.9\% \text{ [v/v]}$) instead of the desired 5 gl^{-1} .

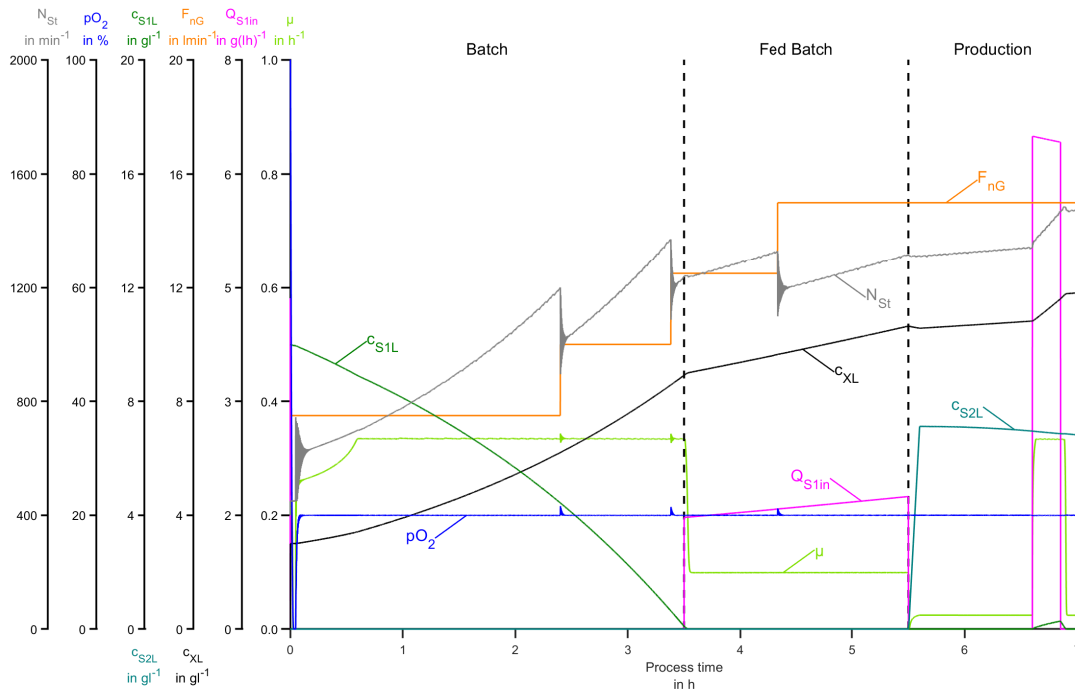


Figure 4.12: Plot of the simulated process

Shown is the plot of a simulated fed-batch fermentation of *P. pastoris*. The process is divided into three phases. In the batch phase, the cells have metabolized the provided substrate glycerol (c_{S1L}). As soon as this was completely digested, c_{S1L} was continuously pumped in via an exponential feeding profile. In the subsequent production phase, methanol (c_{S2L}) was pumped in, which induced the production of a target protein.

In order to evaluate the dynamics in the presence of two substrates, glycerol was pumped into the bioreactor for 0.25 hours at a rate of 0.25 l h^{-1} . After $t = 7.1 \text{ h}$, methanol was steadily pumped into the bioreactor at a rate of 0.5 l h^{-1} , reaching a concentration of more than 70 gl^{-1} ($\sim 9\% \text{ [v/v]}$) in the end of the cultivation. The dynamics of the production phase are discussed in the following sections. Based on the course of the simulated *in silico* *P. pastoris* cultivation, similar dynamics to real cultivations can be observed [8, 6, 26].

4.4.2 Evaluation AOX Induction and Target Protein Production

AOX Induction

For the evaluation of the newly implemented AOX induction and target protein production models, a simulation process with *P. pastoris* was conducted. In the cultivation, subsequent to the fed-batch phase, methanol was pumped into the bioreactor. To observe the dynamics of the AOX induction in the presence of glycerol, mentioned substrate was continuously given into the bioreactor, one hour after the production phase has started.

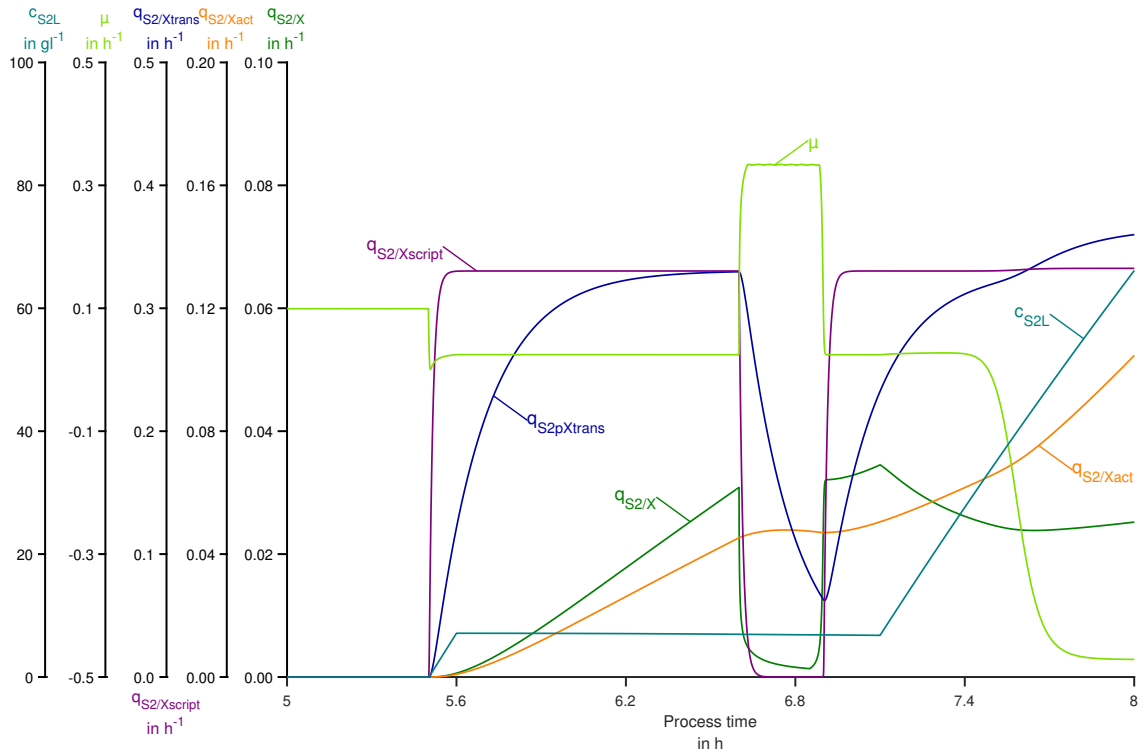


Figure 4.13: AOX induction in a simulated *P. pastoris* process

Figure 4.13 shows the progression of all variables related to AOX induction. It can be seen that the induction rate $q_{S2/Xind}$ rose sharply in parallel with the start of the methanol feed at $t = 5.5$ h. This is due to the switch-on and -off functions. Since there was already no glycerol in the medium at the end of the fed-batch phase, the switch-off function was no longer active. With the additional activation of the switch-on function by the addition of methanol, the induction rate was activated. Slightly delayed but quickly

following $q_{S2/X_{ind}}$, the AOX induction transcription rate $q_{S2/X_{script}}$ increased. The progression of $q_{S2/X_{trans}}$ then shows the rate at which the mRNA for the AOX gene was transported to the ribosomes. The activity of AOX expression can then be recognized via $q_{S2/X_{act}}$. Thereupon, the substrate uptake rate $q_{S2/X}$ gradually increases with the increasing amount of AOX. At $t = 6.6$ h, glycerol was added via pulse feed, which led to an abrupt drop in the induction potential $q_{S2/X_{ind}}$ and subsequently in the coupled expression of AOX. As a result, the substrate uptake rate $q_{S2/X}$ rapidly decreased, since it is regulated by both glycerol and methanol concentration (see Equation 2.7). Meanwhile, the AOX activity is slightly decreasing. When the temporarily added glycerol was completely consumed, the induction rate rose again, followed by the substrate uptake of methanol.

In the end, methanol was continuously pumped into the bioreactor to observe the toxic effect of the substrate on the organism. As intended, this led to a negative cell-specific growth rate $\mu = q_{X/X}$, which will be further discussed in Section 2.2.4. The negative growth rate continuously decreased the cell concentration (see Figure 4.12) but also led to an unintended interaction with the AOX induction model. As can be seen in the differential equations 2.4, 2.5 and 2.6, the cell-specific growth rate supposed to have a negative effect on the respective rates. Here, as can be observed at $t = 7.4$ h, μ being negative, it is positively influencing the growth rates. In the end, this leads to an increased methanol uptake of the remaining cells, while they are supposed to be impaired in their vitality. The same applies to the target protein expression.

Target Protein Expression and Secretion

Figure 4.14 shows the expression of the target protein controlled by the strong *AOX1* promoter. It can be seen that, analogous to AOX induction, an induction takes place by the presence of methanol. This then gradually leads to transcription, preparation for translation and finally to expression of the target gene. As a result, an increase in the concentration of protein c_{P1X} within the cells can be observed. This protein is then secreted and is then present in the medium in the form of c_{P1X} .

When glycerol was added again during the production phase, a sudden drop in protein formation was observed. However, protein formation increased again in the same pattern when the glycerol was used up. It is noteworthy that the protein continued

to be secreted, resulting in a delayed stagnation of the change in dissolved protein concentration.

In the course of methanol feeding to verify the toxicity model, the decreasing cell concentration led to a decrease in the intracellular target protein concentration. Similar to the AOX induction, however, the negative growth rate led to a slight increase in the target protein formation rate $q_{P1/X}$ (see Equations 2.17, 2.21 and 2.22). This should also be revised in the future, since target protein formation should decrease rather than increase when cell vitality is impaired.

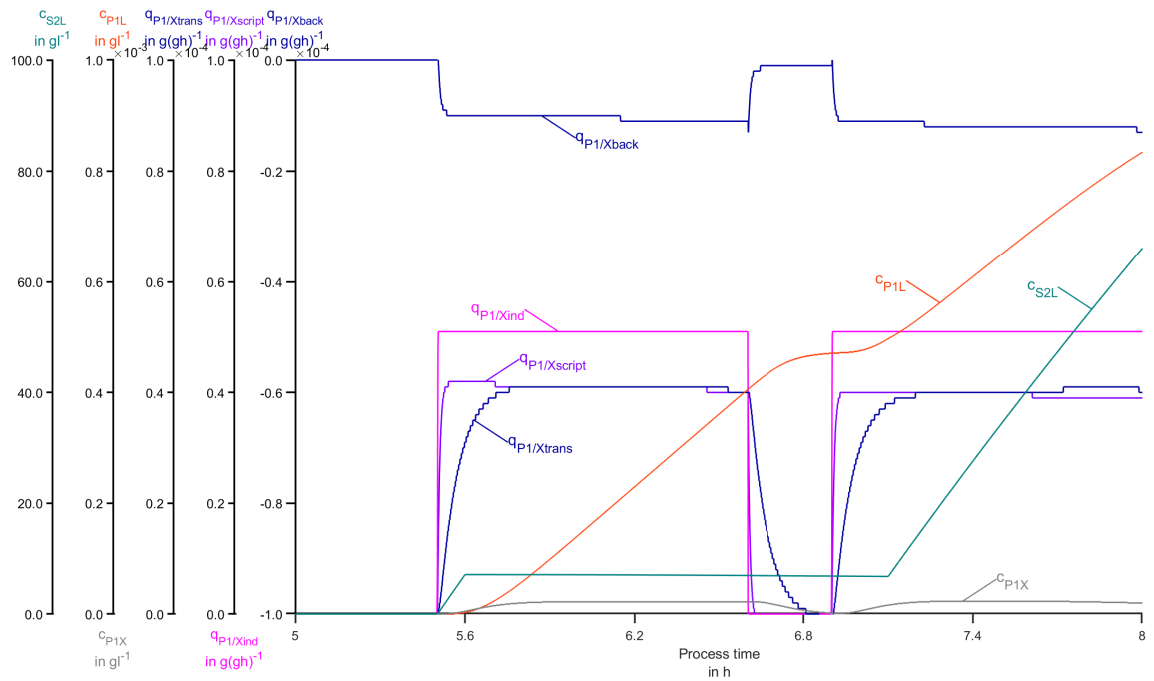


Figure 4.14: Target protein formation in a simulated *P. pastoris* process

4.4.3 Methanol Toxicity

The cytotoxic nature of methanol in cultivations with *P. pastoris* is a major challenge and should therefore be implemented in this model. Figure 4.15 shows the interaction between methanol concentration and cell-specific growth rate. From $t = 7$ h, methanol was added constantly until a concentration of over 70 gl^{-1} ($\sim 9\% \text{ [v/v]}$) was reached. The volumetric substrate feeding rate of methanol decreased despite the constant addition rate due to the increasing volume. While the methanol concentration increased constantly, the growth rate began to decrease at around 30 gl^{-1} It decreased with a sigmoidal curve profile. From about 55 gl^{-1} the growth rate reached a negative plateau, with the maximum methanol-induced death rate $q_{X/X_{\max 1}}$ according to Equation 2.28, leading to a death rate of -0.472 h^{-1} . With this curve, the model for the toxicity of methanol has been successfully represented.

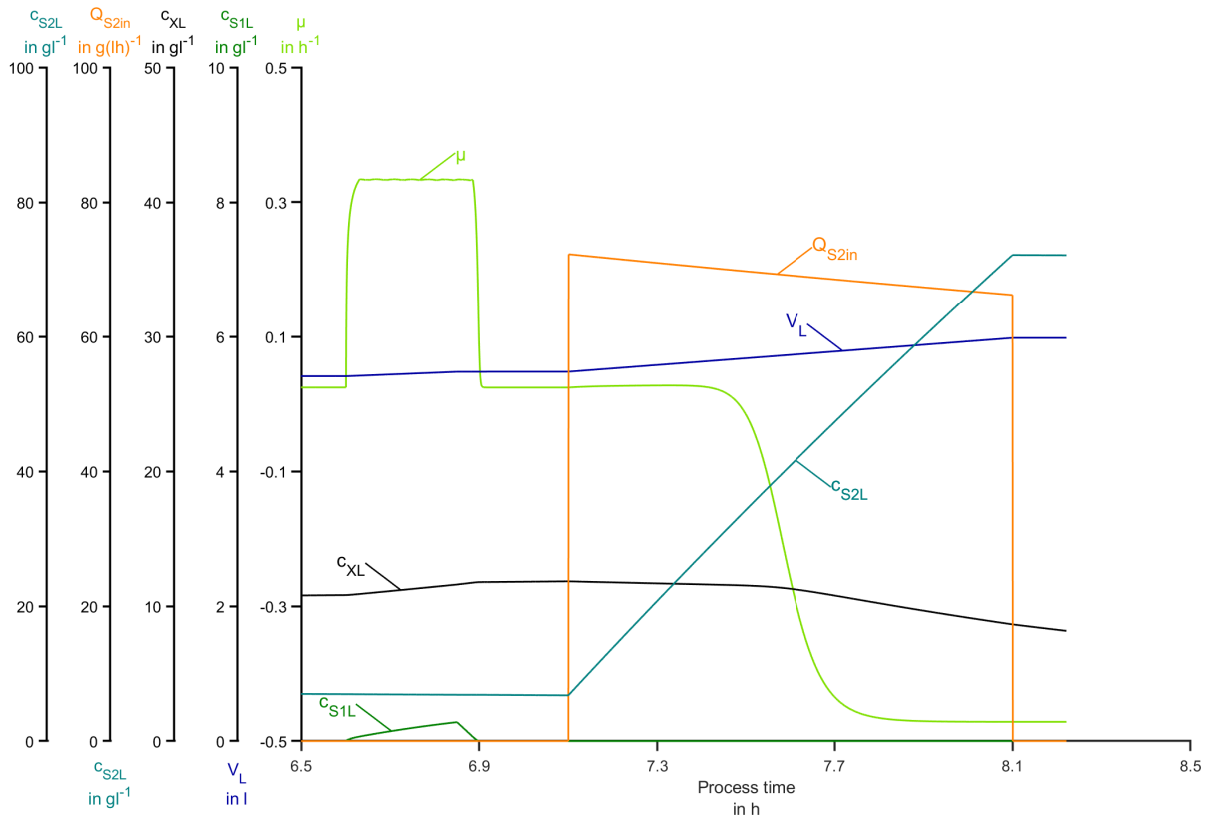


Figure 4.15: Methanol toxicity in a simulated *P. pastoris* process

4.4.4 Limitations of the Platform Concept

Due to the division of the platform into database, script and clients, performance difficulties arose within MATLAB®. The timer function calls the respective functions on all open clients at regular intervals of Δt . Since clients and script both run via MATLAB®, all timer functions share a queue. This can lead to long delays, especially when the speed factor is set to more than 5. If the calculation within the simulation takes longer than Δt , the functions called afterwards are queued. The queued functions are then executed in prioritized order, causing all clients to freeze. As a result, the simulation can no longer be stopped from the Control App, nor can parameter changes be forwarded to the database. For this reason, it is important to limit the speed factor and, in the long term, to run clients and the script on independent threads.

Another problem that occurred a few times was that a client wanted to access the SQLite database while it was locked. This led to MATLAB® trying to access it repeatedly, even though the database was unlocked. The set time out time could not help in this scenario either. This is a disadvantage if a lightweight database is used instead of a server-based database with access management in the middleware. In the long term, especially with the function as a Digital Twin, and multiple interfaces regularly access the database parallelly, it is advisable to switch to a server-based database.

The Process Manager currently only allows the input of one process line, whereby, for example, the automated update of parameter values within the fed-batch phase does not work, as the exponential feeding profile would be interrupted. Therefore, it would be logical to consider how to accommodate the open-loop feed control differently in future versions.

5 Summary and Outlook

In this work, a platform programmed in MATLAB® was presented that allows the simulation of different organisms. This was achieved by creating a framework that can be continuously expanded. The user interfaces act as clients that access a central database. All variable data is stored in a standardized way in the database, allowing users to access and continue past processes. Approaches were developed to remove all organism-specific structures from the core framework of the platform and to convert them into callable modules. In the context of software as OTS, the platform can be used not only as a training tool for process control but also allows students to independently develop new automation concepts and test them within the simulation. In this context, students could be given the task to independently develop a function within MATLAB® that recognizes a batch end, based on the course of variables. In addition to the development of the simulation platform, a new organism was introduced. The methylotrophic yeast *P. pastoris* brings new characteristics that were tried to be modeled within the simulation. For this purpose, the biological sub-model of *E. coli* was adapted and the methanol-induced production of the target protein was introduced. Therefore, the model developed by Cornelissen (2004) for AOX induction and target protein expression was translated into MATLAB® and extended to include protein secretion dynamics. Furthermore, a concept was developed to simulate methanol-induced cell death. However, unwanted interactions due to a negative cell-specific growth rate were discovered, which could be a basis for future improvements and optimizations of the model.

In addition to the further development, optimization and validation of the *P. pastoris* model, other organisms could be incorporated into the simulation platform. An example of this would be mammalian cells, which are not yet practically handled in the BPA laboratory. The simulation of mammalian cell cultures could thus provide students with new insights that they have not been able to gain before.

In the fermentation run shown in Figure 4.12, there was a delayed response of the PID controller due to the accumulated integral term (I term). This occurred because a significant integral sum value had built up shortly before, which had to be compensated for before the controller could issue a signal to increase the stirrer speed. Such delays can be prevented by using an anti-windup system. An anti-windup mechanism limits

the accumulation of the I-component, so that the controller can react faster to changes without being blocked by previous integral sums [2]. However, appropriate limits for limiting the integral component must be determined experimentally, as they depend on the specific characteristics of the fermentation process. The introduction of an anti-windup mechanism therefore represents a possible target for future optimization.

Following this work, an offline mode will be integrated to improve performance for using the platform as an OTS. When this mode is activated, the each client will no longer operate independently but will instead build upon one another, similar to the previous version. Since only a single timer runs within MATLAB®, freezing caused by long queues can be largely avoided. Furthermore, it completely eliminates the possibility of multiple parties accessing the SQLite database.

The aim of this work was to lay the foundation for the development of a Digital Twin within the Bioprocess Automation Laboratory. This Digital Twin is to be used to optimize processes and be able to predict the course of the process. The idea was to have a central database that is supplied with data via several interfaces. These data can then be evaluated in real time using simulation models and problematic dynamics can be detected early on.

In the long term, the database scheme introduced in this work could be moved to a server-based database and manage all models and projects centrally there. The UI could request information from the database via application programming interfaces (APIs) and send parameter adjustments. The user interfaces can be translated from MATLAB® into the programming languages suitable for websites (HTML/JS/CSS). The simulation could continue to run via a MATLAB® script, but it could also be translated into a similar, more powerful programming language such as Python.

Bibliography

- [1] Jörg Andrä, Falk Beyer, Gesine Cornelissen, Jörn Einfeldt, Jens Heseding, Klaus Kümmerer, Kim Oelkers, and Carolin Floeter. PharmCycle: a holistic approach to reduce the contamination of the aquatic environment with antibiotics by developing sustainable antibiotics, improving the environmental risk assessment of antibiotics, and reducing the discharges of antibiotics in the wastewater outlet. *Environmental Sciences Europe*, 30(1):24, December 2018. ISSN 2190-4707, 2190-4715. doi: 10.1186/s12302-018-0156-y. URL <https://enveurope.springeropen.com/articles/10.1186/s12302-018-0156-y>.
- [2] Ahmad Taher Azar and Fernando E. Serrano. Design and Modeling of Anti Wind Up PID Controllers. In Quanmin Zhu and Ahmad Taher Azar, editors, *Complex System Modelling and Control Through Intelligent Soft Computations*, volume 319, pages 1–44. Springer International Publishing, Cham, 2015. ISBN 978-3-319-12882-5 978-3-319-12883-2. doi: 10.1007/978-3-319-12883-2_1. URL http://link.springer.com/10.1007/978-3-319-12883-2_1. Series Title: Studies in Fuzziness and Soft Computing.
- [3] Herbert H. Bell and Wayne L. Waag. Evaluating the Effectiveness of Flight Simulators for Training Combat Skills: A Review. *The International Journal of Aviation Psychology*, 8(3):223–242, July 1998. ISSN 1050-8414, 1532-7108. doi: 10.1207/s15327108ijap0803_4. URL http://www.tandfonline.com/doi/abs/10.1207/s15327108ijap0803_4.
- [4] Andree Blesgen and Volker C. Hass. Efficient Biogas Production through Process Simulation. *Energy & Fuels*, 24(9):4721–4727, September 2010. ISSN 0887-0624, 1520-5029. doi: 10.1021/ef9012483. URL <https://pubs.acs.org/doi/10.1021/ef9012483>.
- [5] Amy Brown. Top product forecasts for 2023. *Nature Reviews Drug Discovery*, 22(1):8–8, January 2023. ISSN 1474-1776, 1474-1784. doi: 10.1038/d41573-022-00193-0. URL <https://www.nature.com/articles/d41573-022-00193-0>.

-
- [6] Phoebe Chan. Investigations in Raman Spectroscopy as a PAT Tool Applied In Cultivations with *Pichia pastoris*. Master's thesis, June 2023. URL <https://reposit.haw-hamburg.de/bitstream/20.500.12738/13770/1/ChanPhoebeMA.pdf>.
- [7] Fuduan Chen, Hao Zhong, Ging Chan, and Defang Ouyang. A Comprehensive Analysis of Biopharmaceutical Products Listed in the FDA's Purple Book. *AAPS PharmSciTech*, 25(5):88, April 2024. ISSN 1530-9932. doi: 10.1208/s12249-024-02802-0. URL <https://link.springer.com/10.1208/s12249-024-02802-0>.
- [8] Gesine Cornelissen. *Integrierte Bioprozessentwicklung zur Herstellung pharmakologisch wirksamer Proteine mit Pichia pastoris*. PhD thesis, Hamburg, October 2004.
- [9] Davor Cotoras. Enhancing complex bioprocess learning through simulation technology and hybrid teaching: A case study in university education. *Biochemistry and Molecular Biology Education*, 52(5):519–526, September 2024. ISSN 1470-8175, 1539-3429. doi: 10.1002/bmb.21838. URL <https://iubmb.onlinelibrary.wiley.com/doi/10.1002/bmb.21838>.
- [10] J M Cregg, K J Barringer, A Y Hessler, and K R Madden. *Pichia pastoris* as a host system for transformations. *Molecular and Cellular Biology*, 5(12):3376–3385, December 1985. ISSN 0270-7306, 1098-5549. doi: 10.1128/MCB.5.12.3376. URL <http://mcb.asm.org/lookup/doi/10.1128/MCB.5.12.3376>.
- [11] James M. Cregg, Joan Lin Cereghino, Jianying Shi, and David R. Higgins. Recombinant protein expression in *Pichia pastoris*. *Molecular Biotechnology*, 16(1):23–52, September 2000. ISSN 1073-6085, 1559-0305. doi: 10.1385/MB:16:1:23. URL <https://link.springer.com/10.1385/MB:16:1:23>.
- [12] Azadeh Eskandari, Nima Ghahremani Nezhad, Thean Chor Leow, Mohd Basyaruddin Abdul Rahman, and Siti Nurbaya Oslan. Current achievements, strategies, obstacles, and overcoming the challenges of the protein engineering in *Pichia pastoris* expression system. *World Journal of Microbiology and Biotechnology*, 40(1):39, January 2024. ISSN 0959-3993, 1573-0972. doi: 10.1007/s11274-023-03851-6. URL <https://link.springer.com/10.1007/s11274-023-03851-6>.

- [13] Frank Geisler. *Datenbanken: Grundlagen und Design*. mitp, Heidelberg München Landsberg Frechen Hamburg, 5., aktualisierte und erweiterte auflage edition, 2014. ISBN 978-3-8266-9707-4 978-3-8266-8719-8.
- [14] Inga Gerlach, Volker C. Hass, Simone Brüning, and Carl-Fredrik Mandenius. Virtual bioreactor cultivation for operator training and simulation: application to ethanol and protein production. *Journal of Chemical Technology & Biotechnology*, 88(12):2159–2168, December 2013. ISSN 0268-2575, 1097-4660. doi: 10.1002/jctb.4079. URL <https://onlinelibrary.wiley.com/doi/10.1002/jctb.4079>.
- [15] Inga Gerlach, Volker Hass, and Carl-Fredrik Mandenius. Conceptual Design of an Operator Training Simulator for a Bio-Ethanol Plant. *Processes*, 3(3):664–683, September 2015. ISSN 2227-9717. doi: 10.3390/pr3030664. URL <https://www.mdpi.com/2227-9717/3/3/664>.
- [16] Inga Gerlach, Sören Tholin, Volker Hass, and Carl-Fredrik Mandenius. Operator Training Simulator for an Industrial Bioethanol Plant. *Processes*, 4(4):34, September 2016. ISSN 2227-9717. doi: 10.3390/pr4040034. URL <https://www.mdpi.com/2227-9717/4/4/34>.
- [17] Edward H. Glaessgen and Doane Stargel. The digital twin paradigm for future nasa and u.s. air force vehicles. 2012. URL <https://api.semanticscholar.org/CorpusID:110572580>.
- [18] Sartorius Stedim Biotech GmbH. Digitalize Your Bioprocess Control Training with BIOSTAT® T CHO, 2018. URL <https://www.sartorius.com/resource/blob/58178/b1c7589ae0828e89236f716d5c5a16e9/broch-biostat-t-cho-training-tool-sbi1573-e-data.pdf>.
- [19] M. E. Gregory and C. Turner. Open-loop control of specific growth rate in fed-batch cultures of recombinant E.coli. *Biotechnology Techniques*, 7(12):889–894, December 1993. ISSN 0951-208X, 1573-6784. doi: 10.1007/BF00156368. URL <http://link.springer.com/10.1007/BF00156368>.
- [20] Michael Grieves. Digital twin: Manufacturing excellence through virtual factory replication. 03 2015.
- [21] Christoph Herwig, Ralf Pörtner, and Johannes Möller, editors. *Digital Twins: Applications to the Design and Optimization of Bioprocesses*, volume 177

- of *Advances in Biochemical Engineering/Biotechnology*. Springer International Publishing, Cham, 2021. ISBN 978-3-030-71655-4 978-3-030-71656-1. doi: 10.1007/978-3-030-71656-1. URL <https://link.springer.com/10.1007/978-3-030-71656-1>.
- [22] Angela Faustino Jozala, Danilo Costa Geraldês, Louise Lacalendola Tundisi, Valter De Araújo Feitosa, Carlos Alexandre Breyer, Samuel Leite Cardoso, Priscila Gava Mazzola, Laura De Oliveira-Nascimento, Carlota De Oliveira Rangel-Yagui, Pérola De Oliveira Magalhães, Marcos Antonio De Oliveira, and Adalberto Pessoa. Biopharmaceuticals from microorganisms: from production to purification. *Brazilian Journal of Microbiology*, 47:51–63, December 2016. ISSN 15178382. doi: 10.1016/j.bjm.2016.10.007. URL <https://linkinghub.elsevier.com/retrieve/pii/S1517838216310413>.
- [23] Lena Sophia Kaletsch. Development of an interactive Escherichia coli fed-batch fermentation simulation. Master's thesis, March 2023.
- [24] Mohsen Karbalaei, Seyed A. Rezaee, and Hadi Farsiani. *Pichia pastoris* : A highly successful expression system for optimal synthesis of heterologous proteins. *Journal of Cellular Physiology*, 235(9):5867–5881, September 2020. ISSN 0021-9541, 1097-4652. doi: 10.1002/jcp.29583. URL <https://onlinelibrary.wiley.com/doi/10.1002/jcp.29583>.
- [25] Jae-Sug Ki. Interactive training simulator for aerial working platform in a virtual environment. *Computer Applications in Engineering Education*, 19(4):733–738, December 2011. ISSN 1061-3773, 1099-0542. doi: 10.1002/cae.20358. URL <https://onlinelibrary.wiley.com/doi/10.1002/cae.20358>.
- [26] Christina Kirschnick. Etablierung einer kultivierung von pichia pastoris in laborfermenter zur expression von egfp, 2019. URL <http://hdl.handle.net/20.500.12738/8634>.
- [27] Roger Kneebone. Simulation in surgical training: educational issues and practical implications. *Medical Education*, 37(3):267–277, March 2003. ISSN 0308-0110, 1365-2923. doi: 10.1046/j.1365-2923.2003.01440.x. URL <http://doi.wiley.com/10.1046/j.1365-2923.2003.01440.x>.
- [28] Mario Lederle, Mircea Tric, Tatjana Roth, Lina Schütte, Anke Rattenholl, Dirk Lütkemeyer, Stefan Wölfl, Tobias Werner, and Philipp Wiedemann. Continuous

- optical in-line glucose monitoring and control in CHO cultures contributes to enhanced metabolic efficiency while maintaining darbepoetin alfa product quality. *Biotechnology Journal*, 16(8):2100088, August 2021. ISSN 1860-6768, 1860-7314. doi: 10.1002/biot.202100088. URL <https://onlinelibrary.wiley.com/doi/10.1002/biot.202100088>.
- [29] Reiner Luttmann. *BIOSIM: Ein Online-Simulationssystem für Lehre und Forschung, Version 1*. Fachhochschule Hamburg, Hamburg, Deutschland, 1 edition, 2000. Manual.
- [30] Biswanath Mahanty. Hybrid modeling in bioprocess dynamics: Structural variabilities, implementation strategies, and practical challenges. *Biotechnology and Bioengineering*, 120(8):2072–2091, August 2023. ISSN 0006-3592, 1097-0290. doi: 10.1002/bit.28503. URL <https://onlinelibrary.wiley.com/doi/10.1002/bit.28503>.
- [31] MathWorks. Designing apps using app designer, 2023. <https://www.mathworks.com/products/matlab/app-designer.html>.
- [32] MathWorks. Building interactive applications with app designer, 2023. <https://www.mathworks.com/help/matlab/app-designer.html>.
- [33] MathWorks. App designer code generation, 2023. https://de.mathworks.com/help/matlab/creating_guis/app-designer-code-generation.html.
- [34] MathWorks. Get and manage add-ons, 2023. https://de.mathworks.com/help/matlab/matlab_env/get-add-ons.html.
- [35] MathWorks. Write callbacks for gui in app designer, 2023. https://de.mathworks.com/help/matlab/creating_guis/write-callbacks-for-gui-in-app-designer.html.
- [36] MathWorks. Standalone applications, 2023. <https://de.mathworks.com/help/compiler/standalone-applications.html>.
- [37] MathWorks. Ordinary differential equations (ode) solvers, 2024. <https://www.mathworks.com/help/matlab/math/choose-an-ode-solver.html>.

-
- [38] MathWorks. Database Application Deployment, n.d.. URL https://de.mathworks.com/help/database/database-application-deployment.html?s_tid=CRUX_lftnav. Accessed: 2024-11-18.
- [39] MathWorks. Database Toolbox Documentation, n.d.. URL https://de.mathworks.com/help/database/index.html?s_tid=CRUX_lftnav. Accessed: 2024-11-18.
- [40] Mohammad Javad Saeidi. Etablierung einer dreistufigen Kultivierung der Hefe *Pichia pastoris* in einem Laborfermenter, May 2017. URL https://reposit.haw-hamburg.de/bitstream/20.500.12738/8632/1/Saeidis_Bachelorarbeit.pdf.
- [41] SQLite. About SQLite, n.d.. URL <https://www.sqlite.org/about.html>. Accessed: 2024-11-18.
- [42] SQLite. Datatypes In SQLite, n.d.. URL <https://sqlite.org/datatype3.html>. Accessed: 2024-11-18.
- [43] Ulrich Stein. *Programmieren mit MATLAB: Programmiersprache, grafische Benutzeroberflächen, Anwendungen*. Hanser, München, 7., vollständig überarbeitete auflage edition, 2023. ISBN 978-3-446-47703-2.
- [44] René Steiner, editor. *Grundkurs Relationale Datenbanken: Einführung in die Praxis der Datenbankentwicklung für Ausbildung, Studium und IT-Beruf*. Springer-Link Bücher. Vieweg+Teubner, Wiesbaden, 7., überarbeitete und aktualisierte auflage edition, 2009. ISBN 978-3-8348-0710-6 978-3-8348-9565-3. doi: 10.1007/978-3-8348-9565-3.
- [45] Inc. The MathWorks. Matlab documentation: ode15s, 2024. URL <https://de.mathworks.com/help/matlab/ref/ode15s.html#bu8tot8>. Accessed: 2024-11-27.
- [46] Adam Thelen, Xiaoge Zhang, Olga Fink, Yan Lu, Sayan Ghosh, Byeng D. Youn, Michael D. Todd, Sankaran Mahadevan, Chao Hu, and Zhen Hu. A comprehensive review of digital twin—part 2: roles of uncertainty quantification and optimization, a battery digital twin, and perspectives. *Structural and Multidisciplinary Optimization*, 66(1):1, January 2023. ISSN 1615-147X, 1615-1488.

- doi: 10.1007/s00158-022-03410-x. URL <https://link.springer.com/10.1007/s00158-022-03410-x>.
- [47] Alexander Thoma. Untersuchung und Automatisierung der Expressionsphase einer dreistufigen Fed Batch Kultivierung von *Pichia pastoris* zur Herstellung von eGFP, February 2019. URL https://reposit.haw-hamburg.de/bitstream/20.500.12738/9077/1/Thoma_geschwaerzt.pdf.
- [48] Hansruedi Tremp. *Architekturen verteilter Softwaresysteme: SOA, EDA & Microservices Mehrschichtenarchitekturen Anwendungsintegration*. erfolgreich studieren. Springer Fachmedien Wiesbaden, Wiesbaden, 2024. ISBN 978-3-658-45191-2 978-3-658-45192-9. doi: 10.1007/978-3-658-45192-9. URL <https://link.springer.com/10.1007/978-3-658-45192-9>.
- [49] Moritz Von Stosch, Rui Oliveira, Joana Peres, and Sebastião Feyo De Azevedo. Hybrid semi-parametric modeling in process systems engineering: Past, present and future. *Computers & Chemical Engineering*, 60:86–101, January 2014. ISSN 00981354. doi: 10.1016/j.compchemeng.2013.08.008. URL <https://linkinghub.elsevier.com/retrieve/pii/S0098135413002639>.
- [50] D. Whitman. Process, People, Perfection: Learning from the Pioneers in Human Performance. *PDA Journal of Pharmaceutical Science and Technology*, 68(5): 384–385, September 2014. ISSN 1079-7440. doi: 10.5731/pdajpst.2014.01008. URL <http://journal.pda.org/cgi/doi/10.5731/pdajpst.2014.01008>.
- [51] B. Xu, M. Jahic, and S.-O. Enfors. Modeling of Overflow Metabolism in Batch and Fed-Batch Cultures of *Escherichia coli*. *Biotechnology Progress*, 15(1):81–90, February 1999. ISSN 8756-7938. doi: 10.1021/bp9801087. URL <http://doi.wiley.com/10.1021/bp9801087>.

Eidesstattliche Erklärung

Hiermit versichere ich, Philipp Yuma Iff, die vorliegende Abschlussarbeit selbstständig und nur unter Verwendung der von mir angegebenen Quellen und Hilfsmittel verfasst zu haben. Sowohl inhaltlich als auch wörtlich entnommene Inhalte wurden als solche kenntlich gemacht. Die Arbeit hat in dieser oder vergleichbarer Form noch keinem anderem Prüfungsgremium vorgelegen.

Hamburg, den 29.11.2024

6 Appendix

6.1 MATLAB® Code

6.1.1 MATLAB Model Implementation

Listing 6.1: MATLAB code of AOX induction

```
1 % AOX induction for methanol metabolism
2 bS2script = app.p.aS2script + my2max;
3 bS2trans = app.p.aS2trans + my2max;
4 bS2act = app.p.aS2act + my2max;
5
6 % Turn on/turn Off functions
7 VS2ind = ((app.v.cS2L(end)/app.p.kS2ind)^app.p.kappaind)/(1+(app.v.cS2L(end)/app.p.kS2ind)
    ^app.p.kappaind);
8 VS1rep = 1/(1+(app.v.cS1L(end)/app.p.kS1rep)^app.p.kapparep);
9
10 app.v.qS2pXind(end+1) = VS1rep*VS2ind*app.p.qS2pXsup;
11
12 ODE_Induction = @(t,y) Pichia_Induction_Cornelissen(app.v.qXpX(end), app.v.qS2pXind(end),
    app.p.aS2script, app.p.aS2trans, app.p.aS2act, bS2script, bS2trans, bS2act, t, y);
13
14 [~,y] = ode45(ODE_Induction, [0 app.deltat], [app.v.qS2pXscript(end), app.v.qS2pXtrans(end),
    app.v.qS2pXact(end)]);
15
16 app.v.qS2pXscript(end+1) = y(end,1);
17 app.v.qS2pXtrans(end+1) = y(end,2);
18 app.v.qS2pXact(end+1) = y(end,3);
19
20 % Glycerol dependend methanol uptake rate [1/h]
21 app.v.qS2pX = app.v.qS2pXact(end)*(app.v.cS2L(end)/(app.p.kS2+app.v.cS2L(end)))*(app.p.
    kI22/(app.p.kI22+app.v.cS2L(end)))*(app.p.kI21/(app.p.kI21+app.v.cS1L(end)));
```

Listing 6.2: MATLAB code of AOX induction ODE function

```
1 %% Target protein expression
2 bP1script = app.p.aP1script + my2max;
3 bP1trans = app.p.aP1trans + my2max;
4 bP1act = app.p.kP1alpha + my2max;
5
6 % Turn on/turn Off functions
7 VS2ind = ((app.v.cS2L(end)/app.p.kS2ind)^app.p.kappaind)/(1+(app.v.cS2L(end)/app.p.kS2ind)
    ^app.p.kappaind);
8 VS1rep = 1/(1+(app.v.cS1L(end)/app.p.kS1rep)^app.p.kapparep);
9
```

```

10 app.v.qP1pXind(end+1) = VS1rep*VS2ind*app.p.qP1pXsup;
11
12 delta_qP1pXscript = app.v.qP1pXscriptw - app.v.qP1pXscript; % Calculate control deviation
13 if length(app.t) > 1 && length(delta_qP1pXscript) > 1 % need at least two values for
    trapz()
14     app.v.qP1pXscriptw(end+1) = VS1rep*VS2ind*app.p.qP1pXmax; % Calculate scriptw for
        next cycle
15     app.v.qP1pXback(end+1) = app.p.KCP1script*(delta_qP1pXscript(end)+trapz(app.t,
        delta_qP1pXscript)/app.p.TIP1script);
16 else
17     app.v.qP1pXscriptw(end+1) = 0;
18     app.v.qP1pXback(end+1) = 0;
19 end
20 ODE_Expression = @(t,y) Pichia_Expression_Cornelissen(app.v.qXpX(end),app.v.qP1pXind(end)
    ,app.v.qP1pXback(end),app.p.aP1script,app.p.aP1trans,app.p.kP1alpha,bP1script,
    bP1trans,bP1act,t,y);
21
22 % IVP
23 [~,y] = ode45(ODE_Expression, [0 app.deltat], [app.v.qP1pXscript(end),app.v.qP1pXtrans(
    end),app.v.qP1pXact(end)]);
24
25 app.v.qP1pXscript(end+1) = y(end, 1);
26 app.v.qP1pXtrans(end+1) = y(end, 2);
27 app.v.qP1pXact(end+1) = y(end, 3);
28
29 % Glycerol and methanol dependend product formation rate [1/h]
30 app.v.qP1pX(end+1) = app.v.qP1pXact(end);

```

Listing 6.3: MATLAB code of protein expression

```

1 function [dy] = Pichia_Expression_Cornelissen(qXpX,qP1pXind,qP1pXback,aP1script,aP1trans,
    kP1alpha,bP1script,bP1trans,bP1act,t,y)
2     % y0 = [qP1pXscript = 0, qP1pXtrans = 0, qP1pXact = 0]
3     % Transcription
4     dy(1) = -(aP1script + qXpX) * y(1) + bP1script * (qP1pXind + qP1pXback);
5     % Translation
6     dy(2) = -(aP1trans + qXpX) * y(2) + bP1trans * y(1);
7     % Activity
8     dy(3) = -(kP1alpha + qXpX) * y(3) + bP1act * y(2);
9
10    dy = dy';
11 end

```

Listing 6.4: MATLAB code of protein expression ODE function

```

1 function [dy] = Pichia_Induction_Cornelissen(qXpX,qS2pXind,aS2script,aS2trans,aS2act,
    bS2script,bS2trans,bS2act,t,y)
2 % y0 = [qS2pXscript = 0, qS2pXtrans = 0, qS2pXact = 0]
3 % Transcription
4 dy(1) = -(aS2script+qXpX)*y(1)+bS2script*qS2pXind;
5 % Translation
6 dy(2) = -(aS2trans+qXpX)*y(2)+bS2trans*y(1);
7 % Activity

```

```
8 dy(3) = -(aS2act+qXpX)*y(3)+bS2act*y(2);  
9 dy = dy';  
10 end
```

6.2 Dropbox

Since the simulation platform is made up of many classes, they cannot all be listed in this text. For this reason, all MATLAB® files were uploaded to a dropbox. With the submission of this work, not all bugs could be removed from the program code due to the limited time available. For this reason, no compiled application is available at this time. Any error in the runtime would cause the entire application to crash. Following this work, the compiled app will also be made available in the Dropbox. Until then, the biofermentation simulation software must be started via the “StartingScreen”. The functions do not change. However, a licensed MATLAB® installation is required in this case.

The Dropbox can be accessed via the following link:

https://www.dropbox.com/scl/fo/gk24excsno3cdnc128guf/ANHaf0CF_DjL0nQVCeV2098?rlkey=vi3nl48lkmc4fyh0rctwej8a8st=g26gvmctdl=0



Figure 6.1: Dropbox QR code

6.3 Database Schemes

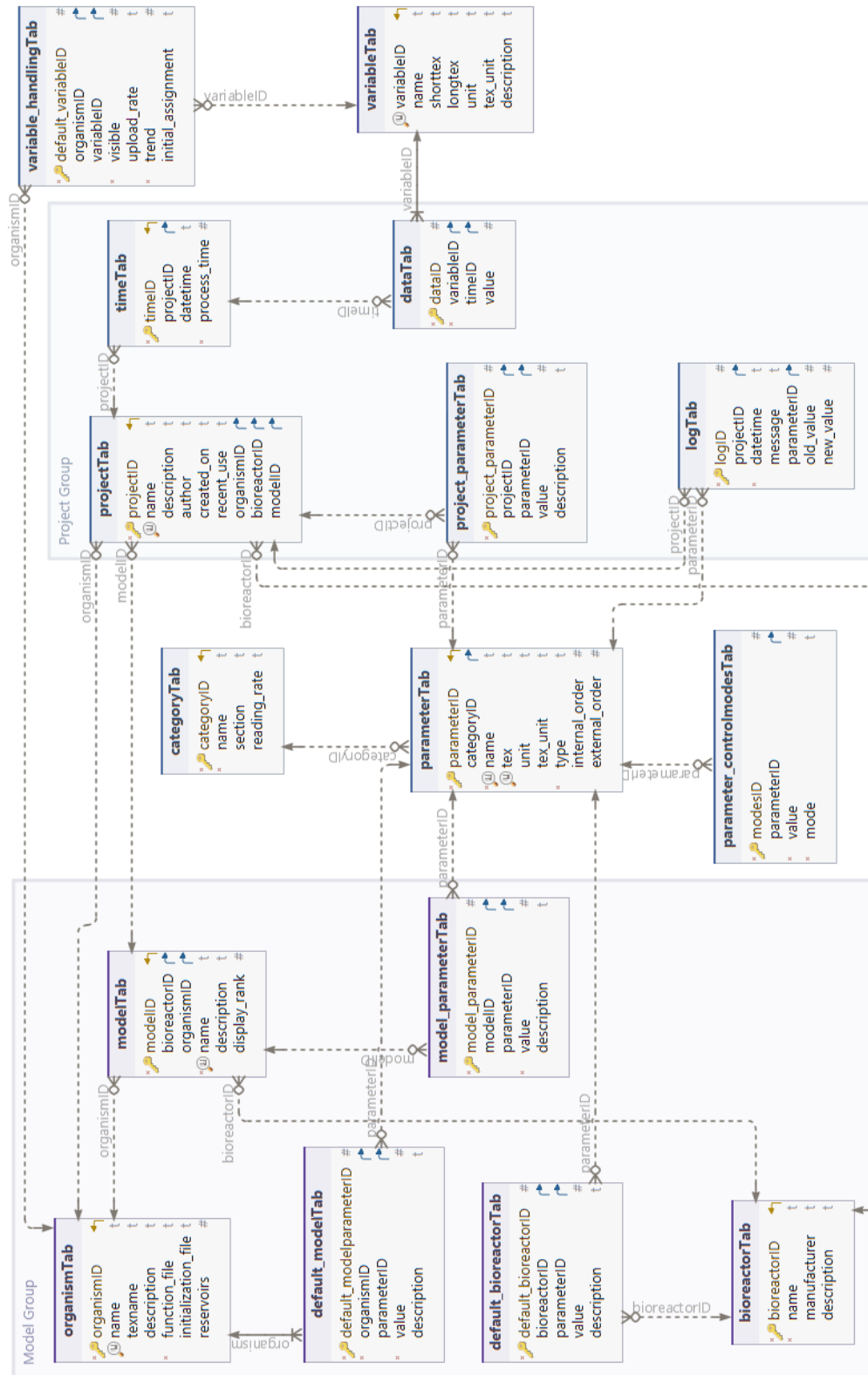


Figure 6.2: ER diagram of simulation related tables
Created with DbSchema

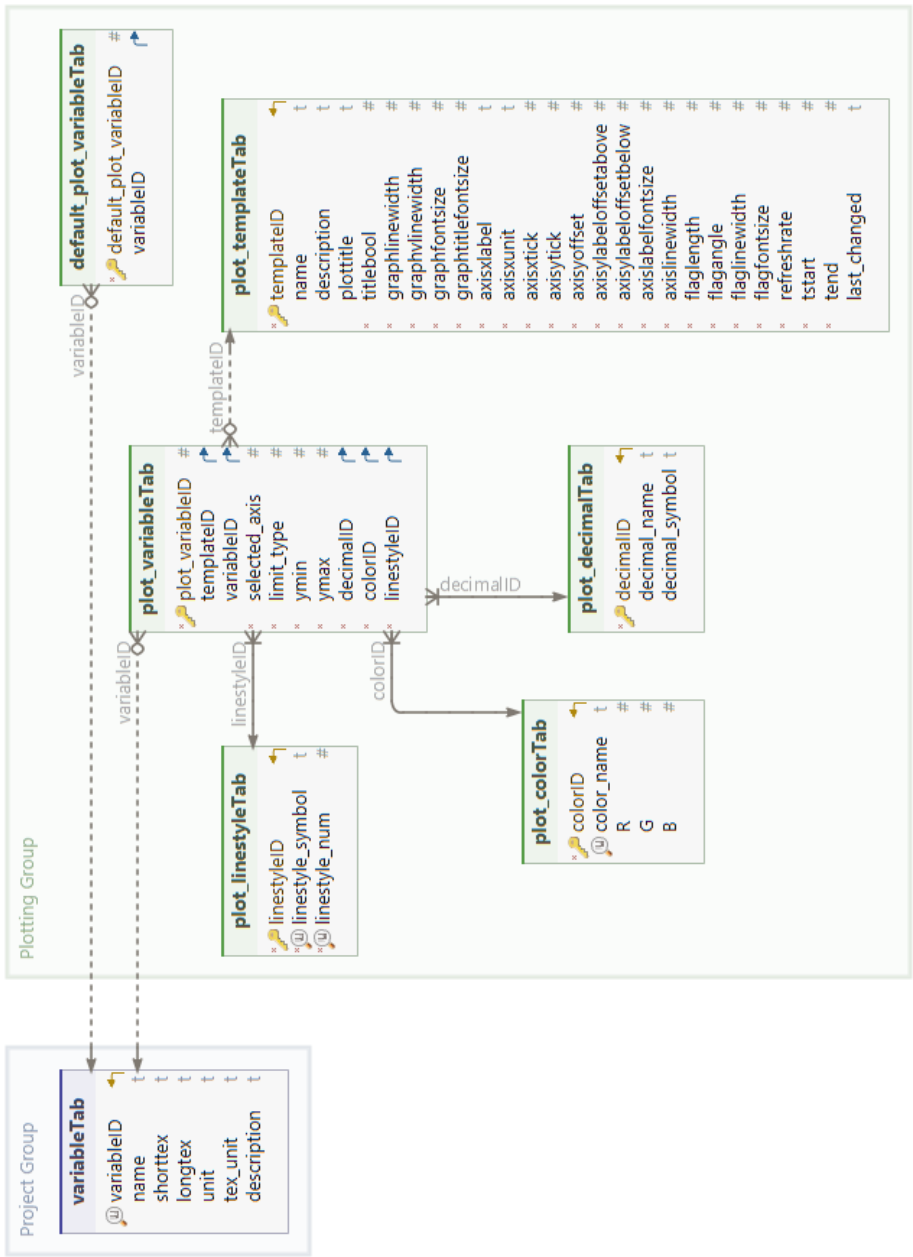


Figure 6.4: ER diagram of plotting related tables
Created with DbSchema

6.4 Simulation Process Information

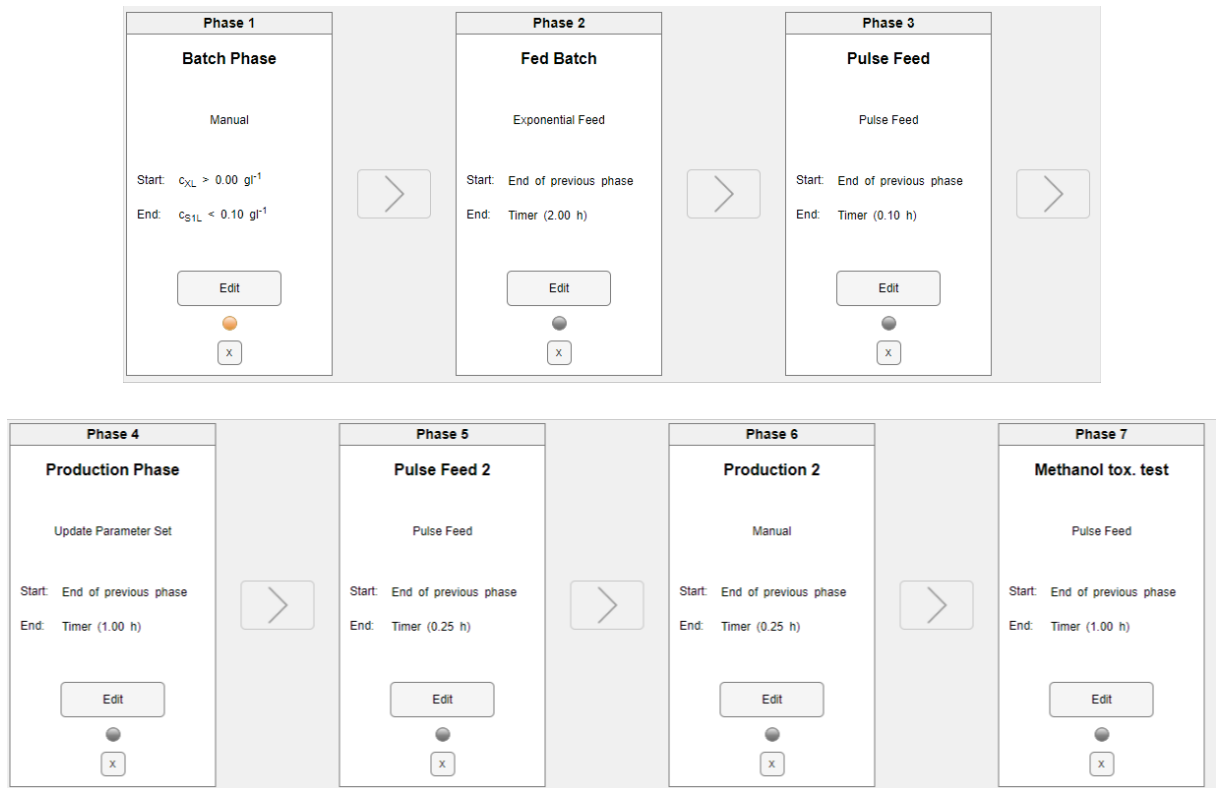


Figure 6.5: Process panels of discussed simulation

The panels show process phases planned within the Process Manger in the Control App.

Listing 6.5: Logger of simulated process

```

1 [21.11.2024 08:26:49.639, t = 0.000 h] Event: Process started
2 [21.11.2024 08:26:53.000, t = 0.002 h] Parameter update: f_Inoc was switched on
3 [21.11.2024 08:26:53.721, t = 0.002 h] Event: Inoculation occurred
4 [21.11.2024 08:26:55.666, t = 0.002 h] Event: Batch Phase [Phase 1] started at t = 0.002 h.
5 [21.11.2024 08:38:19.000, t = 2.400 h] Parameter update: FnAIRw was changed from 7.5 to 10 l/min
6 [21.11.2024 08:43:07.000, t = 3.382 h] Parameter update: FnAIRw was changed from 10 to 12.5 l/min
7 [21.11.2024 08:50:13.020, t = 3.500 h] Event: Batch Phase [Phase 1] just ended at t = 3.500 h.
8 [21.11.2024 08:50:14.931, t = 3.500 h] Event: Fed Batch [Phase 2] started at t = 3.500 h.
9 [21.11.2024 08:50:15.037, t = 3.500 h] Event:
10 =====
11 Open loop feed control parameters:
12 qX/X1w: 0.100 l/h
13 qS1/Xm: 0.027 l/h
14 yX/S1gr: 0.670
15 cS1R1: 150.0 g/l
16 cXLj: 8.92 g/l
17 with:
18 tij: 3.500 h
19 FR1j: 0.054 l/h
20 =====
21 [21.11.2024 09:17:54.000, t = 4.358 h] Parameter update: FnAIRw was changed from 12.5 to 15 l/min
22 [21.11.2024 09:25:13.419, t = 5.500 h] Event: Fed Batch [Phase 2] just ended at t = 5.500 h.
23 [21.11.2024 09:25:14.200, t = 5.500 h] Event: Pulse Feed [Phase 3] started at t = 5.500 h.
24 [21.11.2024 09:25:14.000, t = 5.500 h] Parameter update: kR2 was changed from 0.5 to 1
25 [21.11.2024 09:25:14.000, t = 5.500 h] Parameter update: R_feed was changed from 1 to 2
26 [21.11.2024 09:25:14.287, t = 5.500 h] Event: Pulse feed with FR2 = 0.250 l/h

```

```

27 [21.11.2024 09:27:50.816, t = 5.600 h] Event: Pulse Feed [Phase 3] just ended at t = 5.600 h.
28 [21.11.2024 09:27:52.723, t = 5.600 h] Event: Production Phase [Phase 4] started at t = 5.600 h.
29 [21.11.2024 09:27:52.000, t = 5.600 h] Parameter update: cS2Lw was changed from 2 to 2 g/l
30 [21.11.2024 09:27:52.000, t = 5.600 h] Parameter update: R_feed was changed from 2 to 2
31 [21.11.2024 09:27:52.000, t = 5.600 h] Parameter update: Mode_feed was switched on
32 [21.11.2024 09:39:52.880, t = 6.600 h] Event: Production Phase [Phase 4] just ended at t = 6.600 h.
33 [21.11.2024 09:39:54.250, t = 6.600 h] Event: Pulse Feed 2 [Phase 5] started at t = 6.600 h.
34 [21.11.2024 09:39:54.000, t = 6.600 h] Parameter update: R_feed was changed from 2 to 1
35 [21.11.2024 09:39:54.335, t = 6.600 h] Event: Pulse feed with FR1 = 0.250 l/h
36 [21.11.2024 09:42:53.727, t = 6.850 h] Event: Pulse Feed 2 [Phase 5] just ended at t = 6.850 h.
37 [21.11.2024 09:42:55.139, t = 6.850 h] Event: Production 2 [Phase 6] started at t = 6.850 h.
38 [21.11.2024 09:45:54.930, t = 7.100 h] Event: Production 2 [Phase 6] just ended at t = 7.100 h.
39 [21.11.2024 09:45:56.695, t = 7.100 h] Event: Methanol tox. test [Phase 7] started at t = 7.100 h.
40 [21.11.2024 09:45:56.000, t = 7.100 h] Parameter update: kR2 was changed from 1 to 1
41 [21.11.2024 09:45:56.000, t = 7.100 h] Parameter update: R_feed was changed from 1 to 2
42 [21.11.2024 09:45:56.791, t = 7.100 h] Event: Pulse feed with FR2 = 0.500 l/h
43 [21.11.2024 10:02:38.619, t = 8.100 h] Event: Methanol tox. test [Phase 7] just ended at t = 8.100 h.

```

6.5 *P. pastoris* Model Script

Listing 6.6: Model script of *P. pastoris*

```

1 function app_new = Pichia_pastoris(app)
2
3 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4 %% Feeding
5
6 % Declare feeding rate with zero and adjust the value afterwards
7 app.v.FR1(end+1) = 0;
8 app.v.FR2(end+1) = 0;
9
10 % Feeding is predefined as zero but if one of the following condition is
11 % true, feeding will take place. The hierachy is important to only allow
12 % one type of feeding
13 if app.switch_exp
14     %% Exponential feed phase
15     k = num2str(app.p.R_feed);
16     % Calculate feeding rate FR(t)
17     FRwj = app.p(['FR' k 'j']);
18     qxpxwj = app.p(['qXpX' k 'w']);
19     tj = app.p(['t' k 'j']);
20     FR = FRwj*exp(qxpxwj*(app.t(end)-tj));
21     if FR < app.p(['FR' k 'max'])
22         app.v(['FR' k])(end) = FR;
23     else
24         msg = sprintf("The desired pumping capacity for FR%i is greater than the
25             maximum pumping capacity.",k);
26         fprintf(append(msg,"\n"));
27         logMessage(app.Startingscreen, msg)
28         app.switch_exp = false;
29     end
30 elseif app.switch_pulse

```

```

31     %% Pulse feed phase
32
33     k = num2str(app.p.R_feed);
34     app.v.(['FR' k])(end) = app.p.(['FR' k 'max']) * app.p.(['kR' k]);
35
36 elseif app.p.Mode_p02 == 4
37     %% p02 feed
38     % Calculate deviation from setpoint
39     cEfeedp02 = app.p.p02w - app.v.p02(end); % Error/Controller difference
        between measured p02 and Setpoint e = (w-x)
40     app.a.cE_feedp02 = (cEfeedp02 + 0.001/app.a.deltat*app.a.cE_feedp02)/(0.001/
        app.a.deltat+1); % Filtered error through a PT1 delay with T = 0.001 and
        deltat
41     app.a.ce_feedp02(end+1) = app.a.cE_feedp02/(100-0); % Normalized controller
        difference e = (w-x)/(w_max - w_min); [-1,+1]
42
43     % Calculate controller outputs
44     app.a.cP_feedp02 = app.a.ce_feedp02(end)*app.p.KP_feedp02; % P-part of controller
        with KP = -2.0
45     app.a.cI_feedp02 = app.a.cI_feedp02+(app.a.ce_feedp02(end)+app.a.ce_feedp02(end
        -1))/2*app.a.deltat*app.p.KI_feedp02; % I-part of controller with KI = -15.0
        and deltat
46     app.a.cD_feedp02 = (app.a.ce_feedp02(end)-app.a.ce_feedp02(end-1))/app.a.deltat*
        app.p.KD_feedp02; % D-part of controller with KD = -0.009
47
48     % Calculation of new relative setpoint for feed pump
49     % PID sum in percent
50     app.a.yfeedp02 = ((app.a.cP_feedp02+app.a.cI_feedp02+app.a.cD_feedp02)*100+app.a
        .yfeedp02)/2;
51
52     % Define limits for new setpoint
53     if app.a.yfeedp02 < 0
54         app.a.yfeedp02 = 0;
55     elseif app.a.yfeedp02 > 100
56         app.a.yfeedp02 = 100;
57     end
58     % Maybe add a parameter for reservoir selection later
59     % Here reservoir 1 is standard
60     % Calculation of new agitation speed setpoint [1/h]
61     app.v.FR1(end+1) = app.a.yfeedp02/100*app.p.FR1max;
62
63 elseif app.p.f_feed == 1 % feed control as second priority
64     % Current reservoir
65     n = num2str(app.p.R_feed);
66     %% Constant feeding
67     if app.p.Mode_feed == 0
68         % Set pump rate in relation to FRmax
69         app.v.(['FR' n])(end) = app.p.(['FR' n 'w']);
70     elseif app.p.Mode_feed == 1
71         %% Closed-loop feed control
72         cSLw = app.p.(['cS' n 'Lw']);
73         cSL = app.v.(['cS' n 'L'])(end);
74         cSLwmax = app.p.(['cS' n 'R' n]) * 0.2; % Limit max concentration

```

```

75     cSLwmin = 0.01; % Set low minimum concentration
76     % Calculate deviation from setpoint
77     cEfeedR      = cSLw - cSL; % Error/Controller difference between
        measured pO2 and Setpoint e = (w-x)
78     T            = 0.001; % Time delay in h
79     app.a.(['cE_feedR', n]) = (cEfeedR+T/app.a.deltat*app.a.(['cE_feedR', n]))/(T/
        app.a.deltat+1); % Error filtered through a time delay of first order
        with delta t and T = 0.001
80     app.a.(['ce_feedR', n])(end+1) = app.a.(['cE_feedR', n])/(cSLwmax-cSLwmin); %
        Normalized controller difference e = (w-x)/(w_max-w_min); [-1,+1], 75% of
        substrate concentration in reservoir as max, 0.01 g/l min
81
82     % Calculate controller gains
83     app.a.(['cP_feedR', n]) = app.a.(['ce_feedR', n])(end)*app.p.(['KP_feedR',
        n]); % P-part of controller with KP = 10.0
84     app.a.(['cI_feedR', n]) = app.a.(['cI_feedR', n]) + (app.a.(['ce_feedR',
        n])(end) + app.a.(['ce_feedR', n])(end-1)) / 2 * app.a.deltat * app.p.(['
        KI_feedR', n]); % I-part of controller with KI = 1000 and with time
        increment deltat
85     app.a.(['cD_feedR', n]) = (app.a.(['ce_feedR', n])(end)-app.a.(['
        ce_feedR', n])(end-1))/app.a.deltat*app.p.(['KD_feedR', n]); % D-part of
        controller with KD = 0.015
86
87     % Calculation of new relative setpoint for rump rate
88     yFR = app.a.(['cP_feedR', n])+app.a.(['cI_feedR', n])+app.a.(['cD_feedR', n]); %
        PID sum
89
90     % Define limits for new setpoint
91     if yFR < 0
92         yFR = 0;
93     elseif yFR > 1
94         yFR = 1;
95     end
96     % Set pump rate in relation to FRmax
97     app.v.(['FR', n])(end) = app.p.(['FR', n 'max']) * yFR;
98 end
99 end
100
101 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
102 %% pO2 control
103 % Check if aeration with pure nitrogen or pure carbon dioxide
104 % is turned on
105 if app.p.f_aeration == 1
106     if app.p.f_N2 == 1
107         app.v.FnN2(end+1) = app.p.FnN2w;
108     else
109         app.v.FnN2(end+1) = 0;
110     end
111
112     if app.p.f_CO2 == 1
113         app.v.FnCO2(end+1) = app.p.FnCO2w;
114     else
115         app.v.FnCO2(end+1) = 0;

```

```

116     end
117 end
118 % Calculation of quasi stationary SPC-actual values
119 switch app.p.Mode_p02
120     case 1 % p02 agitation
121         % Calculate deviation from setpoint
122         cEagi = app.p.p02w-app.v.p02(end); % Error/Controller difference
            between measured p02 and Setpoint e = (w-x)
123         T = 0.001; % Time delay in h
124         app.a.cE_agi = (cEagi+T/app.a.deltat*app.a.cE_agi)/(T/app.a.deltat+1);
            % Error filtered through a time delay of first order with delta t and T
            = 0.001
125         app.a.ce_agi(end+1) = app.a.cE_agi/(100-1); % Normalized controller
            difference e = (w-x)/(w_max-w_min); [-1,+1]
126
127         % Calculate controller gains
128         app.a.cP_agi = app.a.ce_agi(end)*app.p.KP_agi; % P-part of controller
            with KP = 10.0
129         app.a.cI_agi = app.a.cI_agi + (app.a.ce_agi(end) + app.a.ce_agi(end-1)
            ) / 2 * app.a.deltat * app.p.KI_agi; % I-part of controller with KI =
            1000 and with time increment deltat
130         app.a.cD_agi = (app.a.ce_agi(end)-app.a.ce_agi(end-1))/app.a.deltat*
            app.p.KD_agi; % D-part of controller with KD = 0.015 and with time
            increment deltat PARAMETER ANGEPASST VON 0.25, DA SONST SCHWINGUNG ZU
            GRO
131
132         % Calculation of new relative setpoint for agitation
133         % speed
134         yNSt = app.a.cP_agi+app.a.cI_agi+app.a.cD_agi; % PID sum
135
136         % Define limits for new setpoint
137         if yNSt < 0.3
138             yNSt = 0.3;
139         elseif yNSt > 1
140             yNSt = 1;
141         end
142
143         % Calculation of new agitation speed setpoint [1/min]
144         app.v.NSt(end+1) = yNSt*app.p.NStmax;
145
146     case 2 % p02 aeration
147
148         % Calculate deviation from setpoint
149         cEaeration = app.p.p02w-app.v.p02(end); % Error/Controller
            difference between measured p02 and Setpoint e = (w-x)
150         app.a.cE_aeration = (cEaeration+0.001/app.a.deltat*app.a.cE_aeration)
            /(0.001/app.a.deltat+1); % Filtered error through a PT1 delay with T =
            0.001 and deltat
151         app.a.ce_aeration(end+1) = app.a.cE_aeration(end)/(100-1); % Normalized
            controller difference e = (w-x)/(w_max - w_min); [-1,+1]
152
153         % Calculate controller gains

```

```

154     app.a.cP_aeration = app.a.ce_aeration(end)*app.p.KP_aeration; % P-part of
        controller with KP = 20.0
155     app.a.cI_aeration = app.a.cI_aeration+(app.a.ce_aeration(end)+app.a.
        ce_aeration(end-1))/2*app.a.deltat*app.p.KI_aeration; % I-part of
        controller with KI = 0.003 with time increment 0.005 h
156     app.a.cD_aeration = (app.a.ce_aeration(end)-app.a.ce_aeration(end-1))/app.a.
        deltat*app.p.KD_aeration; % D-part of controller with KD = 0.0188
157
158     % Calculation of new setpoint for aeration rate (add O2
159     % aeration if AIR aeration is no longer sufficient)
160     yaeration = (app.a.cP_aeration+app.a.cI_aeration+app.a.cD_aeration)*100; %
        PID sum in percent
161     diff = 0;
162
163     % Define limits for new setpoint
164     if yaeration < 30
165         yaeration = 30;
166     elseif yaeration > 100
167         diff = yaeration-100;
168         if diff > 100
169             diff = 100;
170         end
171         yaeration = 100;
172     end
173
174     % Calculation of setpoints for AIR and O2 aeration [l/h]
175     app.v.FnAIR(end+1) = yaeration/100*app.p.FnAIRmax;
176     app.v.FnO2(end+1) = diff/100*app.p.FnO2max;
177
178     % Calculate overall aeration rate [l/h]
179     app.v.FnG(end+1) = app.v.FnAIR(end)+app.v.FnO2(end)+app.v.FnN2(end)+app.v.
        FnCO2(end);
180
181     case 3 % pO2 gasmix
182
183         % Calculate deviation from setpoint
184         cEgasmix = app.p.pO2w-app.v.pO2(end); % Error/Controller
        difference between measured pO2 and Setpoint e = (w-x)
185         app.a.cE_gasmix = (cEgasmix+0.001/app.a.deltat*app.a.cE_gasmix)
        /(0.001/app.a.deltat+1); % Filtered error through a PT1 delay with T =
        0.001 and deltat
186         app.a.ce_gasmix(end+1) = app.a.cE_gasmix(end)/(1-app.p.xOAIR); % Normalized
        controller difference e = (w-x)/(w_max - w_min); [-1,+1]
187
188         app.a.cP_gasmix = app.a.ce_gasmix(end)*app.p.KP_gasmix; % P-part of
        controller with KP = 0.4
189         app.a.cI_gasmix(end+1) = app.a.cI_gasmix(end)+(app.a.ce_gasmix(end)+app.a.
        ce_gasmix(end-1))/2*app.a.deltat*app.p.KI_gasmix; % I-part of controller
        with KI = 0.003 with time increment 0.005 h
190         app.a.cD_gasmix = (app.a.ce_gasmix(end)-app.a.ce_gasmix(end-1))/app.a.
        deltat*app.p.KD_gasmix; % D-part of controller with KD = 0.0005
191
192         % Calculation of new relative setpoint for xO2in

```

```

193         ygasmix = (app.a.cP_gasmix+app.a.cI_gasmix(end)+app.a.cD_gasmix(end))*100; %
           PID sum in percent
194
195         % Define limits for new setpoint
196         if ygasmix < app.p.xOAIR*100
197             ygasmix = app.p.xOAIR*100;
198         elseif ygasmix > 100
199             ygasmix = 100;
200         end
201
202         % Calculation of setpoint fir xOGin [-]
203         xOGinw = ygasmix/100*1;
204
205         % Calculation of AIR and O2 aeration rates [l/min]
206         app.v.FnAIR(end+1) = (app.p.FnGw*(xOGinw-1))/(app.p.xOAIR-1);
207         app.v.FnO2(end+1) = app.p.FnGw-app.v.FnAIR(end);
208
209         % Calculation of overall aeration rate [l/min]
210         app.v.FnG(end+1) = app.v.FnAIR(end)+app.v.FnO2(end)+app.v.FnN2(end)+app.v.
           FnCO2(end);
211
212     end
213     % pO2 feed control is handled in the feeding section and therefore cannot run in
           parallel with the exp feed or pulse feed
214
215     % Set stirrer speed if it is not pO2-controlled [1/min]
216     if app.p.Mode_pO2 ~= 1
217         if app.p.f_motor == 1
218             app.v.NSt(end+1) = app.p.NStw;
219         else
220             app.v.NSt(end+1) = 0;
221         end
222     end
223
224     %% Aeration rate
225
226     % Calculate aeration rate [l/min]
227     if app.p.Mode_pO2 ~= 2 && app.p.Mode_pO2 ~= 3
228         if app.p.f_aeration == 1
229             if app.p.f_air == 1
230                 app.v.FnAIR(end+1) = app.p.FnAIRw;
231             else
232                 app.v.FnAIR(end+1) = 0;
233             end
234
235             if app.p.f_O2 == 1
236                 app.v.FnO2(end+1) = app.p.FnO2w;
237             else
238                 app.v.FnAIR(end+1) = 0;
239             end
240         end
241
242         % Total unfiltered aeration rate

```



```

242         app.v.FnG(end+1) = app.v.FnAIR(end)+app.v.FnO2(end)+app.v.FnN2(end)+app.v.
           FnCO2(end);
243     else
244         app.v.FnG(end+1) = 0;
245     end
246 end
247
248 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
249 %% Liquid Weight Control
250 if app.p.Mode_harvest == 1
251
252     % Calculate difference to setpoint
253     cELW = app.v.VL(end)*app.p.rhoL-app.p.LWw; % Error/Controller
           difference between measured liquid weight and Setpoint e = (w-x)
254     app.a.cE_LW = (cELW+0.001/app.a.deltat*app.a.cE_LW)/(0.001/app.a.deltat+1)
           ; % Error filtered through a time delay of first order with deltat and T =
           0.001 h
255     app.a.ce_LW(end+1) = app.a.cE_LW/(app.p.VLmax*app.p.rhoL-app.p.VLmin*app.p.rhoL);
           % Normalized controller difference e = (w-x)/(w_max - w_min); [-1,+1]
256
257     % Calculate controller gains
258     cP_LW = app.a.ce_LW(end)*app.p.KP_LW; % P-part of controller with KP =
           10.0
259     app.a.cI_LW(end+1) = app.a.cI_LW(end)+(app.a.ce_LW(end)+app.a.ce_LW(end-1))/2*app
           .a.deltat*app.p.KI_LW; % I-part of controller with KI = 0.3 and with time
           increment deltat
260     cD_LW = (app.a.ce_LW(end)-app.a.ce_LW(end-1))/app.a.deltat*app.p.KD_LW
           ; % D-part of controller with KD = 1.0
261
262     % Calculation of new relative setpoint for harvest pump
263     yLW = (cP_LW+app.a.cI_LW(end)+cD_LW)*100; % PID sum in percent
264
265     % Define limits for new setpoint
266     if yLW < 0
267         yLW = 0;
268     elseif yLW > 100
269         yLW = 100;
270     end
271
272     % Calculation of new setpoint for harvest pump [l/h]
273     app.v.FH(end+1) = yLW/100*app.p.FHmax;
274 else
275     % Set harvest rate with harvest pump ON/OFF [l/h]
276     if app.p.f_harvest == 1
277         app.v.FH(end+1) = app.p.FHrelw/100*app.p.FHmax;
278     else
279         app.v.FH(end+1) = 0;
280     end
281 end
282
283 % Calculate molefraction at reactor inlet [-]
284 if app.v.FnG(end) > 0

```

```

285     app.v.xOGin(end+1) = (app.p.xOAIR*app.v.FnAIR(end)+app.v.FnO2(end))/app.v.FnG(end
286     );
287     %app.v.xCGin(end+1) = (app.p.xCAIR*app.v.FnAIR(end))/app.v.FnG(end); % CO2 term
288     % was deleted as no aeration with CO2 will take place in this simulation
289
290 else
291     app.v.xOGin(end+1) = 0;
292     %app.v.xCGin(end+1) = 0;
293 end
294
295 % Set tONi to time of inoculation, if it has already taken
296 % place [h]
297 if app.a.ToI ~= 0
298     tONi = app.a.ToI;
299 end
300
301 % Calculate anti foam addition activity
302 if app.p.f_antifoam == 1 && app.a.ToI ~= 0
303     TON = app.t(end)-tONi;
304     AAFin = app.p.AAFast*app.p.VAF^(TON/app.p.Ttast);
305 else
306     tONi = app.t(end);
307     AAFin = 0;
308     TON = 0;
309 end
310
311 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
312 %% pH Control (Manual)
313 if app.p.Mode_pH == 0
314     % Calculation of titration rate with alkali pump ON/OFF [l/h]
315     if app.p.f_alkali == 1
316         app.v.FT2(end+1) = app.p.FT2max;
317     else
318         app.v.FT2(end+1) = 0;
319     end
320
321     % Calculation of titration rate with acid pump ON/OFF [l/h]
322     if app.p.f_acid == 1
323         app.v.FT1(end+1) = app.p.FT1max;
324     else
325         app.v.FT1(end+1) = 0;
326     end
327 end
328
329 %% pH Control (Auto)
330 if app.p.Mode_pH == 1
331     % Calculation of pH difference to Setpoint and
332     % corresponding controller output as setpoint for
333     % acid/alkali pumps (Cascade Control)
334     if abs(app.p.pHw-app.v.pHL(end)) < 0.1
335         app.v.FT1(end+1) = 0;
336         app.v.FT2(end+1) = 0;
337     else
338         % Calculate difference to setpoint

```

```

336     epH      = app.p.pHw-app.v.pHL(end); % Error/Controller difference between
           pH in Liquid and Setpoint e = (w-x)
337     app.a.cEpH = (epH+0.001/app.a.deltat*app.a.cEpH)/(0.001/app.a.deltat+1); %
           Filtered error through a PT1 delay with T = 0.001 h and deltat
338     cepH      = app.a.cEpH/(app.p.pHLmaxgr-app.p.pHLmingr); % Normalized
           controller difference e = (w-x) / (w_max - w_min) ; [-1,+1]
339
340     % Calculate controller gains
341     cP_pH      = cepH*app.p.KP_pH; % P-part of controller with KP = 10000
342
343     % Calculation of new relative difference setpoint
344     % by master controller
345     % in percent
346     ypH        = cP_pH*100;
347     if ypH > 100
348         ypH = 100;
349     elseif ypH < -100
350         ypH = -100;
351     end
352
353     % Calculation of deviation from difference setpoint
354     cEypH       = app.p.ypH_SET-(ypH/100); % Error/Controller difference between
           measured pH difference to setpoint and wanted difference
355     ceypH       = cEypH/(1-(-1)); % Normalized controller difference if the maximum
           ypH is 1 and the minimum ypH is -1
356
357     % Calculation of new setpoint for acid/alkali pump
358     % by slave controller
359     % [l/h]
360     cP_ypH      = ceypH*100; % P-part of controller in percent
361
362     % Calculation of T1 or T2 flow rate
363     % Define limits for new setpoint
364     if cP_ypH > 0
365         yT1 = cP_ypH*app.p.KP_pH2a;
366         if yT1 > 100
367             yT1 = 100;
368         end
369         app.v.FT1(end+1) = yT1/100*app.p.FT2max;
370         app.v.FT2(end+1) = 0;
371     elseif cP_ypH < 0
372         yT2 = cP_ypH*app.p.KP_pH2b;
373         if yT2 > 100
374             yT2 = 100;
375         end
376         app.v.FT1(end+1) = 0;
377         app.v.FT2(end+1) = yT2/100*app.p.FT1max;
378     else
379         app.v.FT1(end+1) = 0;
380         app.v.FT2(end+1) = 0;
381     end
382 end
383 end

```

```

384
385 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
386 %% Temperature Control (Manual)
387 if app.p.Mode_temp == 0
388     % Calculation of cooling power with cooling flux ON/OFF
389     % [kg/h]
390     if app.p.f_cooling == 1
391         app.p.mdotC = app.p.mdotCmax;
392     else
393         app.p.mdotC = 0;
394     end
395
396     % Calculation of heating power with heating rod ON/OFF [W]
397     if app.p.f_heating == 1
398         PH = app.p.PHmax;
399     else
400         PH = 0;
401     end
402
403     %% Temperature Control (Auto)
404 elseif app.p.Mode_temp == 1
405     % Calculation of temperature difference to Setpoint and corresponding controller
406     % parameters for temperature control at split range
407     % A PT1 controller is used to filter the error signal and dampen its fluctuations
408     e = app.p.thetaLw-app.v.thetaL(end); % Error/Controller difference
409     % between Temperature in Liquid and Setpoint e = (w-x)
410     app.a.cE = (e+0.001/app.a.deltat*app.a.cE)/(0.001/app.a.deltat+1); %
411     % Filtered error through a PT1 delay with T = 0.001 h and deltat
412     app.a.ce(end+1) = app.a.cE/(app.p.thetaLmaxgr-app.p.thetaLmingr); % Normalized
413     % controller difference e = (w-x) / (w_max - w_min) ; [-1,+1]
414
415     app.a.cP_Part = app.a.ce(end)*app.p.KP_temp1; % P-part of controller with
416     % KP = 0.1
417     app.a.cI_Part(end+1) = app.a.cI_Part(end)+(app.a.ce(end)+app.a.ce(end-1))/2*app.a
418     % .deltat*app.p.KI_temp1; % I-part of controller with KI = 0.01 with time
419     % increment deltat
420
421     % Calculation of steam mass flux and cooling flux in case of steam heating at
422     % split-range
423     wDJ = app.a.cP_Part+app.a.cI_Part(end)+app.p.thetaDJ_WP; % PI sum + Workingpoint
424     % (ThetaDJw = ThetaLw)
425
426     cDJ = wDJ-app.a.thetaDJ; % Error/controller difference between temperature in
427     % double jacket and the calculated setpoint cDJ
428     CDJ = cDJ/(100-0); % Normalized controller difference assuming the double jacket
429     % should not be cooler than 0 C or hotter than 100 C
430
431     yDJ = CDJ*100; % Master controller output in percent
432
433     % Define limits for new setpoint
434     if yDJ > 100
435         yDJ = 100;
436     elseif yDJ < -100

```

```

426         yDJ = -100;
427     end
428
429     if yDJ > 0
430         yH = yDJ*app.p.KP_temp2h; % Slave controller output with KP = 10
431         if yH > 100
432             yH = 100;
433         end
434         app.p.mdotH = (yH/100)*app.p.mdotHmax;
435         app.p.mdotC = 0;
436     else
437         yC = -yDJ*app.p.KP_temp2c; % Slave controller output with KP = -10
438         if yC < -100
439             yC = -100;
440         end
441         app.p.mdotH = 0;
442         app.p.mdotC = (yC/100)*app.p.mdotCmax*10;
443     end
444 end
445
446 % Eigenvalues of foam and anti foam deq. [1/h]
447 lambdaF = -app.v.AAF(end)/app.p.tauF0/(1+app.p.KFpX*app.v.cXL(end));
448 lambdaAF = -app.v.cXL(end)/app.p.KAF;
449 %% Volume
450 DR1 = 0;
451 DR2 = 0;
452 DT1 = 0;
453 DT2 = 0;
454 if app.v.VL(end) > 0
455     if app.v.FR1(end) > 0
456         DR1 = app.v.FR1(end)/app.v.VL(end); % Dilution rate [1/h]
457     end
458     if app.v.FR2(end) > 0
459         DR2 = app.v.FR2(end)/app.v.VL(end); % Dilution rate [1/h]
460     end
461     DT1 = app.v.FT1(end)/app.v.VL(end); % Referred acid titration rate [1/h]
462     DT2 = app.v.FT2(end)/app.v.VL(end); % Referred alkali titration rate [1/h]
463 end
464 % Define ODE function for volume calculation
465 ODE_Vol = @(t,yV) Pichia_ODE_Volume(app.v.FR1(end),app.v.FR2(end),app.v.FH(end),
466     app.v.VL(end),app.v.VR1(end),app.v.VR2(end),app.v.VT2(end),app.v.VT1(end),t,yV);
467
468 % Liquid volume balance
469 [~,yV] = ode45(ODE_Vol,[0 app.a.deltat],[app.v.VL(end) app.v.VR1(end) app.v.VR2(end)
470     app.v.VT2(end) app.v.VT1(end)]);
471 app.v.VL(end+1) = yV(end,1);
472 app.v.VR1(end+1) = yV(end,2);
473 app.v.VR2(end+1) = yV(end,3);
474 app.v.VT2(end+1) = yV(end,4);
475 app.v.VT1(end+1) = yV(end,5);
476
477 app.v.VR1in(end+1) = app.p.VR10-app.v.VR1(end); % Volume added from glycerol
478     reservoir

```

```

476 app.v.VR2in(end+1) = app.p.VR20-app.v.VR2(end); % Volume added from methanol
      reservoir
477 app.v.Vbase(end+1) = app.p.VT20-app.v.VT2(end);
478 app.v.Vacid(end+1) = app.p.VT10-app.v.VT1(end);
479
480 % Referred dilution rate [1/h]
481 Din = DR1+DR2+DT1+DT2;
482 % Temperature liquid phase [K]
483 TL = app.v.thetaL(end)+app.p.TnG;
484
485 % Pressure in reactor measured in gas phase [N/m^2]
486 if app.v.thetaL(end) < 100
487     app.v.pG(end+1) = app.a.pGw;
488     %ExppDL = 0;
489     %pDL = 0;
490 else
491     ExppDL = 10.9-2461/TL-2.065*log10(app.v.thetaL(end)/app.p.TnG);
492     % Steam pressure in liquid phase
493     pDL = 9.8067*10^ExppDL;
494     app.v.pG(end+1) = pDL;
495 end
496
497 % Over pressure indication [bar]
498 app.a.deltapG = (app.v.pG(end)-app.p.pnG)/10^5;
499
500
501 % Quasistationary molar respiration quotient (offgas)
502 RQ_Z = app.v.xCO2(end)/100*(1-app.v.xOGin(end))-app.a.xCGin*(1-app.v.xO2(end)/100); %
      Check if xCGin is correct
503 RQ_N = app.v.xOGin(end)*(1-app.v.xCO2(end)/100)-app.v.xO2(end)/100*(1-app.a.xCGin);
504 if RQ_N ~= 0
505     app.v.RQ(end+1) = RQ_Z/RQ_N;
506 else
507     app.v.RQ(end+1) = 1;
508 end
509 if app.v.RQ(end) <= 0
510     app.v.RQ(end) = 0.0001; % Avoid NaN error for C balance as it requires division
      by RQ
511 end
512
513 % to compare - RQ over metabolism
514 % RQ_int = app.a.yCpO*app.a.MO2/app.a.MCO2;
515
516 % Iterative calculation of pH in liquid phase
517 % Cations of the buffer
518 y1 = (app.v.CB1Ltot(end)+2*app.v.CB2Ltot(end))/app.p.CH0;
519
520 % Anions of the buffer = total phosphoric acid
521 y2 = (app.v.CB1Ltot(end)+app.v.CB2Ltot(end))/app.p.CH0;
522
523 % Dissolved-CO2
524 y3 = app.v.CCLtot(end)/app.p.CH0;
525

```

```

526 % Product acetate
527 y4 = app.v.cP1L(end)/app.p.CH0;
528
529 % Titrated base
530 y5 = app.v.CAlLtot(end)/app.p.CH0;
531
532 % Titrated acid
533 y6 = app.v.CAcLtot(end)/app.p.CH0;
534
535 % Iterative solution
536 xpH = 10^(7-app.v.pH(end));
537 QuotpH = 0.9;
538
539 % Iteration
540 for i = 1:100
541     if (QuotpH <= 0.999 || QuotpH >= 1.001) && xpH >= 0 % If xpH < 0 NaN error occurs
542         SET FLAG THIS WOULD HAVE HAPPENED
543         app.a.xpHkm1 = xpH;
544         fpH0 = xpH^2-1;
545         fpH1 = xpH*y1;
546         fpH2 = -(app.a.ApH*xpH^2+2*app.a.BpH*xpH+3*app.a.CpH)*xpH/(xpH^3+app.a.ApH
547             *xpH^2+app.a.BpH*xpH+app.a.CpH)*y2;
548         fpH3 = -(app.a.DpH+2*app.a.EpH/xpH)*y3;
549         fpH4 = -app.a.FpH*xpH/(app.a.FpH+xpH)*y4;
550         fpH5 = app.a.GpH*xpH^2/(1+app.a.GpH*xpH)*y5;
551         fpH6 = -(app.a.HpH*xpH+2*app.a.IpH)*xpH/(xpH^2+app.a.HpH*xpH+app.a.IpH)*y6
552             ;
553         fpH = fpH0+fpH1+fpH2+fpH3+fpH4+fpH5+fpH6;
554         fstrpH0 = 2*xpH;
555         fstrpH1 = y1;
556         fstrpH2 = -((app.a.ApH^2-2*app.a.BpH)*xpH^4+ ...
557             (2*app.a.ApH*app.a.BpH-6*app.a.CpH)*xpH^3+ ...
558             2*app.a.BpH^2*xpH^2+4*app.a.BpH*app.a.CpH*xpH+ ...
559             3*app.a.CpH^2)/((xpH^3+app.a.ApH*xpH^2+app.a.BpH*xpH+app.a.CpH)^2)*y2;
560         fstrpH3 = 2*app.a.EpH/(xpH*xpH)*y3;
561         fstrpH4 = -app.a.FpH^2/((xpH+app.a.FpH)^2)*y4;
562         fstrpH5 = app.a.GpH*xpH*(2+app.a.GpH*xpH)/((1+app.a.GpH*xpH)^2)*y5;
563         fstrpH6 = -((app.a.HpH^2-2*app.a.IpH)*xpH^2+2*app.a.HpH*app.a.IpH*xpH+2*app.a
564             .IpH^2)/((xpH^2+app.a.HpH*xpH+app.a.IpH)^2)*y6;
565         fstrpH = fstrpH0+fstrpH1+fstrpH2+fstrpH3+fstrpH4+fstrpH5+fstrpH6;
566         xpH = xpH-fpH/fstrpH;
567         QuotpH = app.a.xpHkm1/xpH;
568     else
569         break
570     end
571 end
572
573 if xpH < 0
574     xpH = -xpH; % Set flag that this occurred
575 end
576
577 % pH value in liquid phase
578 app.v.pHL(end+1) = 7-log10(xpH);

```

```

575 % Molar concentration of the H+ ions in the liquid phase
576 CHL = xPH*app.p.CH0;
577 % Influence of temperature and pH of the growth
578 if app.v.pHL(end) >= app.p.pHLmingr && app.v.pHL(end) <= app.p.pHLmaxgr
579     fpH = (app.v.pHL(end)-app.p.pHLmingr)*(app.v.pHL(end)-app.p.pHLmaxgr)/((app.v.
580         pHL(end)-app.p.pHLmingr)*(app.v.pHL(end)-app.p.pHLmaxgr)-(app.v.pHL(end)-app.
581         p.pHLoptgr)^2);
582 else
583     fpH = 0;
584 end
585 if app.v.thetaL(end) >= app.p.thetaLmingr && app.v.thetaL(end) <= app.p.thetaLmaxgr
586     ftheta = ((app.v.thetaL(end)-app.p.thetaLmaxgr)*(app.v.thetaL(end)-app.p.
587         thetaLmingr)^2)/((app.p.thetaLoptgr-app.p.thetaLmingr)*((app.p.thetaLoptgr-
588         app.p.thetaLmingr)*(app.v.thetaL(end)-app.p.thetaLoptgr)-(app.p.thetaLoptgr-
589         app.p.thetaLmaxgr)*(app.p.thetaLoptgr+app.p.thetaLmingr-2*app.v.thetaL(end)))
590     );
591 else
592     ftheta = 0;
593 end
594 % Maximum specific growth rate glycerol [1/h]
595 my1max = app.p.my1opt*fpH*ftheta;
596 % Maximum specific growth rate methanol [1/h]
597 %my2max = app.p.my2opt*fpH*ftheta; % Now calculated with qS2pXmax
598 % Maximum specific glycerol uptake rate [1/h]
599 app.a.qS1pXmax = (my1max+app.a.mySm)/app.p.yXpS1gr;
600 % Maximum specific methanol uptake rate [1/h]
601 % app.a.qS2pXmax = (my2max+app.a.mySm)/app.p.yXpS2gr; % Old model
602 % Alternative calculation of qS2pXmax (Cornelissen)
603 cS2Lopt = sqrt(app.p.kS2*app.p.kI22); % g/l
604 app.a.qS2pXmax = app.p.qS2pXsup*(cS2Lopt/(app.p.kS2+cS2Lopt))*(app.p.kI22/(app.p.kI22
605     +cS2Lopt)); % 1/h
606 % Optimal cellspecific growth rate (methanol dependend) [1/h]
607 app.p.my2opt = app.p.yXpS2gr*app.a.qS2pXmax-app.a.mySm;
608 % Maximum specific growth rate methanol [1/h]
609 my2max = app.p.my2opt*fpH*ftheta; % New model
610 %% Respiration
611 % Maximum specific oxygen uptake rate [1/h]
612 app.a.qOpXmax = (my1max+app.a.mySm)/app.p.yXp0gr+app.p.qOpXm;
613 % Calculation of O2 quantities
614 % Maintenance O2 uptake rate [1/h]
615 app.v.OURm(end+1) = app.p.qOpXm*app.v.cXL(end);
616 % Maximum O2 uptake rate [1/h]

```



```

621 app.v.OURmax(end+1) = app.a.qOpXmax*app.v.cXL(end);
622
623 % Calculation of O2 Henry constant [Nm/kg]
624 app.a.HO2 = app.p.HnO2/(1+app.p.K1HO2*app.v.thetaL(end)+app.p.K2HO2*app.v.thetaL(end)
    ^2+app.p.K3HO2*app.v.thetaL(end)^3+app.p.K4HO2*app.v.thetaL(end)^4);
625
626 % O2-concentration in liquid phase at 100 % pO2-indication
627 app.a.cOL100 = app.p.pGcal*app.p.xOGcal/app.a.HO2;
628
629 % Maximum potential O2 concentration in liquid phase [g/l]
630 app.a.cOLmax = app.v.pG(end)/app.a.HO2;
631
632 % Maximum oxygen supply rate [g/(l*h)]
633 if app.v.VL(end) > 0
634     app.v.QO2max(end+1) = app.v.FnG(end)*60*app.p.MO2/(app.p.VnM*app.v.VL(end));
635 else
636     app.v.QO2max(end+1) = 0;
637 end
638
639 % Maximum CO2 supply rate [g/(l*h)]
640 app.v.QCO2max(end+1) = app.v.QO2max(end)*app.p.MCO2/app.p.MO2;
641
642 % Calculation of viscosity [Ns/m^2]
643 eta = app.p.etaXL*(app.p.etaH2O/app.p.etaXL)^(app.v.cXL(end)/app.p.cXLeta);
644
645 % Volume refered O2-transition coefficient kLa [1/h]
646 VLwert = (app.v.VL(end)/app.p.VLmin)^app.p.alpha;
647 NStwert = (app.v.NSt(end)/app.p.NStmax)^(3*app.p.alpha);
648 FGwert = (app.v.FnG(end)/app.p.FnGmax)^app.p.beta;
649 etawert = (eta/app.p.etaH2O)^app.p.gamma;
650
651 app.v.kLa(end+1) = app.p.kLamin+app.p.kLamax*FGwert*NStwert/VLwert*etawert*(1-app.v.
    AAF(end));
652
653 % Theoretical maximum O2 transfer rate [g/(l*h)]
654 app.v.OTRmax(end+1) = app.v.kLa(end)*app.a.cOLmax;
655
656 % Calculation of Stanton coefficient of oxygen
657 if app.v.QO2max(end) > 0
658     St0 = app.v.OTRmax(end)/app.v.QO2max(end);
659 else
660     St0 = 10^-20;
661 end
662
663 % Optimum O2 limiting transport rate
664 % OTRopt = app.v.OTRmax(end)/(1+St0);
665
666 % O2-pulp quantity in reactor liquid phase
667 app.v.xOL(end+1) = app.v.cOL(end)/app.a.cOLmax;
668
669 % OTR [g/(l*h)]
670 app.v.OTR(end+1) = app.v.OTRmax(end)*(app.v.xOGin(end)-app.v.xOL(end))*2/((1+St0-(1-
    app.v.RQ(end))*St0*app.v.xOL(end))+sqrt((1+St0-(1-app.v.RQ(end))*St0*app.v.xOL(

```

```

        end))^2-4*(1-app.v.RQ(end))*StO*(app.v.xOGin(end)-app.v.xOL(end)));
671 if app.v.OTR(end) < 0
672     app.v.OTR(end) = 0;
673 end
674
675 % Calculation of quasistationary O2 gas phase mole fraction [-]
676 if app.v.OTR(end) ~= 0
677     app.v.xOG(end+1) = (app.v.QO2max(end)*app.v.xOGin(end)-app.v.OTR(end))/(app.v.
        QO2max(end)-(1-app.v.RQ(end))*app.v.OTR(end));
678 else
679     app.v.xOG(end+1) = app.v.xOGin(end);
680 end
681
682 % Calculation of CO2 Henry constant [Nm/kg]
683 app.a.HnCO2 = app.p.HnCO2/(1+app.p.K1HCO2*app.v.thetaL(end)+app.p.K2HCO2*app.v.thetaL(
        end)^2+app.p.K3HCO2*app.v.thetaL(end)^3+app.p.K4HCO2*app.v.thetaL(end)^4);
684
685 % Maximum CO2 concentration [g/l]
686 app.a.cCLmax = app.v.pG(end)/app.a.HnCO2;
687
688 % Dissolved CO2-concentration in liquid phase [g/l]
689 app.a.cCL = (CHL^2*app.p.MCO2*app.v.CCLtot(end))/(CHL^2+app.p.KC1*CHL+app.p.KC1*app.p
        .KC2);
690
691 % CO2-pulp quantity in reactor liquid phase [-]
692 app.a.xCL = app.a.cCL/app.a.cCLmax;
693
694 % Theoretical maximum CO2 transfer rate [g/(l*h)]
695 app.a.CTRmax = app.p.deltaCpO*app.v.kLa(end)*app.a.cCLmax;
696
697 % Stanton coefficient of CO2
698 if app.v.QCO2max(end) > 0
699     app.a.StC = app.a.CTRmax/app.v.QCO2max(end);
700 else
701     app.a.StC = 10^20;
702 end
703
704 % CO2 transfer rate
705 app.v.CTR(end+1) = app.a.CTRmax*(app.a.xCGin-app.a.xCL)*2/((1+app.a.StC-(1-1/app.v.RQ
        (end))*app.a.StC*app.a.xCL)+sqrt((1+app.a.StC-(1-1/app.v.RQ(end))*app.a.StC*app.a
        .xCL)^2-4*(1-1/app.v.RQ(end))*app.a.StC*(app.a.xCGin-app.a.xCL)));
706
707 % Calculation of the quasistationary CO2 gas phase mole fraction
708 app.v.xCG(end+1) = (app.v.QCO2max(end)*app.a.xCGin-app.v.CTR(end))/(app.v.QCO2max(end)
        )-(1-1/app.v.RQ(end))*app.v.CTR(end));
709
710 % Optimum specific substrate uptake rate (no O2 limitation) [1/h]
711 if app.v.cS1L(end) <= 0
712     qS1pXopt = 0;
713 else
714     qS1pXopt = app.a.qS1pXmax*app.v.cS1L(end)/(app.v.cS1L(end)+app.p.kS1);
715 end
716

```

```

717 % Optimum specific substrate uptake rate 2 [1/h]
718 if app.v.cS2L(end) <= 0
719     qS2pXopt = 0;
720 else
721     qS2pXopt = app.a.qS2pXmax*app.v.cS2L(end)/(app.v.cS2L(end)+app.p.kS2)*app.p.kI21
        /(app.v.cS1L(end)+app.p.kI21);
722 end
723
724 % Specific cell growth rate at substrate limitation
725 qXpXSgr = app.p.yXpS1gr*qS1pXopt+app.p.yXpS2gr*qS2pXopt;
726
727 % Specific cell growth rate at O2 limitation
728 if app.v.cOL(end) <= 0
729     qXpX0gr = 0;
730 else
731     qXpX0gr = app.p.yXp0gr*(app.a.qOpXmax-app.p.qOpXm)*app.v.cOL(end)/(app.p.kO+app.v
        .cOL(end));
732 end
733
734 % Specific cell growth rate
735 if qXpXSgr < qXpX0gr
736     qXpXgr = qXpXSgr;
737     if any(app.p.f_Inoc == 1)
738         % "Substrate Limitation"
739     end
740 else
741     qXpXgr = qXpX0gr;
742     if any(app.p.f_Inoc == 1)
743         % "Oxygen Limitation"
744     end
745 end
746
747 % Determine methanol toxicity and its influence on cell growth
748 % Toxicity turn on function
749 VS2tox = ((app.v.cS2L(end)./app.p.kS2tox).^app.p.kappatox)./(1+((app.v.cS2L(end)./app
        .p.kS2tox).^app.p.kappatox));
750
751 % Calculation of cell specific reaction rates
752 % Specific cell reaction rate
753 if any(app.p.f_Inoc == 1)
754     app.v.qXpX(end+1) = qXpXgr-app.a.mySm-VS2tox*app.p.qXpXtox;
755 else
756     app.v.qXpX(end+1) = 0;
757 end
758
759 % Specific substrate uptake rate glycerine [1/h]
760 SSG = qXpXgr/app.p.yXpS1gr;
761 if SSG < qS1pXopt
762     qS1pX = SSG;
763 else
764     qS1pX = qS1pXopt;
765 end
766

```

```

767 %% AOX induction for methanol metabolism
768 bS2script = app.p.aS2script + my2max;
769 bS2trans = app.p.aS2trans + my2max;
770 bS2act = app.p.aS2act + my2max;
771
772 % Turn on/turn Off functions
773 VS2ind = ((app.v.cS2L(end)/app.p.kS2ind)^app.p.kappaind)/(1+(app.v.cS2L(end)/app.p.
    kS2ind)^app.p.kappaind);
774 VS1rep = 1/(1+(app.v.cS1L(end)/app.p.kS1rep)^app.p.kapparep);
775
776 app.v.qS2pXind(end+1) = VS1rep*VS2ind*app.p.qS2pXsup;
777
778 ODE_Induction = @(t,y) Pichia_Induction_Cornelissen(app.v.qXpX(end),app.v.qS2pXind(
    end),app.p.aS2script,app.p.aS2trans,app.p.aS2act,bS2script,bS2trans,bS2act,t,y);
779 [~,y] = ode45(ODE_Induction,[0 app.a.deltat],[app.v.qS2pXscript(end), app.v.
    qS2pXtrans(end), app.v.qS2pXact(end)]);
780
781 app.v.qS2pXscript(end+1) = y(end,1);
782 app.v.qS2pXtrans(end+1) = y(end,2);
783 app.v.qS2pXact(end+1) = y(end,3);
784
785 % Glycerol dependend methanol uptake rate [1/h]
786 app.v.qS2pX = app.v.qS2pXact(end)*(app.v.cS2L(end)/(app.p.kS2+app.v.cS2L(end)))*(app.
    p.kI22/(app.p.kI22+app.v.cS2L(end)))*(app.p.kI21/(app.p.kI21+app.v.cS1L(end)));
787
788 %% Target protein expression
789 bP1script = app.p.aP1script + my2max;
790 bP1trans = app.p.aP1trans + my2max;
791 bP1act = app.p.kP1alpha + my2max;
792
793 % Turn on/turn Off functions
794 VS2ind = ((app.v.cS2L(end)/app.p.kS2ind)^app.p.kappaind)/(1+(app.v.cS2L(end)/app.p.
    kS2ind)^app.p.kappaind);
795 VS1rep = 1/(1+(app.v.cS1L(end)/app.p.kS1rep)^app.p.kapparep);
796
797 app.v.qP1pXind(end+1) = VS1rep*VS2ind*app.p.qP1pXsup;
798
799 delta_qP1pXscript = app.v.qP1pXscriptw - app.v.qP1pXscript; % Calculate control
    deviation
800 if length(app.t) > 1 && length(delta_qP1pXscript) > 1 % need at least two values for
    trapz()
801     app.v.qP1pXscriptw(end+1) = VS1rep*VS2ind*app.p.qP1pXmax; % Calculate scriptw for
        next cycle
802     app.v.qP1pXback(end+1) = app.p.KCP1script*(delta_qP1pXscript(end)+trapz(app.t,
        delta_qP1pXscript)/app.p.TIP1script);
803 else
804     app.v.qP1pXscriptw(end+1) = 0;
805     app.v.qP1pXback(end+1) = 0;
806 end
807 ODE_Expression = @(t,y) Pichia_Expression_Cornelissen(app.v.qXpX(end),app.v.qP1pXind(
    end),app.v.qP1pXback(end),app.p.aP1script,app.p.aP1trans,app.p.kP1alpha,bP1script
    ,bP1trans,bP1act,t,y);
808

```

```

809 % IVP
810 [~,y] = ode45(ODE_Expression, [0 app.a.deltat], [app.v.qP1pXscript(end),app.v.
      qP1pXtrans(end),app.v.qP1pXact(end)]);
811
812 app.v.qP1pXscript(end+1) = y(end, 1);
813 app.v.qP1pXtrans(end+1) = y(end, 2);
814 app.v.qP1pXact(end+1) = y(end, 3);
815
816 % Glycerol and methanol dependend product formation rate [1/h]
817 app.v.qP1pX(end+1) = app.v.qP1pXact(end);
818
819 % Specific ammonia uptake rate [1/h]
820 qAlpX = app.p.yAlpXgr*qXpXgr;
821
822 % Specific uptake rate of titrated acid [1/h]
823 qAcpX = app.p.yAcpXgr*qXpXgr;
824
825 % Specific oxygen uptake rate [1/h]
826 qOpX = qXpXgr/app.p.yXpOgr+app.p.qOpXm;
827
828 % Resulting O2 yield coefficient
829 %yXpO = app.p.yXpOgr*app.v.qXpX(end)/(app.v.qXpX(end)+app.a.myOm);
830
831 % Volumetric oxygen uptake rate [g/(l*h)] and kLa [1/h]
832 if any(app.p.f_Inoc == 1)
833     app.v.OUR(end+1) = qOpX*app.v.cXL(end);
834 else
835     app.v.OUR(end+1) = 0;
836 end
837
838 % Volumetric oC02 production rate [g/(l*h)]
839 CER = app.p.yCpO*app.v.OUR(end);
840
841 % Specific production rate, S-limited, O2-inhibited [1/h]
842 % % qPpX = (app.p.yPpS1*qS1pXopt+app.p.yPpS2*qS2pXopt)*app.p.kIPO/(app.p.kIPO+app.v.
      cOL(end))-app.p.yPpS3*qS3pX;
843
844 % Calculation of ammonia transfer rate (volatile) [g/(l*h)]
845 AlTR = -app.p.KAlvol*app.v.FnG(end)*app.v.CAlLtot(end)*app.p.MAl/app.v.VL(end);
846
847 % Calculation of variables in the temperature systems
848 % Quasitatory calculation of the heat exchanger
849 % temperature
850 % Dilution rate of the steam flux [1/h]
851 if app.p.Mode_temp == 1
852     DH = app.p.mdotH/app.a.mHmax;
853 end
854
855 % Time constant of the primary heating cycle [h]
856 tauHTh = app.a.mHmax*app.p.ch20/(app.a.kHTh*app.p.AHTh);
857
858 % Dimensionless quantities
859 if app.p.Mode_temp == 1

```

```

860     phiHTh = DH*tauHTh;
861     phiHT = app.p.mdotH/app.p.mdotT;
862 end
863
864 % Temperature of heating outlet
865 % Electrical heating [ C ]
866 if app.p.Mode_temp == 0
867     if app.p.f_heating == 1
868         app.a.thetaTh = app.v.thetaD(end)+app.a.RT*PH;
869     else
870         app.a.thetaTh = app.v.thetaL(end); % Added this line to account for thetaTh
            when the heating power is turned off
871     end
872 end
873
874 % Steam heating as alternative for electrical heating
875 if app.p.Mode_temp == 1
876     app.a.thetaTh = ((1+phiHTh)*app.a.CHmax*app.v.thetaD(end)+phiHT*(app.a.CHmax*app.
            p.thetaHin+app.a.Qny(end)))/(app.a.CHmax*(1+phiHTh+phiHT));
877 end
878
879 % Dilution rate of cooling flux [1/h]
880 DC = app.p.mdotC/app.p.mC;
881 phiCTc = DC*app.a.tauCTc;
882
883 % Cooling outlet temperature [ C ]
884 app.a.thetaTc = (phiCTc*app.p.thetaCin+(1+phiCTc)*app.a.phiTcC*app.a.thetaTh)/((1+app.
            .a.phiTcC)*(1+phiCTc)-1);
885
886 % Cooling water outlet temperature [ C ]
887 app.a.thetaC = (phiCTc*app.p.thetaCin+app.a.thetaTc)/(1+phiCTc);
888
889 % Cascade control quantity
890 app.a.thetaDJ = app.a.thetaTc;
891
892 % Volumetric heat capacity of the liquid phase and share of
893 % the wall [Wh/K]
894 CL = app.p.rhoL*app.v.VL(end)*app.p.cH2O+app.p.mWL*app.p.cW;
895
896 % Time constant liquid phase - double jacket [h]
897 tauLD = CL/(app.a.kDL*app.p.ADL);
898
899 % Time constant liquid phase - environment [h]
900 tauLU = CL/(app.a.kLU*app.p.ALU);
901
902 % Time constant reactor liquid phase [h]
903 tauL = 1/(1/tauLD+1/tauLU);
904
905 % Microbiological heat generation [W]
906 QdotM = app.p.KHM*app.v.VL(end)*app.v.OUR(end);
907
908 % Thermal efficiency of the stirrer
909 QdotSt = app.p.KHSt*app.v.VL(end)*app.v.NSt(end)^3;

```

```

910
911 % DEFINITION OF DYNAMIC MODEL EQUATIONS
912 % Cell mass balances and inoculation
913 ODE_concbbal = @(t,y) Pichia_ODE_Luttmann(app.v.qXpX(end),Din,app.p.cS1R1,qS1pX,app.p.
    cS2R1,app.v.qS2pX,app.p.cS1R2,app.p.cS2R2,app.p.cP1R1,app.p.cP1R2,app.v.qP1pX(end)
    ),app.p.kP1alpha,app.p.cOT1,app.p.cOT2,app.p.cOR1,app.p.cOR2,app.v.OTR(end),app.v
    .OUR(end),app.a.cOL100,app.p.TMpO2,DT1,app.p.CAcT1tot,qAcpX,app.p.MAc,DT2,app.p.
    CA1T2tot,qAlpX,AlTR,app.p.MA1,DR1,DR2,app.p.CCR1tot,app.p.CCT1tot,app.p.CCT2tot,
    app.v.CTR(end),CER,app.p.MCO2,app.v.pHL(end),app.p.TMpH,app.p.TMxO2,app.v.xOG(end)
    ),app.p.TMxC02,app.v.xCG(end),lambdaF,app.p.qhpX,lambdaAF,AAFin,app.a.tauD,app.a.
    DD,app.a.thetaTc,app.a.tauDL,app.p.thetaU,app.a.tauDU,tauL,tauLD,tauLU,QdotM,
    QdotSt,CL,t,y);
914
915 [~,y] = ode45(ODE_concbbal,[0 app.a.deltat],[app.v.cXL(end) app.v.cS1L(end) app.v.cS2L
    (end) app.v.cP1X(end) app.v.cP1L(end) app.v.cOL(end) app.v.pO2(end) app.v.CB1Ltot
    (end) app.v.CB2Ltot(end) app.v.CAcLtot(end) app.v.CA1Ltot(end) app.v.CCLtot(end)
    app.v.pH(end) app.v.xO2(end) app.v.xCO2(end) app.v.hF(end) app.v.AAF(end) app.v.
    thetaD(end) app.v.thetaL(end)]);
916 % all y values can not be smaller than 0
917 for i = 1:18
918     if y(end,i) < 0
919         y(end,i) = 0;
920     end
921 end
922 % Check if inoculation occurred
923 if app.p.f_Inoc == 1 && app.a.inoc_occ == 0
924     app.v.cXL(end+1) = app.p.cXL0; % [g/l]
925     app.a.ToI = app.t(end);
926     app.a.inoc_occ = 1;
927     logMessage(app.Startingscreen, app.projectID, "Inoculation occurred")
928 else
929     app.v.cXL(end+1) = y(end,1);
930 end
931 app.v.cS1L(end+1) = y(end,2);
932 app.v.cS2L(end+1) = y(end,3);
933 app.v.cP1X(end+1) = y(end,4);
934 app.v.cP1L(end+1) = y(end,5);
935 app.v.cOL(end+1) = y(end,6);
936 app.v.pO2(end+1) = y(end,7);
937 app.v.CB1Ltot(end+1) = y(end,8);
938 app.v.CB2Ltot(end+1) = y(end,9);
939 app.v.CAcLtot(end+1) = y(end,10);
940 app.v.CA1Ltot(end+1) = y(end,11);
941 app.v.CCLtot(end+1) = y(end,12);
942 app.v.pH(end+1) = y(end,13);
943 app.v.xO2(end+1) = y(end,14);
944 app.v.xCO2(end+1) = y(end,15);
945 app.v.hF(end+1) = y(end,16);
946 app.v.AAF(end+1) = y(end,17);
947 app.v.thetaD(end+1) = y(end,18);
948 app.v.thetaL(end+1) = y(end,19);
949 if app.v.thetaL(end) > 100.0
950     app.v.thetaL(end) = 100.0; % Set flag that this happened!

```

```

951     logMessage(app.Startingscreen, app.projectID, "Temperature >100 C  was reached");
952 end
953
954 % Volumetric substrate intake
955 app.v.QS1in(end+1) = (app.v.FR1(end)*app.p.cS1R1+app.v.FR2(end)*app.p.cS1R2)/app.v.VL
    (end);
956 app.v.QS2in(end+1) = (app.v.FR1(end)*app.p.cS2R1+app.v.FR2(end)*app.p.cS2R2)/app.v.VL
    (end);
957
958 % Measurment data
959 app.v.pHLm(end+1) = meas_transfer_function(app.v.pHL(end), app.v.pHLm(end), app.p.
    taupHL, app.a.deltat);
960 app.v.pO2m(end+1) = meas_transfer_function(app.v.pO2(end), app.v.pO2m(end), app.p.
    taupO2, app.a.deltat);
961 app.v.thetaLm(end+1) = meas_transfer_function(app.v.thetaL(end), app.v.thetaLm(end),
    app.p.tauthetaL, app.a.deltat);
962 app.v.cS1Lm(end+1) = meas_transfer_function(app.v.cS1L(end), app.v.cS1Lm(end), app.p.
    taucS1L, app.a.deltat);
963 app.v.cS2Lm(end+1) = meas_transfer_function(app.v.cS2L(end), app.v.cS2Lm(end), app.p.
    taucS2L, app.a.deltat);
964
965 % Add time to process time
966 app.t(end+1) = app.t(end) + app.a.deltat;
967
968 % Set Volume Flag if VLmax was reached
969 if app.v.VL(end) >= app.p.VLmax
970     app.a.VolumeFlag = 1;
971 end
972
973 % Display error message if VLmax was reached and set VLFlag
974 % to avoid displaying the message multiple times
975 if any(app.a.VolumeFlag == 1) && any(app.a.VLflag == 0)
976     app.a.VLflag = 1;
977     stop(app.Timer);
978     msg = sprintf('VLmax of %g l was reached', app.v.VL(end));
979     errordlg(msg);
980     logMessage(app.Startingscreen, app.projectID, msg)
981 end
982 app_new = app;
983 end

```


6.6 *P. pastoris* Parameter Table

Table 6.1: Model Parameters

Parameter	Value	Unit	Category	Reading Rate
NStw	400	1/min	Setpoint	cyclic
pHw	4.8	-	Setpoint	cyclic
pO2w	20	%	Setpoint	cyclic
thetaLw	30	°C	Setpoint	cyclic
LWw	14	kg	Setpoint	cyclic
FnGw	6	l/min	Setpoint	cyclic
FnAIRw	7.5	l/min	Setpoint	cyclic
FnO2w	0	l/min	Setpoint	cyclic
FnN2w	0	l/min	Setpoint	cyclic
FnCO2w	0	l/min	Setpoint	cyclic
deltapGw	0.5	bar	Setpoint	cyclic
FHrelw	20	%	Setpoint	cyclic
cS1Lw	5	g/l	Setpoint	cyclic
cS2Lw	2	g/l	Setpoint	cyclic
FR1w	0.155	l/h	Setpoint	cyclic
FR2w	0.05	l/h	Setpoint	cyclic
cXL0	3	g/l	Initial value	once
pO20	100	%	Initial value	once
pH0	5	-	Initial value	once
VL0	5	l	Initial value	once
thetaL0	30	°C	Initial value	once
thetaD0	30	°C	Initial value	once
cS1L0	25	g/l	Initial value	once
cS2L0	0	g/l	Initial value	once
cS3L0	0	g/l	Initial value	once
cP1L0	0	g/l	Initial value	once
cP1X0	0	g/l	Initial value	once
cOL0	0.014735071	g/l	Initial value	once

Parameter	Value	Unit	Category	Reading Rate
pGcal	150000	n/m ²	Calibration	once
xOGcal	0.2094	-	Calibration	once
xCGcal	0.004	-	Calibration	once
cS1R1	150	g/l	Substrate reservoir	once
cS2R1	0	g/l	Substrate reservoir	once
cXR1	0	g/l	Substrate reservoir	once
VR10	3	l	Substrate reservoir	once
rhoR1	1.263	g/l	Substrate reservoir	once
cOR1	0	g/l	Substrate reservoir	once
cP1R1	0	g/l	Substrate reservoir	once
CCR1tot	0	mol/l	Substrate reservoir	once
cS1R2	0	g/l	Substrate reservoir	once
cS2R2	792	g/l	Substrate reservoir	once
cXR2	0	g/l	Substrate reservoir	once
VR20	2	l	Substrate reservoir	once
rhoR2	0.79	g/l	Substrate reservoir	once
cOR2	0	g/l	Substrate reservoir	once
cP1R2	0	g/l	Substrate reservoir	once
CCR2tot	0	mol/l	Substrate reservoir	once
kR1	0.5	-	Feed control	cyclic
FR1max	0.5	l/h	Feed control	cyclic
qXpX1w	0.1	1/h	Feed control	cyclic
kR2	0.5	-	Feed control	cyclic
FR2max	0.5	l/h	Feed control	cyclic
qXpX2w	0.05	1/h	Feed control	cyclic
R_feed	1		Feed control	cyclic
CPLtot0	0	mol/l	pH control	once
CAcLtot0	0	mol/l	pH control	once
CAiLtot0	0.05	mol/l	pH control	once
CB1Ltot0	0.098	mol/l	pH control	once
CB2Ltot0	0.03	mol/l	pH control	once
CAcT1tot	2.44	mol/l	pH control	once
CAiT2tot	16.47	mol/l	pH control	once
CCT1tot	0	mol/l	pH control	once
CCT2tot	0	mol/l	pH control	once
cOT1	0	g/l	pH control	once
cOT2	0	g/l	pH control	once
KAc1	0.0154	mol/l	pH control	once
KAc2	0.000000102	mol/l	pH control	once
KAI	0.0000176	mol/l	pH control	once

Parameter	Value	Unit	Category	Reading Rate
KAlvol	0.00462	-	pH control	once
KB1	0.00752	mol/l	pH control	once
KB2	0.0000000623	mol/l	pH control	once
KB3	0.0000000000002	mol/l	pH control	once
KP1	0.0000176	mol/l	pH control	once
MAc	82	g/mol	pH control	once
MAI	17	g/mol	pH control	once
Mode_pH	1		Modes	cyclic
Mode_temp	1		Modes	cyclic
Mode_pO2	1		Modes	cyclic
Mode_harvest	0		Modes	cyclic
Mode_feed	0		Modes	cyclic
f_InocStart	0		Flags	cyclic
f_motor	1		Flags	cyclic
f_aeration	1		Flags	cyclic
f_air	1		Flags	cyclic
f_O2	1		Flags	cyclic
f_N2	0		Flags	cyclic
f_CO2	0		Flags	cyclic
f_acid	0		Flags	cyclic
f_alkali	0		Flags	cyclic
f_antifoam	0		Flags	cyclic
f_heating	1		Flags	cyclic
f_cooling	1		Flags	cyclic
f_harvest	0		Flags	cyclic
f_feed	0		Flags	cyclic
f_FR1	0		Flags	cyclic
f_FR2	0		Flags	cyclic
KP_pH	10000		Controller gain	cyclic
KP_pH2a	1		Controller gain	cyclic
KP_pH2b	-1		Controller gain	cyclic
taupHL	20	s	Controller gain	cyclic
KP_temp1	8		Controller gain	cyclic
KI_temp1	0.1		Controller gain	cyclic
KP_temp2c	10		Controller gain	cyclic
KP_temp2h	0.01		Controller gain	cyclic
tauthetaL	20	s	Controller gain	cyclic
KP_agi	10		Controller gain	cyclic
KI_agi	1000		Controller gain	cyclic
KP_aeration	20		Controller gain	cyclic
KD_agi	0.015		Controller gain	cyclic
KI_aeration	1000		Controller gain	cyclic
KD_aeration	0.0188		Controller gain	cyclic

Parameter	Value	Unit	Category	Reading Rate
KP_gasmix	0.4		Controller gain	cyclic
KI_gasmix	0.003		Controller gain	cyclic
KD_gasmix	0.0005		Controller gain	cyclic
KP_feedpO2	-2		Controller gain	cyclic
KI_feedpO2	-15		Controller gain	cyclic
KD_feedpO2	-0.009		Controller gain	cyclic
taupO2	25	s	Controller gain	cyclic
KP_LW	10		Controller gain	cyclic
KI_LW	0.3		Controller gain	cyclic
KD_LW	1		Controller gain	cyclic
KP_feedR1	15		Controller gain	cyclic
KI_feedR1	500		Controller gain	cyclic
KD_feedR1	0.02		Controller gain	cyclic
KP_feedR2	-2		Controller gain	cyclic
KI_feedR2	-15		Controller gain	cyclic
KD_feedR2	-0.009		Controller gain	cyclic
taucS1L	100	s	Controller gain	cyclic
taucS2L	600	s	Controller gain	cyclic
ypH_SET	0		Controller gain	cyclic
xOAIR	0.2094	-	General	once
xCAIR	0.0004	-	General	once
VnM	22.412	l/mol	General	once
TnG	273.15	K	General	once
R	8314	N*m/(kmol*K)	General	once
pnG	101330	N/m^2	General	once
MO2	32	g/mol	General	once
MCO2	44	g/mol	General	once
mdotT	1500	kg/h	General	once
etaH2O	0.001002	Ns/m^2	General	once
etaXL	0.001	Ns/m^2	General	once
cXLeta	100	g/l	General	once
HO20	1440000	N*m/kg	General	once
HnO2	1448000	N*m/kg	General	once
HCO20	30010	N*m/kg	General	once
HnCO2	30010	N*m/kg	General	once
K1HO2	-0.02723	°C ⁽⁻¹⁾	General	once
K2HO2	0.0005627	°C ⁽⁻²⁾	General	once
K3HO2	-0.000006597	°C ⁽⁻³⁾	General	once
K4HO2	0.0000000328	°C ⁽⁻⁴⁾	General	once
K1HCO2	-0.03889	°C ⁽⁻¹⁾	General	once
K2HCO2	0.0009442	°C ⁽⁻²⁾	General	once
K3HCO2	-0.00001328	°C ⁽⁻³⁾	General	once
K4HCO2	0.0000000811	°C ⁽⁻⁴⁾	General	once

Parameter	Value	Unit	Category	Reading Rate
KC1	0.000000431	mol/l	General	once
KC2	0.0000000000561	mol/l	General	once
alpha	0.5	-	General	once
beta	0.5	-	General	once
gamma	-0.05	-	General	once
cH2O	1.167	h/(kgK)	General	once
deltahv	2170	kJ/kg	General	once
CH0	0.0000001	mol/l	General	once
my1opt	0.4	1/h	Cell growth	once
yXpS1	0.7235	-	Cell growth	once
yXpS1gr	0.67	-	Cell growth	once
yP1pS1	0	-	Cell growth	once
qS1pXm	0.0268	1/h	Cell growth	once
kS1	0.01327	g/l	Cell growth	once
kl21	0.01233	g/l	Cell growth	once
my2opt	0.01	1/h	Cell growth	once
yXpS2	0.2771	-	Cell growth	once
yXpS2gr	0.2	-	Cell growth	once
qS2pXm	0.02	1/h	Cell growth	once
kS2	0.7429	g/l	Cell growth	once
yXpOgr	1.773	-	Cell growth	once
yXpOgr	40	-	Cell growth	once
yCpO	1.375	-	Cell growth	once
yCpO	15	-	Cell growth	once
qOpXm	0.0117	1/h	Cell growth	once
qOpXm	0.5	1/h	Cell growth	once
deltaCpO	0.69	-	Cell growth	once
kO	0.0004	g/l	Cell growth	once
klP1O	0.0005	g/l	Cell growth	once
MP1	28200	g/mol	Cell growth	once
yAcpXgr	0	-	Cell growth	once
yAlpXgr	0.185	-	Cell growth	once
thetaDJ_WP	32	°C	Growth temperature	once
thetaLoptgr	30	°C	Growth temperature	once
thetaLmingr	5	°C	Growth temperature	once
thetaLmaxgr	47	°C	Growth temperature	once
AAFast	100	1/h	Foam	once
KAF	4	(g*h)/l	Foam	once
KFpX	0.09	l/g	Foam	once
qhpX	0.1	l/(g*h)	Foam	once
tauF0	0.0125	h	Foam	once
Ttast	0.00008333	h	Foam	once
VAF	0.975	-	Foam	once

Parameter	Value	Unit	Category	Reading Rate
pHLoptgr	4.8	-	pH	once
pHLmingr	2	-	pH	once
pHLmaxgr	9	-	pH	once
deltatsec	2	s	Settings	invisible
speedfactor	1		Settings	invisible
tmax	5	h	Settings	invisible
f_Inoc	0		Settings	invisible
aP1script	104	1/h	Pichia induction/protein expression	once
aP1trans	15.04	1/h	Pichia induction/protein expression	once
aS2act	0.1348	1/h	Pichia induction/protein expression	once
aS2script	76.6	1/h	Pichia induction/protein expression	once
aS2trans	5.463	1/h	Pichia induction/protein expression	once
kappaind	6	-	Pichia induction/protein expression	once
kapparep	4	-	Pichia induction/protein expression	once
KCP1script	0.3106	-	Pichia induction/protein expression	once
kl22	21.29	g/l	Pichia induction/protein expression	once
kP1alpha	19.58	1/h	Pichia induction/protein expression	once
kS1rep	0.00162	g/l	Pichia induction/protein expression	once
kS2ind	0.001	g/l	Pichia induction/protein expression	once
qP1pXmax	0.00001073	g/gh	Pichia induction/protein expression	once
qP1pXsup	0.00005139	g/gh	Pichia induction/protein expression	once
qS2pXsup	0.3303	1/h	Pichia induction/protein expression	once
TIP1script	5.367	h	Pichia induction/protein expression	once
kS2tox	40	g/l	Pichia induction/protein expression	once
kappatox	15	-	Pichia induction/protein expression	once
qXpXtox	0.5	1/h	Pichia induction/protein expression	once